“DrillTek”

A Borehole Management Application for Diamond Drilling in the Mining Industry

FINAL REPORT

Name: Joshua Smiles
Student Number: 20108881
Lecturer: Dave Drohan

Declaration of authenticity

declare that the work which follows is my own, and that any quotations from any sources (e.g. books, journals, the internet) are clearly identified as such by the use of 'single quotation marks', for shorter excerpt and identified italics for longer quotations. All quotations and paraphrases are accompanied by (date, author) in the text and a fuller citation is the bibliography. I have not submitted the work represented in this report in any other course of study leading to an academic award.

Student.....  ...Date.. 29-03-2026

Work Place Mentor...N/A..... Date .....

Table Of Contents

Table Of Figures

List Of Abbreviations

- **RDBMS** - Relational Database Management System
- **API** - Application Programming Interface
- **SQL** - Structured Query Language
- **JS/TS** - JavaScript/TypeScript
- **UI** - User Interface
- **UX** - User Experience
- **CSV** - Comma-Separated Values
- **JWT** - JSON Web Token
- **DRF** - Django Rest Framework
- **SSR** - Server-Side Rendering
- **PK** - Primary Key
- **SBC** - Single Board Computer
- **IoT** - Internet of Things
- **RPi4** - Raspberry Pi 4

1. Introduction

1.1. Project Background

Diamond drilling is a process in mining whereby drill cores (drillholes) are extracted using a drill rig. These drillholes are then logged and split into samples. These samples analysed in a laboratory, producing assay data (concentrations of metals within the sample). This logging and assay data is used in 3D modelling software to generate a 3D model of the orebody. This 3D model is then used to estimate mining potential, economics and to design the mining of said orebody.

Figure 1 Diamond drillers set up a drill rig to extract drill cores from subsurface. (Shutterstock 2026)

Figure 2 A Geologist at an exploration camp 'logging' geological data using pencil and paper. (Shutterstock 2026b)

These 'drillholes' are often designed using a 3D modelling package, targeting areas of economic interest, often as part of programs containing multiple drillholes. Drillholes are composed of a Collar or 'start' coordinates (X,Y,Z), an Azimuth representing compass direction, a Dip representing angle from horizontal and a Depth representing the Length the hole is to be drilled to.

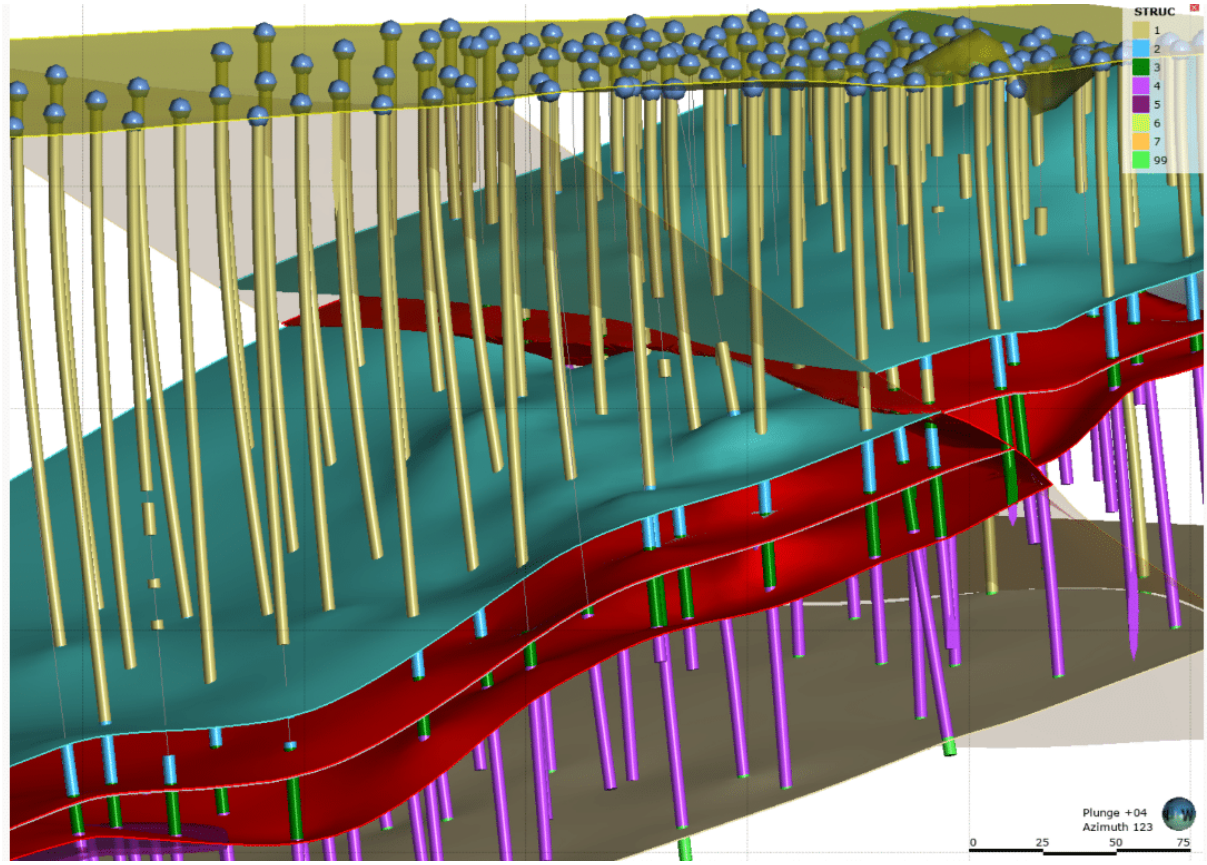


Figure 3 Boreholes plotted using Leapfrog Geo, a popular 3D modelling package in the mining and exploration industries (Seequent 2026). Note that spheres at top of image represent borehole 'collars' and trace orientation defined by 'dip' and 'azimuth

Traditionally, logging would be done using pen and paper, then hand typed into a database GUI like Microsoft Access. Assay results would have been uploaded to a database via transfer of .csv files. There has been a move towards digitising this information over the last 15 years but there are very few complete packages provided. Many service providers only provide part of the systems required around Diamond Drilling and rely on exporting files to import into a clients database. Of software's that allow for input directly into a clients database, these frequently carry a large license fee.

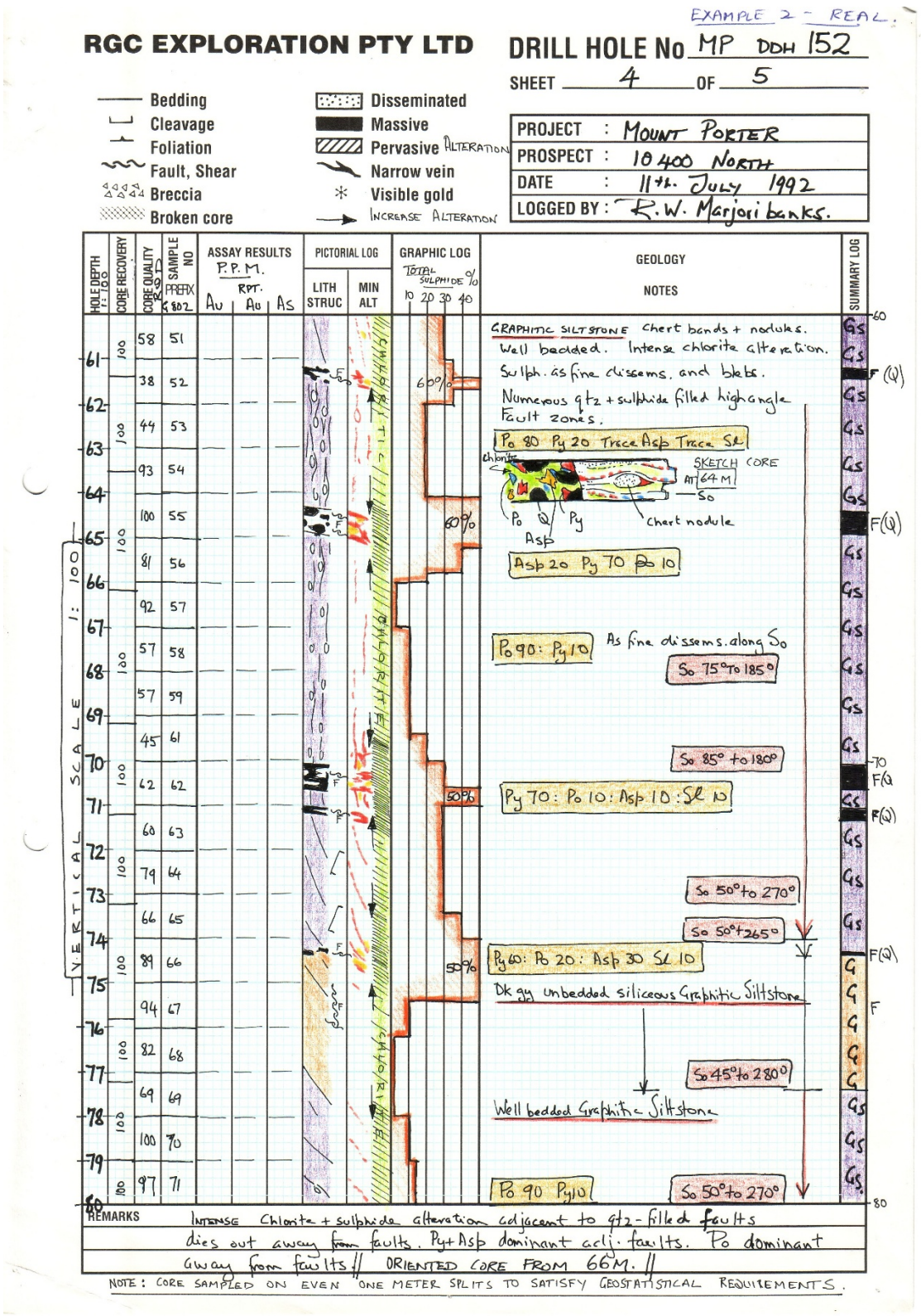


Figure 4 A traditional hand drawn geological log of an exploration drillhole (rogermarjoribanks 2015). Data recorded would need to be transcribed manually into a relational database post drawing.

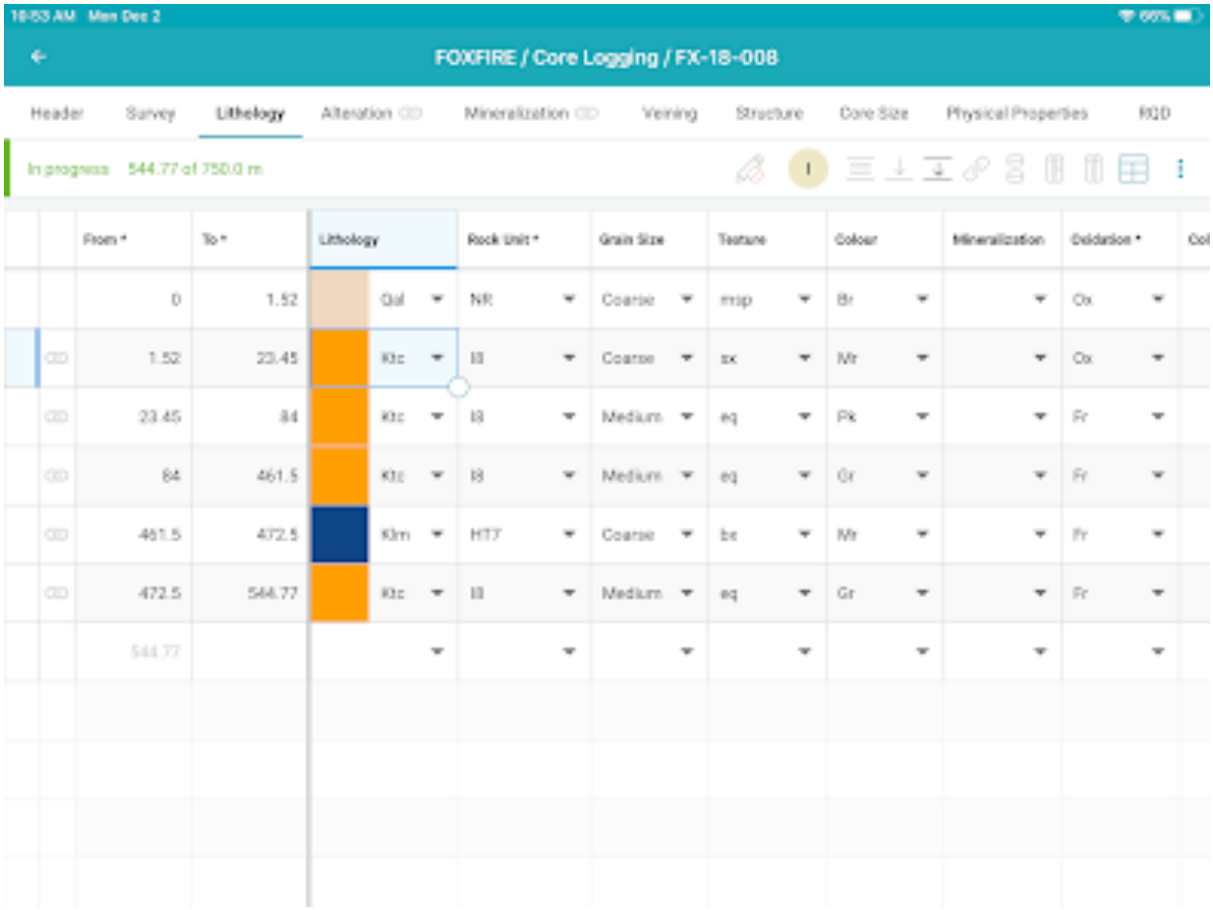


Figure 5 Geological data input directly into MX Deposit (Google Play 2026). MX deposit is a well optimised and popular software that allows direct data input into a borehole database. This does however carry a significant license fee.

On top of this lies a practical problem. Drill rigs require an orientation tool to 'setup' on with the required dip and azimuth. Traditionally drill rigs would have only been able to drill along 2D sections or panels, producing inherently 2D information. With the advent of 3D modelling software and techniques, full 360-degree drilling 'fans' are becoming more commonplace. True-North seeking gyroscopic orientation tools have flooded the market at a high cost, where cheaper alternatives may be possible to develop.

Figure 6 A diamond driller using a Devico alignment tool and tablet to align drill rig position (Devico 2026). Please note the black, handled device on the rig is the gyroscope and the driller is holding a tablet in his hand which interfaces the tool. The system in this image carries a large annual rental fee and cannot be purchased.

1.2. Initial Project Scope

1.2.1. Full stack Diamond drilling application

At a high level, the project was looking to address the issues of disparate information gathering by implementing a full stack JS/TS application connected to an SQL Database. The reason for choosing SQL as a database medium is due to the highly structured nature of drilling data, less suited using NoSQL for storage.

Connection to the SQL database would be managed by an API application using a backend framework chosen during the research phase of the project. This application would contain all the endpoints required for the user to interact with the database from a client application.

The client application would be built using a frontend JS/TS framework, explored in the research phase. This would include a CSS framework in conjunction with the framework to give a professional looking, user friendly experience. This CSS framework was to be component first rather than utility first to encourage rapid development at the expense of some finer grained control over styling.

The application would allow the user to create 'planned' drill programs and populate these with 'planned' drillholes, either through uploading a .csv file or population through an interface. The user would then be able to input logging data on to each drillhole and define samples to be analysed. The application would also contain the ability to produce a sample report to be dispatched to the analysing laboratory containing the details of samples to be analysed. Conversely, the user would also be able to upload returned assay data to be posted to the database via the API application. The user could also amend or delete 'logs', 'drillholes', 'drilling programs.

1.2.2. SBC based Rig Alignment Tool

The issue of rig alignment was to be addressed by attempting to further a project developed in the computer systems and networks modules of the HDip program. Here an RPI/4 with a sense hat, integrated with the IOT platform Blynk was used to create a mobile operated tool that was designed to be mounted on a drill rig. The tool would turn green when it was orientated to the correct dip and azimuth and the user was able to select a button to store the final recording at rest. This system was flawed as the RPI/4 was unable to handle the stream of data well enough to get seamless recording of direction as the machine moved. Blynk was also limited in where it was able to forward recorded data.

To address the weaknesses of the original system this project was to investigate more powerful SBCs that could potentially solve the data stream issues. The project also looked at alternatives to Blynk and Python that may be more suited to the task. To build on this, the device would pull planned drillhole information from the SQL database via the API service to be used as the target orientation (where the Blynk application required the user to input planned details). The system would then be able to post these 'actual' dip and azimuth orientations to a client-side area for approval. Once approved, these orientations could replace the planned information in the database as these represent true measurements.

1.2.3. Security

Diamond drilling information is very sensitive due to its ability to influence investment and economic decisions. Security was a large requirement of this project. Appropriate access control would need to be defined on the SQL database with respect to the two connecting services via the API. Creating users and passwords should not be up to the client user, rather, this should be controlled through an authentication mechanism, only allowing approved users to access the client. There should also be admin only privileges with regards to the SQL database, users should only be able to manipulate appropriate data, not create new tables, other users etc. Certain security considerations will be defined based on the decisions about deployment outlined below.

1.2.4. Other considerations

As well as these general structural considerations, there were also other overarching constraints for the project. The first of these was that the application should be built using platforms and libraries that are entirely open source and do not offer a paid tier. Secondly, the application should use frameworks that lean towards more 'out of the box' functionality rather than overuse of external libraries that might increase the development time whilst on a tight deadline.

1.3. Initial system structure

Based on the constraints above, the initial structure of the system, including all proposed components would look as shown below.

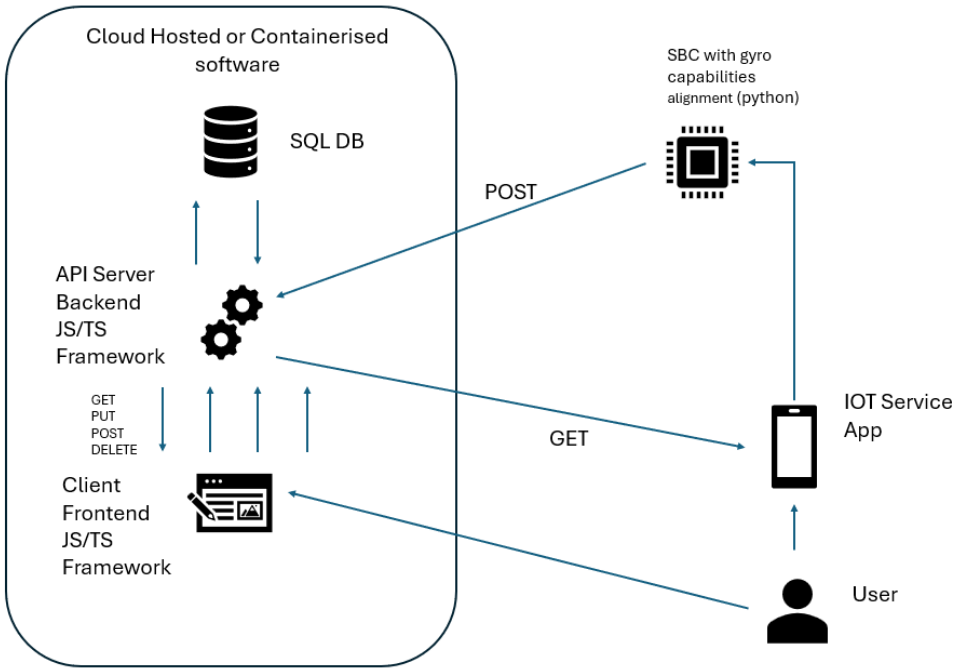


Figure 7 User interacts with SQL DB via Frontend Client App through API Application. User able to operate Gyro Tool using IOT Service app and send results through to client via API.

2. Research

For the purposes of research, the initial application structure was broken down into more distinct components. These included the 'Full Stack' application, comprised of a web frontend, API backend and database ,coined the 'DrillTek web app'. This is joined by the rig alignment solution, comprised of an IOT service app, the physical alignment tool and its underlying software coined 'DrillSight'. Overall, this system was named 'DrillTek'.

2.1. DrillTek web app

As outlined in the initial project scope, the DrillTek web app was to be comprised of a web-based frontend communicating with an SQL database through a backend API service. Its purpose was to allow a user to create drillhole programs, associated drillholes and then input logging information.

2.1.1. Repo analysis

Attempts were made to locate logging app systems uploaded to GitHub, Bitbucket and Dockerhub using Keywords such as 'logging application', 'geologging' and 'borehole'

Research into repos on github were met with very limited success. No repo was found that offered 1:1 functionality akin with what the project was trying to achieve.

Geoscience (Roland Hill, 2025)	offered a plugin for the software 'QGIS' for managing drillholes. This was limited to management and rendering drillholes in QGIS, however. No inspiration could be gleaned from this repo either since it was not relevant to the projects scope.
Bore-Hole-Manager (faraazmalak 2012)	was an incomplete C based program for loading .csv files to generate bore hole traces in a graphical window. This again was not very relevant to the project but did spark the idea to somehow include graphical representations of drillhole traces inside of the approval section in the client-side app related to data generated by the alignment tool.
Dh2Loop (Loop3D 2022)	was a program written in Juypiter and Python for taking csv-based data and standardising it into output csvs to be imported into 3D modelling packages. This was essentially a legacy data cleaning software that gleaned no helpful solutions or inspirations for the project.
SmartDholes (Ieudis 2017)	appears to have been an attempt to create a system like what was proposed using a Django web app. Unfortunately, this appears to have not been completed and the readme is incomplete, not giving a

	clear Idea on what the developer was trying to achieve.
Corelab.py (kevinalexandr19 2025)	appeared to be alternative to proprietary 3D modelling software's, built using python. This would be an alternative destination for the data recorded using the proposed application rather than anything that could be used to gain inspiration for this.
py_desurvey (endarthur 2015)	offered python-based solutions for preparing data to graphically plot surveyed drillhole traces. These repos contained very little of use for the overall scope of the project but had potential to aid with potential visual plotting of drillholes in the client side of this application.
Drill-Hole-Desurveying (WilliamsB39 2023)	offered python-based solutions for preparing data to graphically plot surveyed drillhole traces. These repos contained very little of use for the overall scope of the project but had potential to aid with potential visual plotting of drillholes in the client side of this application.

Figure 8 A table containing repos found on GitHub and observations gleaned from analysis

2.1.2. Market Research

Research was undertaken into potential alternatives readily available on the market and their functionality.

GeoticLog (ALS Global 2026)	Proprietary. Expensive Licenses. Provides live graphical illustration of log, upload of core photography, download of summary data as .csv or paginated report
CoreCAD (Mount Sopris 2026)	Proprietary. Expensive Licenses. Provides live graphical illustration of log, upload of core photography, download of summary data as .csv or paginated report
LogPlot (RockWare 2026)	Proprietary. Expensive Licenses. Provides live graphical illustration of log, upload of

	core photography, download of summary data as .csv or paginated report
TabLogs (Tablogs 2026)	Proprietary. Expensive Licenses. Provides live graphical illustration of log, upload of core photography, download of summary data as .csv or paginated report
MX Deposit (Seequent 2026b)	Proprietary. Expensive Licenses. Provides live graphical illustration of log, upload of core photography, download of summary data as .csv or paginated report
SGS-Geobase	Free, open source. Limited functionality
QGeoloGis	Free, open source. Limited functionality
OpenGeoPlotter	Free, open source. Limited functionality

Figure 9 Table summarising researched products and observations made

Market research showed that most similar software was licensed, however, this did highlight potential features to include in this project.

2.1.3. Database options

The requirement to have a relational database supporting the application immediately ruled out NoSQL solutions like MongoDB.

Opting for a totally open-source database engine that does not include a paid tier also limited options to only a handful of solutions.

MySQL (MySQL 2026), Microsoft SQL Server (Microsoft 2026), Oracle (Oracle 2026) and IBMDB2 (IBM 2026) all offered a paid tier which were against initial specifications so were ruled out.

This left a narrower field of view including PostgreSQL (PostgreSQL 2026), SQLite (SQLite 2026), MariaDB (MariaDB 2026) and Firebird SQL (Firebird 2026). All of these were relational databases that were entirely open source.

2.1.4. Stack options

As a starting point in selecting a technology stack for the DrillTek webapp, Top 10 Tech Stacks for Software Development That Will Rule 2025 (geeksforgeeks 2025) was consulted.

This article highlighted 10 stacks that could be investigated for development of this application. These were:

- MERN - MongoDB - Express - React – Nodejs
- MEAN - MongoDB - Express - Angular – Nodejs

- LAMP - Linux , apache, MySQL , PHP
- Springboot - Java based framework
- PythonDjango stack - Python based - with Django framwork to run this
- Next.js - uses server-side rendering - makes this essentially serverless
- Graphql stack - uses Graph QL
- Flutter - uses Dart OO programming languages
- React Native stack - uses React in conjunction with libraries
- Serverless - uses serverless frameworks

The MERN and MEAN stacks were immediately ruled out since they use MongoDB as the underlying database whereas DrillTek required a relational database.

Springboot was ruled out as a potential solution as this is based on Java and it was decided to stick to browser interpreted languages for development due to development ease.

The LAMP stack was also ruled out. It was not deemed desirable for the application to be dependent solely on a Linux operating system for deployment, rather this would need to be deployable on a variety of operating servers. This was also ruled out since MySQL had already been ruled out as a database solution due to its paid tiers.

Serverless was also ruled out due to the developers lack of experience with serverless architectures and the short development window for the project.

PythonDjango (djangoproject 2026) appeared to be desirable at this stage due to its support for a rest API (critical for backend development in this application) and out of the box security features, containing many of those outlined in the initial project scope.

Ruby-On-Rails (rubyonrails 2026) was encountered outside of the article and was also a prospective solution, particularly for the backend due to its MVC architecture and built in security helpers.

2.1.5. Frontend CSS Frameworks

As outlined in the specification, Utility First frameworks were to be excluded in favour of Component First frameworks, due to the ease of development in a short development timeline. This decision immediately removed Tailwind and Bootstrap as contenders due to their utility-based natures. This left Bulma and Material UI as the logical choices. Material UI would depend on the use of React as the framework for the frontend component of the app, else Bulma would likely be used for styling.

2.1.6. Stack selection

Ultimately there was a vast array of choices when it came to selecting technologies with which to construct the DrillTek web application. It was suggested by Colm Dunphy that it could be advantageous to leverage AI to aid with the selection of an appropriate stack.

Perplexity AI (perplexity 2026) offers deep research capabilities which were Ideal to take and input and draw comparisons between different potential stacks for development of this application.

To achieve this, a rough software requirement specification was written, drawing on tips from Software Requirement Specification (SRS) Format (geeksforgeeks 2023). This specification (appendix 1) was then fed this directly into Perplexity AI's deep research modules.

Based on the time frame for development, ease of development and security considerations among others outlined in the SRS, Perplexity suggested a stack containing

- Sveltekit for Frontend
- Django for backend
- PostgreSQL for database
- Bulma for frontend styling

The full output of this can be found in appendix 2.

The same input was also presented to Google Geminis deep research module (google 2026). This came to the same conclusion except it suggested Bootstrap over Bulma for styling (Appendix 3). This alternative was ignored because Bootstrap is a utility first CSS framework.

2.1.7. Research conclusions

Based on the above research, a technology stack including Svelte for frontend development, Django for development of the backend API, PostgreSQL for the underlying database and Bulma to style the frontend was chosen. It was also decided to include graphical representations of drillholes and drillhole logs in the application as these appeared desirable during repo and market research.

2.2. Drillsight Component

As outlined in the initial specification for the project, the DrillTek system should also contain a physical device for measuring rig alignment that should be controlled via and

IOT platform. These components should communicate with the API backend outlined in the DrillTek Web App.

2.2.1. Physical Device Selection

As stated before, this component of the project was previously attempted using an RPI 4 with a sense-hat during the computer systems and networks module of the Hdip Program. This was only a partial success, but hardware limitations were apparent using the RPI4.

Therefore, re-evaluation was required. Several single board computers were investigated. These were compared on their total cost for the project, their pros and cons and total ram. Their CPUs were also compared using benchmarks on versus.com (versus 2026) as processing power was seen to be the main issue from previous attempts to develop this type of system. The results of these comparisons are shown below.

SBC Name	Pro's	Con's	Total cost * 2	CPU	RAM	Gyro + Accelerometer	CPU Benchmark place	Advised Uses
RPI5 16GB	Powerful , probably enough. X2 ram of last option	Python library available. Probably powerful enough as double the power of the RPI4	240, 400 for starter kit	Broadcom BCM2712 quad-core Arm Cortex A76 processor @ 2.4GHz	16GB	Sense-hat	2	Automation, robotics, IOT, prototyping, projects
RPI5 12GB	Sits in the middle of the 2	Potentially not powerful enough	???	Broadcom BCM2712 quad-core Arm Cortex A76 processor @ 2.4GHz	12GB	Sense-hat	2	Automation, robotics, IOT, prototyping, projects
Nvidia Jetson	Powerful , Maybe too powerful for the	Difficult to configure gyroscope with sensor , little	628 + for this then need two	Quad-core ARM Cortex-A57 MPCore processor	4GB RAM 16GeMMC		1	AI workloads, robotics

	requirement	support available and documentation regarding setting these things up. Possibly out of scope. Very expensive	sensors on top of this					
UNIHIKER M10	Has a screen and built in accelerometer and gyro + python to access	Only 512mb of ram	100	Quad-Core ARM Cortex-A35, up to 1.2GHz	512 mb	Built in	4	IOT, ROBOTICS, STEM EDUCATION
Arduino GIGA R1 WIFI	Can connect a screen. Can connect to IOT Hub	only 1mb of ram. Connects to the pins off the board. Might be awkward practically	200	STM32H747XI dual Cortex®-M7+M4 32bit low power Arm® MCU microcontroller	1mb		NOT BENCHMARKED	IOT, ROBOTICS, 3D PRINTING.

RPI4 4GB	Proven to work	Proven to not work that well Struggles with handling data stream	50	Quad-core ARM Cortex-A72 @ 1.5 or 1.8 GHz	4GB	Sense-hat	3	Automation, robotics, IOT, prototyping, projects

Figure 10 Table containing pro's, con's , specifications and use cases for researched potential devices for the DrillSight system. These were used to decide on an appropriate device to select for development

Based on comparisons, The Nvidia Jetson and RPI5 (15gb + 12gb RAM) were most likely to suit the project specification. The Nvidia Jetson benchmarked highest for CPU power but sat at the highest cost and required an externally provided gyroscope that appeared awkward to install, had no dedicated software API and little documentation. The CPU for the Nvidia Jetson was considered potentially too powerful for what was required. Therefore, the RPI5 16GB was chosen, due to its satisfactory processing power, it's easy to install Sense Hat with a Python API and lower cost.

2.2.2. IOT device client solutions

During the previous attempt at this feature, Blynk IOT was used to control the RPI4 based device installed on a mobile device. This was cumbersome and left few customisation options making it less suitable for the scope of the feature. This was more suited to simpler tasks and passive telemetry.

Several other IOT platforms were investigated as potential alternatives to Blynk IOT as shown below.

Platform	Research Notes
Kaa IOT	Communicates with devices over either MQTT or HTTP No out of the box mobile app for control Can be used to create a mobile app however - this might not be sufficient for this as mobile app not within scope Supports a web dashboard only - not appropriate for the use case Set up more for monitoring telemetry data Does contain wigits for operating the data however

	Does not have a dedicated language API Can generate SMS, Email and webhook notifications - Webhook might be good as this allows for transmission of JSON data - could possibly communicate with a backend directly rather than use RPI to send the data Has an SDK for arduino written in C++ - could be appropriate going with an arduino - possible usecase Not free - cost associated - 100 dollars a month
Thingspeak IOT	Create channels for receiving data. Device writes data to the channel MATLAB used to analyse the data Uses a dashboard to visualise the data - cannot directly control the device - must be done through a talkback app or something else Has no dedicated mobile app - have to use talkback app to communicate with talkback App Uses MATLAB code and a queing system - commands added to the queue in matlab Seems like a bit of a runaround for this type of application Talkback queues commands on thingspeak - then forwards these on to the device connected to thingspeak Thingspeak is just a middleman in this situation Cost associated with rapid data communication - limited to 15 seconds between messages for free - might not be appropriate for this application where a datastream is required
Thingsboard	Allows you to create your own mobile app to be deployed using the flutter SDK Can deploy this on IOS or android Minimal coding effort - ideal for this project as this is not the main focus Is compatible with RPI 5 Uses MQTT to communicate between thingspeak and device

	Uses RPC to communicate with the device from a backend - possibly a mobile app Appears on the service to have minimal actions that a device can do - take photo, make call, use map Seems more appropriate than the top two 10 dollars a month - some cost but low cost
Thinger.io	Has a mobile app for useage Supports web dashboards also Looks relatively simple Main support seems to be for arduino devices - documentation Can connect to any device - device agnostic however Data sent to and from device Data stored in data buckets Dashboard or mobile dashboard used to visualise data Can use the dashboard to send commands to device
Ubidots	Only provides industrial or enterprise pricing - no free Not worth even going down for this

Figure 11 Table containing details on researched IOT platforms that could be used to interface with the selected DrillSight device.

As described in the research notes, no single platform was deemed totally appropriate for the use case for the Drill Sight system desired, due to reasons ranging from limited or cost associated API calls, limited interfaces and lag time between API calls.

This portion of research concluded that using an IOT platform as the client for the RPI5 device would not be the most suitable path for development and deployment.

2.2.3. Research Conclusions

Concluding research, it was decided that the physical device for the Drillsight portion of the application would be the RPI5 16GB with Sense-hat interfaced using the python Sense-hat library. The initial proposal to use an IOT platform to 'control' the device was

abandoned in favour of a Svelte, browser based mobile client styled using Bulma, a mobile friendly, component first CSS framework.

3. Design

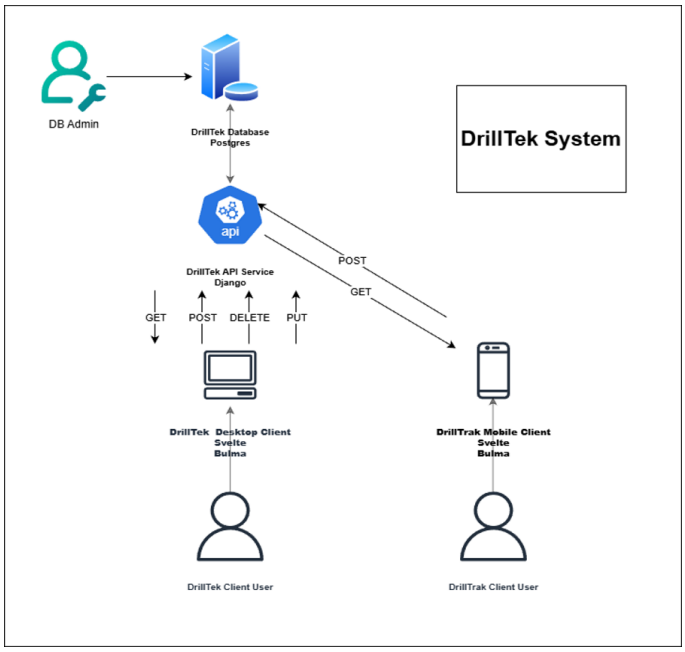


Figure 12 Overall high-level system design for the DrillTek system. Please be advised that since creation, 'DrillTrak' was renamed to 'DrillSight'. Diagram constructed using draw.io (draw.io 2025).

As outlined here the system took 2 distinct shapes. The DrillTek system was to comprise a client system constructed using Svelte and styled with Bulma. This would communicate with an API service constructed using Django, which would communicate with a PostgreSQL database. The DrillSight system was to contain a mobile browser client also constructed with Svelte and Bulma which communicates with the Django API as well as the RPI5 survey device.

3.1. Use Cases

After defining the high-level structure of the application use cases for the application were decided.

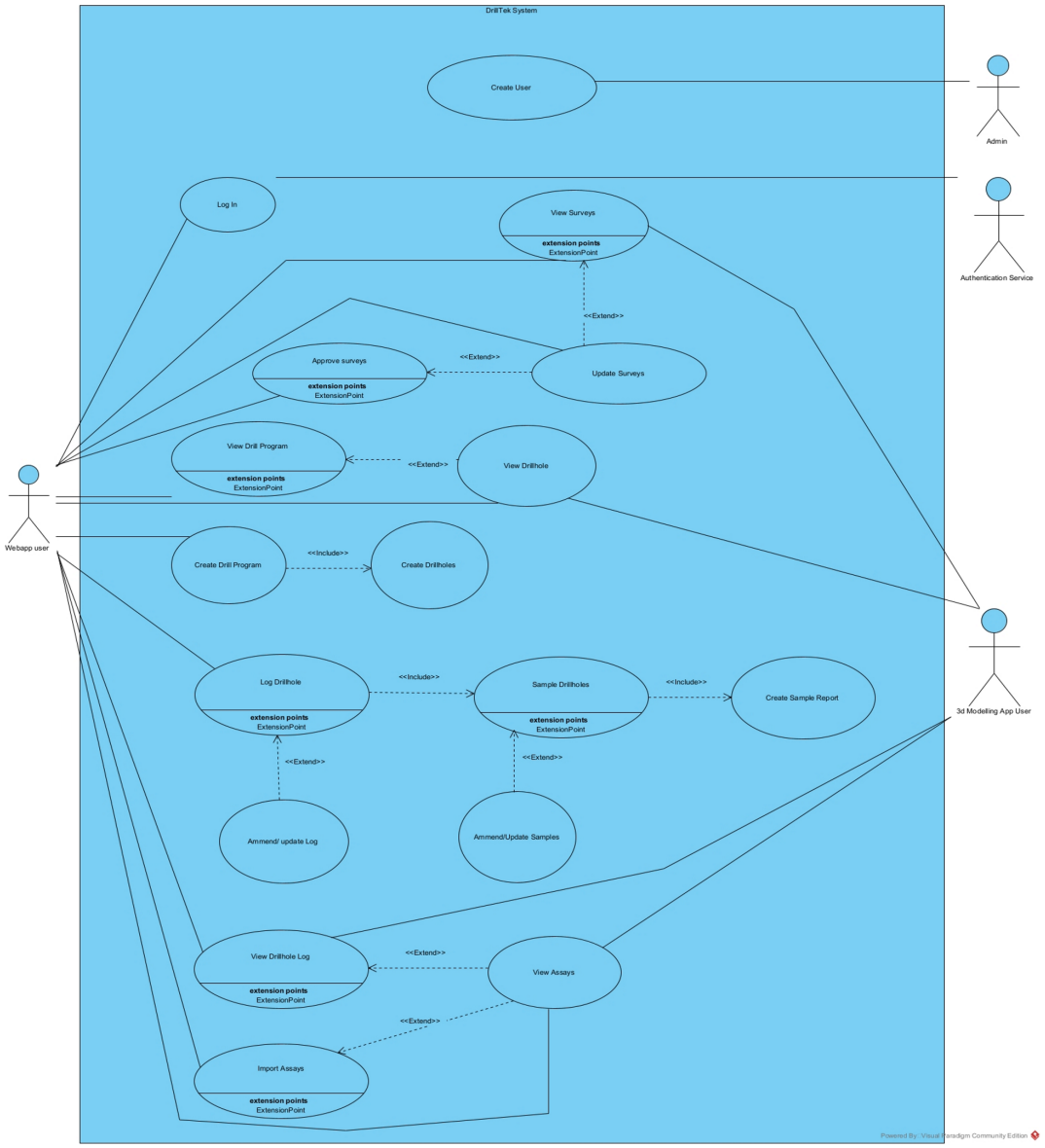


Figure 13 Use case diagram for DrillTek system constructed using visual paradigm.

The above figure defines the fine-grained actions and users for the DrillTek client system. Shown below is the Use Case diagram for the DrillSight system.

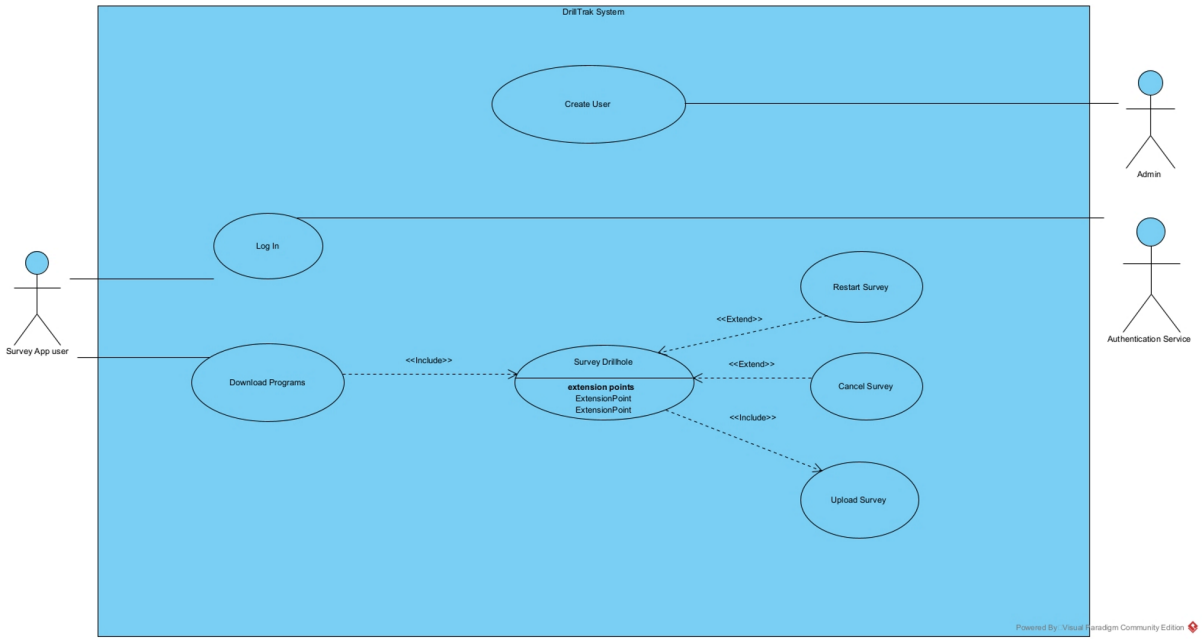


Figure 14 Use case diagram for DrillSight system constructed using visual paradigm.

By defining the use cases for the overall system, it was possible to define the overall flow of both client applications which also hinted at the endpoints required in the Django based API backend and the tables that may be required in the PostgreSQL database.

3.2. Application Client flows

The diagram below models the path a user might take through the DrillTek client application, based on the use cases outlined, to aid in the development of the frontend application.

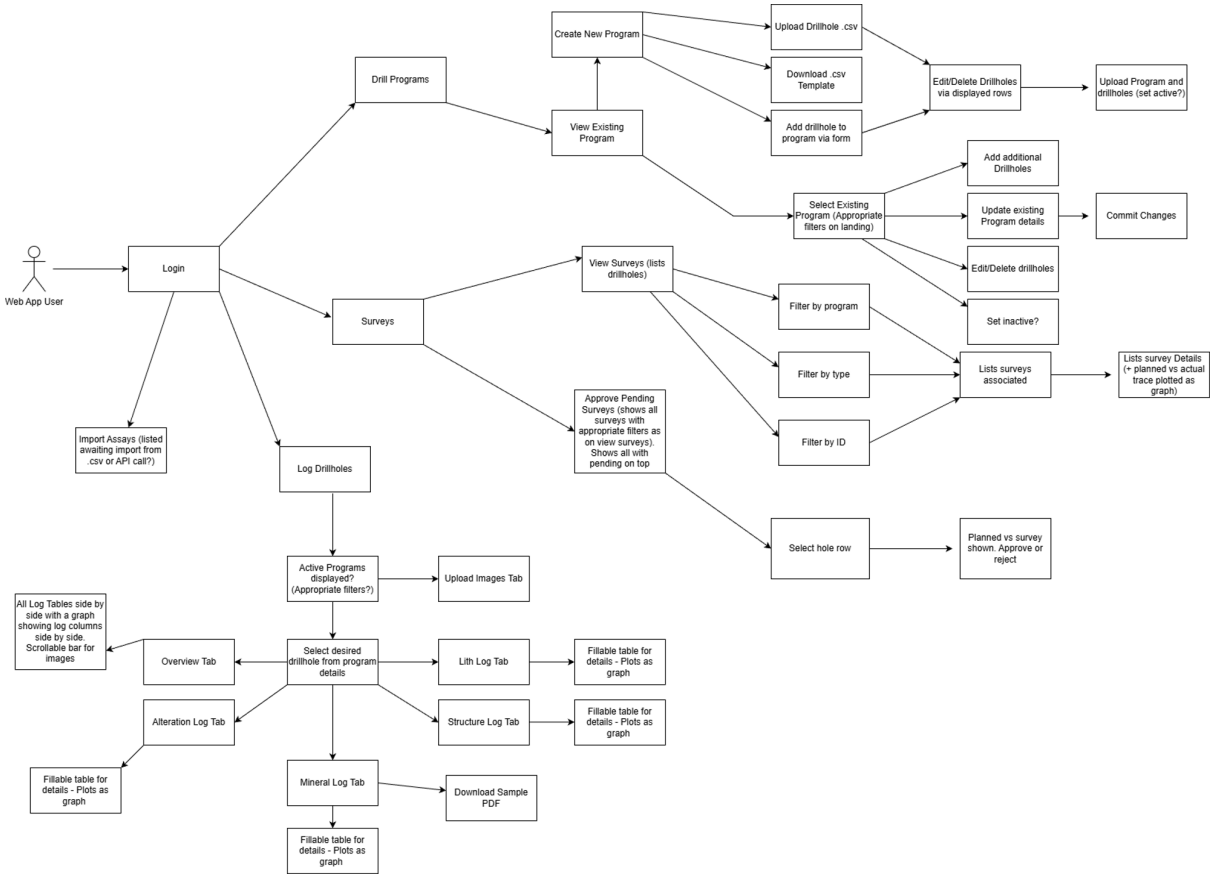


Figure 15 Application flow diagram for DrillTek client application constructed in draw.io

The subsequent diagram shows the path a user of the DrillSight client might take through said application.

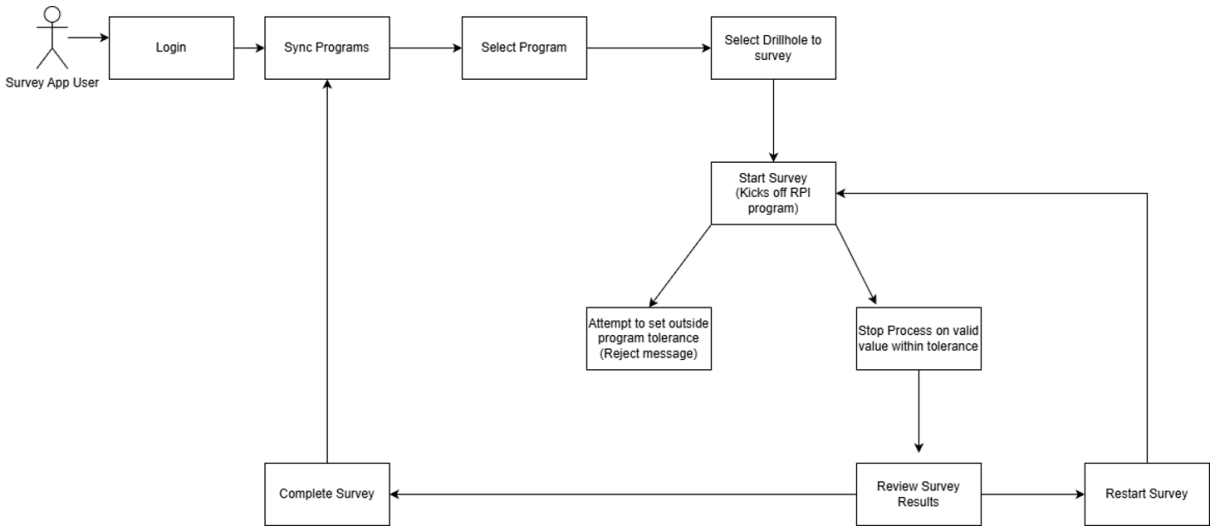


Figure 16 Application flow diagram for DrillSight client application constructed in draw.io

The development of these application flows aided in the definition of user stories followed by screen designs for both clients and how these might interact.

3.3. Database design

Due to the nature of using a relational database, a preliminary design was carried out defining tables and attributes required as well as a set of relations used to store the required data for the application.

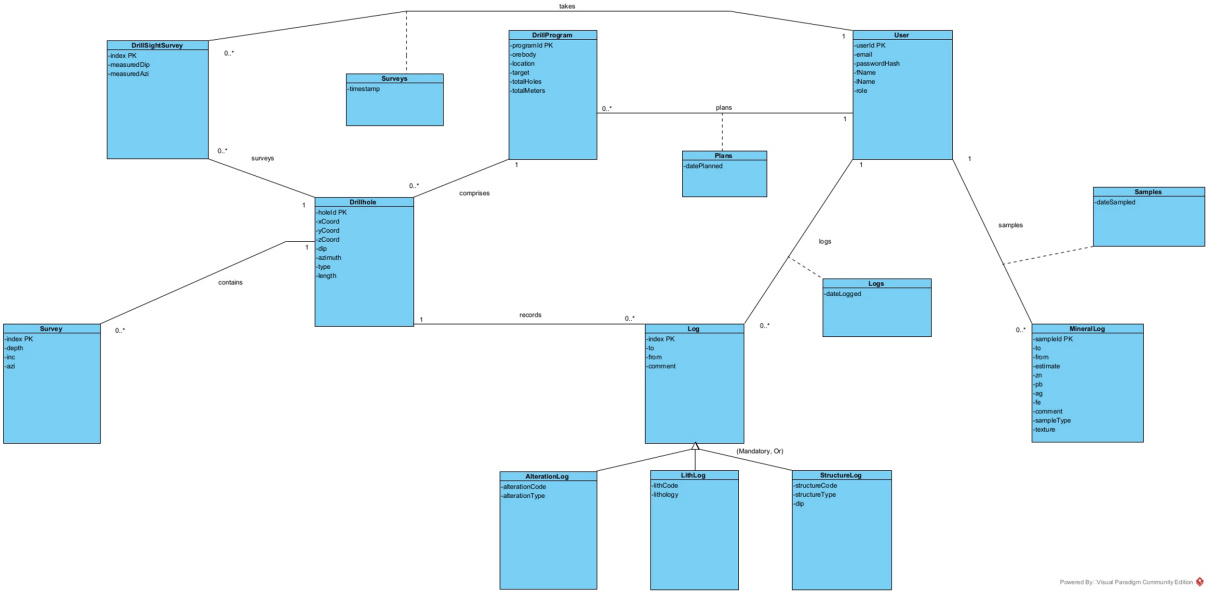


Figure 17 ER Diagram for DrillTek PostgreSQL database constructed using visual paradigm

3.3.1. System relations

For detailed system relations, please see Appendix 27.

3.4. User stories

Based on system designs, 4 separate epics were defined highlighting key large parts of the system. These include the DrillTek Database, DrillSight python (RPI) application, DrillSight mobile web app and DrillTek web app.

Due to potential rescoping issues, the DrillTek web app was then split into user stories. The total list of these stories can be found in appendix 4. These user stories were constructed using IBM best practice (IBM 2026b), with acceptance criteria written using Gherkin syntax.

3.5. Screen Designs

Based on highlighted user stories and the application flow, it was possible to generate screen designs using Figma (figma 2026) for both the DrillTek web client and the DrillSight mobile client. These can be found in appendix 5.

3.6. Late-stage rescoping

Following system design, it was decided, given the length of the project, that the likelihood of being able to complete all aspects of the DrillTek system would be unlikely to be feasible. During the sprint planning phase, it was decided to focus on the DrillTek Web App and DrillTek Database epics. Therefore, project management section will only address producing this part of the system.

4. Project management

4.1. Github

Github was chosen as the vehicle for version control for each component of the application. It was decided early on to have separate repos for the Django API and Svelte Frontend components for development and then combine these into a single repo before project submission

4.2. Sprint management and project tracking

The platform chosen for project management and tracking was Zenhub (Zenhub 2026). Zenhub offers many formats for software project management. In the case of this project, a Scrumban Board template was chosen. This combines the versatility of a Kanban Board with the Scrum methodology of Agile. This board contains a ‘new issues’ bucket which was populated with all user stories (seen in appendix 4) along with any associated screen designs. User stories required to create a minimum viable product were then moved to a ‘Backlog’ bucket and were considered sprint ready. Sprints for the project were defined as 2 weeks long and stories in the backlog bucket were assigned to the first 2 sprints. The sprint schedule was as follows

- Sprint 1 – 19-01-2026 – 02-02-2026
- Sprint 2 – 02-02-2026 – 16-02-2026
- Sprint 3 – 16-02-2026 – 02-03-2026
- Sprint 4 – 02-03-2026 – 16-03-2026
- Sprint 5 – 16-03-2026 – 30-03-2026

Stretch goals were added to an 'Icebox' bucket and all remaining stories remained in the 'New Issues' bucket for assignment at sprint boundaries.

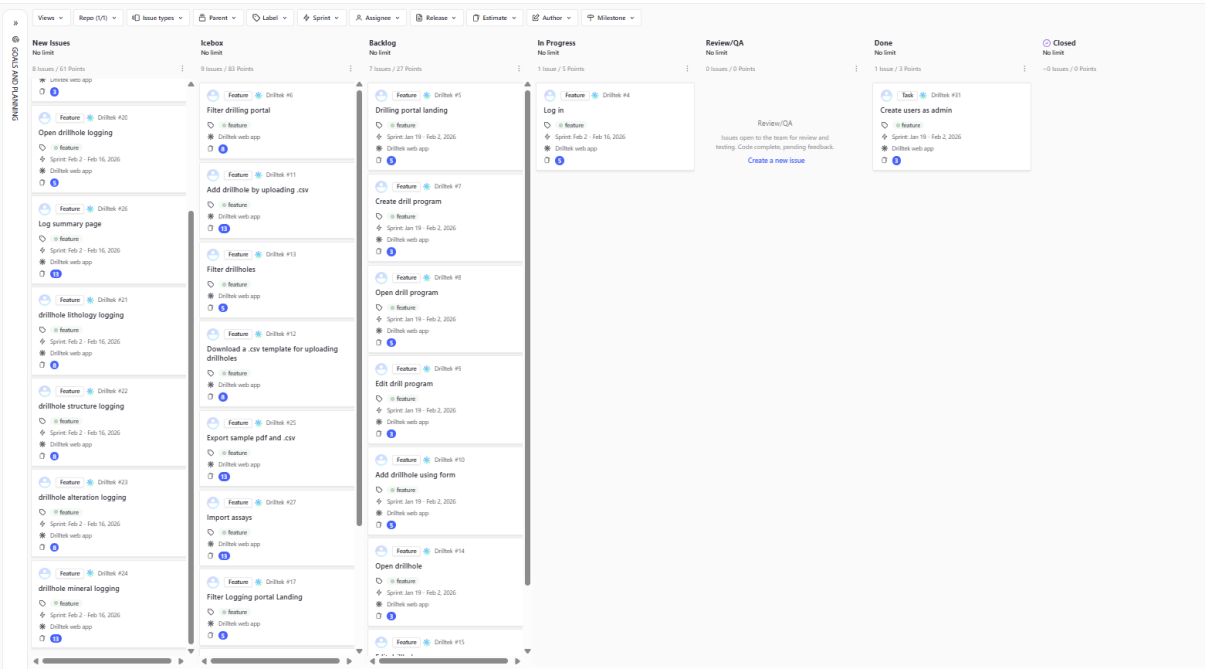


Figure 18 Zenhub based Scrumban board for project.

5. Use of AI

As outlined above AI was used as a tool to select an appropriate technology stack for the project. These conversations can be found in appendices 1-3 (sections 8.1, 8.2 and 8.3). Additionally, Perplexity was used as an alternative in-development research tool during development and aided in debugging errors triggered during development. These conversations are also included in this report, and appendices will be referred to in subsequent sections where referenced. For ease of use, PDF documents have been embedded in this file for reading. If issues accessing files, please contact the author.

6. Implementation

The following sections will outline the implementation phase for the DrillTek web-application. Project diary kept during development has also been attached as appendix 6 (section 8.6) for a day-to-day perspective on development.

6.1. Sprint 1 (19-01-2026 – 02-02-2026)

6.1.1. Project setup

During project setup, separate repos were created for DrillTek Frontend (https://github.com/JoshuaSmiles0/Drilltek-Frontend) and DrillTek Backend (https://github.com/JoshuaSmiles0/Drilltek-Backend) to develop these services separately, with a view to merging them before submission. Each contained a Main branch and a Development branch. In keeping with the Gitflow workflow, it was decided that each 'issue' worked on would be completed on a branch from development and then merged back in (Atlassian 2026). When functionality comprised enough to publish a new version, the development branch was to be merged back into the main branch and a new version/tag published in GitHub.

Subsequently, PostgreSQL (EnterpriseDB 2026), Django (DjangoProject 2026) were installed on the development device. A Svelte project was instantiated using an installation tutorial from the HDip Fullstack 1 Module (SETU 2025)

6.1.2. Story #26 – Create users as admin

It was first pertinent to install the facility for an administrator to create a user with a predefined password as outlined in the project specification. Initially the user table for storing users was created in PgAdmin4 (PostgreSQL RDBMS) with a view to creating users as an admin via this. It was thought at this stage that users would be created by an administrator without a password and that on initial client login the user would be prompted to create one.

On further research, it became clear that Django Rest Framework (DRF) would be the optimal path to follow (Michal Dróżdż 2025) as it would allow tables to be defined using 'Models' which could be configured then pushed to a connected PostgreSQL DB using 'Django Migrations'. Viewsets could then be created for each model, containing API

action methods which could then be linked to specific API URLs. These would also require serializers for use in these actions for converting request JSON into Python objects before committing data to the database or converting Python objects retrieved from the database into JSON format before responding to said API request.

Due to this research, the user table created was scrapped in favour of defining a User model in Django. User creation was still restricted to using RDBMS, however.

To connect to PostgreSQL, a bespoke user was created for the database representing the application and the Psycopg2-binary library was installed to facilitate connection from the DrillTek Backend.

Following connection setup, a tutorial was followed to create a test endpoint to fully understand the mechanics behind DRF (JekayinOluwa Olabemiwo 2021). This endpoint and all other subsequent endpoints were tested using Postman before use in the client application.

A user was also created for testing purposes. User passwords were initially created using plain text, designed to be replaced with a hash of the user chosen password on password change. Although this creates a risk of breach, it was deemed acceptable due to the application being designed to be used inside organisations and not on the public internet.

6.2. Sprint 2 (02-02-2026 – 16-02-2026)

6.2.1. Interim report

Sprint two was entirely dedicated to producing an interim report, highlighting the project specification and RAMP phase of the project.

6.3. Sprint 3 (16-02-2026 – 02-03-2026

6.3.1. Story #1 – Log in

It was highlighted early on that an administrator would create a user for the application using a plain text password. There would need to therefore be a mechanism to force a user to change their password on initial login. To address this a boolean ‘signedUp’ was appended to the user model and the checkUser method created which would identify if the user was signed up or not and send an appropriate response that could be digested by the Svelte client.

This required a mechanism to set the password on this initial login. This method requires the user's email, admin set password and their new desired password. Provided the user exists (email identified) and input matches the set password, the built in Django hasher (default algorithm pbkdf2-sha256) was used to create a fixed length hash which is stored as the user's password value (Appendix 7).

Before attempting to develop a login, it was understood that the application that would contain many endpoints that should be protected from unauthorized access. It was decided that Django Simple JWT would be used for this authentication (Appendix 8). This allows for JSON web token authentication using Access and Refresh tokens. Access tokens are short lived and used to authenticate a user against an API endpoint. Refresh tokens are more long lived and used to generate new access tokens when expired so that the user need not log out to reauthenticate when attempting to access an endpoint.

A login action was created in the backend application that would take a user's email and password, use Djangos hasher to compare the stored hash with the user's input password. If successful, this method would return an Access Token (lifetime 15 mins) and a Refresh Token (lifetime 7 days) for use in further API requests required after login inside the client application.

On the client side, the Svelte application was setup to use largely server-side rendering (SSR). In this methodology, rather than in client-side rendering, API requests are executed on the server then the full HTML page is rendered on the server and passed to the client with the data being accessed through SvelteKits 'Data' prop. Client-side forms then interact with server-side actions to carry out additional function's server-side. In addition to this a 'DrillTek-service' was created to store functions used for calling the backend API.

In this context, a landing page was created with a button to login. Selecting this button would open a modal that was written in-page to display a form to the user to enter their email. This email is then used to execute a checkUser server action which in turn sent a request to the backend to determine if the user is redirected to change their password or to login as they have already done this.

A challenge encountered here was how to pass the email to the next page and prevent the user from having to re-enter their email. This was achieved by parameterising both the set-password and login routes in the client, allowing both routes' actions to access SvelteKit params and extract this user email. Local storage was initially thought of as a potential solution to passing this email but is not accessible inside of a SvelteKit server action, only in a client browser (Appendix 9).

Due to the Svelte client being developed in Typescript, a Session type was created. On login, the returned API response containing the Access Token and Refresh Token is then stored inside a Cookie using SvelteKit.

Using SvelteKit layouts, this Cookie was able to be passed to all subsequent post login pages as a Session if it exists. This allowed for controlled access into the other pages past login as these would only render if a Session existed.

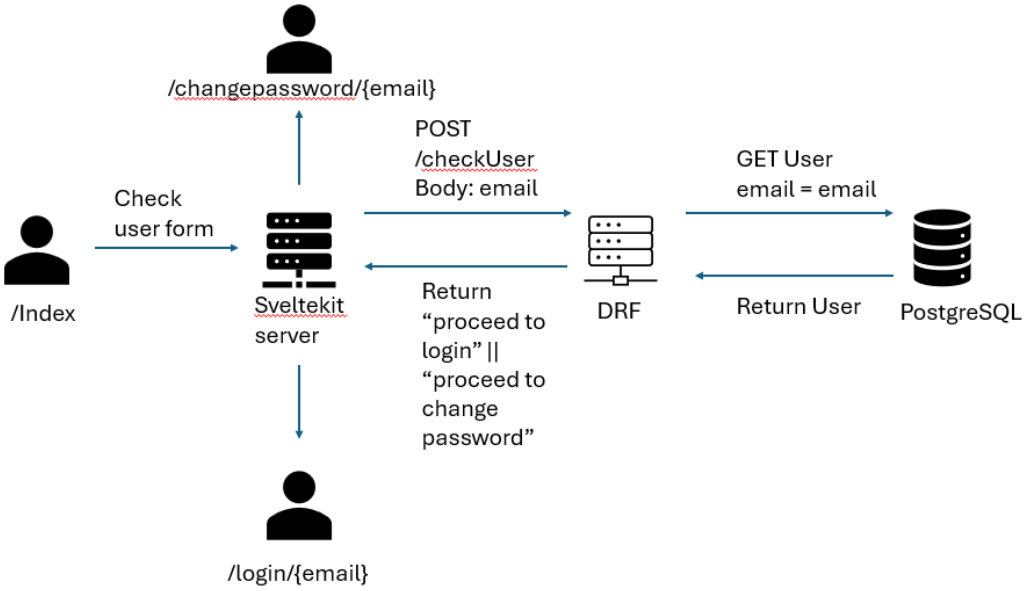


Figure 19 Visual representation of flow. On opening site and selecting 'login', user prompted to enter email into form. Form contents used to post to checkUser endpoint in Django. Django attempts to locate user email in DB. If user not signed up, returns "proceed to change password", if signed up, returns "proceed to login". User redirected to corresponding page by SvelteKit server with email in URL param.

6.4. Sprint 4 (02-03-2026 – 16-03-2026)

6.4.1. Story #2 – Drilling portal landing

The Drilling portal landing required a list of all drilling programs to be displayed to the user, a drillProgram model, serializer and a GET endpoint were created in the Django application. This endpoint, along with all subsequent endpoints was protected by JWT authentication (Appendix 10).

An issue identified at this point is that JWT authentication using the Access Token generated in Django was always failing. It was identified that there was no resolvable Django user_id being generated as a claim on the JWT that could be resolved in authentication causing all requests to protected endpoints to fail. To resolve this, a migration was applied, creating a Django user for the client application. The login action in the user view was modified to generate JWTs for this user and JWT authentication then worked as the required claim existed.

It was also realised at this stage that no mechanism existed to generate a fresh Access Token, so the user would be forced to re-login every 15 minutes. An endpoint was therefore created to take a valid Refresh Token and vend a fresh Access Token.

Axios interceptors were experimented with at this stage to intercept 404 responses from the API and execute a token refresh then retry the original request. This however was unsuccessful as these only work for client side requests and the application is mainly server-side (SSR). To circumvent this, a utility function was created that took a Refresh Token and SvelteKit Cookie as parameter. This was executable on the server and allowed for the refresh to take place, the SvelteKit Cookie to be reset with the new Access Token and for the new session to also be returned if required outside of page load (Appendix 11). The drill program endpoint was added to be called on every page load by embedding it in the layout server load for the application. If this returned a 404 going forward, a refresh was able to take place before the child page load.

Once working, a type was created for drill programs and necessary service methods and were created In the client for displaying the list of drill programs.

6.4.2. Story #4 – Create drill program

The drilling portal landing also required a mechanism for users to create new drill programs. The UI for this was achieved by creating a modal in page with a corresponding form and server action used to pass the drill program to the database via a new API endpoint.

At this stage, it was identified that even though there was a mechanism for gaining fresh Access Tokens on every page load, the same did not exist if a 404 error was encountered during executing a server action which would result in the application failing. To counter this, the server action for creating drill programs was modified to use the refresh utility outlined previously if a 404 was returned by Axios (Appendix 12). This would attempt to refresh the Access Token then retry the original request, logging the user out only if the refresh failed. This pattern was used for all subsequent protected server actions.

6.4.3. Story #5 – Open drill program

In the UI, drill programs would be opened by selecting an ‘open’ tab next to the individual drill program row in the drilling portal landing page. This required an endpoint to retrieve

a drill program by its ID. Initially this ID was passed inside the request body by the client, however, it was identified that DRF get requests ignore the request body so therefore this method was failing. Instead, the ID was passed as a parameter instead (Appendix 13).

Upon opening a program, a table containing its details was displayed, formatted using Bulma (Bulma 2026).

6.4.4. Story #6 – Edit drill program

The page for a specific drill program required a mechanism to edit it. A patch action was created in the backend to facilitate this. Issues were encountered whereby rather than the program being edited, a new program was being created via patch request. It was identified that the arguments were being passed in the wrong order inside the backend view, and this was triggering a 'Create' method in Django rather than an 'Update' method. Upon swapping the order of parameters this was resolved (Appendix 14).

It was also identified that it would not be desirable for users to have to retype all fields, especially if only needing to change one. To do this, an edit program form component was created that required pre-existing details from the page load to be passed as props. These values were then used to prepopulate the form values, allowing the user only to change what they required.

6.4.5. Story #7 – Add drillhole using form

Adding a drillhole to a drill program was achieved with similar UX features as adding a drill program from the drilling portal. This also required a new model and POST endpoint inside Django. Initially, it was thought that the holeid PK on this table should be constructed by incrementing a string. This was investigated using Perplexity (Appendix 15). This method was unnecessarily messy for the scope of the project, so an auto incrementing numeric PK was selected instead.

It was desirable for these drillholes to then be listed on the drill program detail pages with buttons to open their details. This required the creation of an endpoint to retrieve drillholes by programid. The Initial endpoint was flawed by using the same .get() method in its Django view as drill programs. This failed to work when returning multiple objects. This was resolved by using the .filter() method instead (Appendix 16).

Inside the client, many of the database fields for drillholes were floats and integers. Initially after creating the server action to add a drillhole, requests were failing due to

conflicting datatypes. It was realised that all form data is passed to server actions in string form (w3schools 2026). This required converting these fields to the necessary data type using parseFloat () before passing them to the API (w3schools 2026b).

The ‘drillhole type’ attribute required a dropdown style list in the add drillhole form. This was styled using Bulma Select (Bulma 2026b).

6.4.6. Story #11 – Open drillhole

Each drillhole listed in the associated drill program details page required a button to open a parameterised route to direct the user to the desired drillhole. This required a Django action to retrieve a drillhole by its holeid.

Upon plotting up the table inside the UI of the drillhole details page, it was realised that the userid for the drillhole creator was the only detail available to display from the loaded page data. Therefore, another protected API route was created to retrieve a user email by userid. This then allowed for the method to be executed on page load, allowing the users email to be displayed in the hole details table rather than their userid.

This additional API call also created generated typescript errors as there was the potential for the call to executed before the drillhole had been loaded and therefore return unknown. Logic was entered to ensure that the user property exists on the loaded drillhole object before the API call to retrieve the user email was executed (Appendix 17).

6.4.7. Story #12 – Edit drillhole

Functionality was developed in an identical manner to edit drill program story. This was, however, the final story for the drilling portal aspect of the application so upon completion Version 1.0.0 was released upon merger of the Development branch with the Main branch.

6.4.8. Stories #18, #19, #20, #21 – Drillhole logging

Initially it was thought that these stories would be worked on separately with the view that they would represent separate, also server rendered pages as seen with the rest of the application.

The screen designs for this area of the application suggested that a client driven approach would be best for responsivity, as the user should be able to seamlessly switch between individual log tabs. As with all data tables, models were created for Lithology logs, alteration logs, structure logs and mineral logs.

Initially, GET endpoints were created for these. A table component was created, taking retrieved data from page load as props.

A hero ribbon was created with buttons representing each 'tab'. Boolean states were created representing each of these tabs being 'active'. This state was then used to control visibility of components related to each tab, initially, each just showing a scrollable (Geeksforgeeks 2025b) tabular (Bulma 2026) representation of what was already committed to the database. This state would also control which option on the hero was highlighted.

It was initially hoped to add a graphical representation of each 'log' alongside each input and another showing all logs summarised on the overview tab.

There was a requirement for alteration, structural and mineral logs to be able to plot as stacked bars with blank space possible between bars. Layerchart (NPM 2026), Svelte UX Graphs (svelteUX 2026), Frappe-charts (Frappe 2026) and Apache E-charts (echarts 2026) were all analysed for this potential but could not be configured to do so (Appendix 18).

Gantt charts were looked at as an alternative format as offered by Frappe-Gantt (Frappe 2026b). These however are horizontal and designed for scheduling purposes. It was therefore not deemed suitable for this purpose.

Frappe-charts (Frappe 2026) were chosen to display a stacked bar chart of lithology (due to lack of gaps in this log type) on the overview page only.

The subsequent challenge was to replace the tables (Appendix 20) on the lithology, alteration, structure and mineral tabs with input forms. A logform component was created to achieve this. This logform takes a log array, log type and logdelete function as props. The logform is bindable. Each field is therefore bound to the parent state, updating it as the user enters information. The log type controls the conditional structure of the form, rendering slightly different form fields depending on the log type.

The initial state of this component is the data returned on page load or, if this does not exist, a blank record to ensure that a row always populates for user input (Appendix 19).

An add button was appended to each of these. On add, a new blank record is added to the corresponding logform state (MDN 2026). This causes the form list to re-render showing a new row for the user to fill in.

Each record contains a button associated with the corresponding delete method for that log. This replaces the corresponding log state with a new array with the record corresponding to the delete filtered out.

Each of these forms only interacts with the local state. This does not survive page load. It was initially thought to have a single button in the UI to upload all logs. This was decided against as a user might want to only commit one log and return later to log subsequent features. Therefore, each 'tab' has its own save button.

This save function was to be executed on the server as an action as with the other parts of the application. Since a user could edit existing records as well as add new ones at any time, it was decided that it would be best to delete all records before upload and reupload all fresh records. Therefore, an API endpoint was created in the backend to facilitate this. An API endpoint was also created for uploading multiple logs of each type. These forms are not conventional form objects. To execute the associated server actions, it was required to create a new form data object, append the associated log state, then use Axios to call the server action with the form data attached as server actions in SvelteKit are triggered by form submissions. On the server side, the delete log by program id function is first called to clear existing records from the database. Following this, the new log data was uploaded using the form data passed to the server action.

This server action required layering in opportunities for token refreshes for both associated API calls in case access tokens expired during the process.

Following completion of this issue v2.0.0 of both the back and frontend applications were published.

6.5. Sprint 5 (16-03-2026 – 30-03-2026)

During development through sprints 1-4, new issues were identified that would improve the useability and codebase of the application. These new issues can be found in appendix 21. These became the focus of sprint 5.

6.5.1. Story #29 – Blacklist Refresh Tokens on logout

It was identified during development that despite the cookie containing the Refresh and Access Tokens being destroyed on logout, the Refresh Token remains active. This would mean that if an attacker were to obtain a Refresh Token, they could generate Access Tokens for up to a week (until the Refresh Token expires naturally). Therefore, it was a

natural goal to create a mechanism to blacklist Refresh tokens when a user logs out. This was achieved by using Simple JWT Blacklist, an addon for simple JWT (Appendix 22). A logout action was created, taking a refresh token in the request body and using the library to blacklist it. Following blacklisting, a Refresh Token can no longer be used to obtain an Access Token using Simple JWT. A call to this endpoint was also added to logout in the client application to complete the feature.

6.5.2. Story #28 – Minimum password length

Up to this point in development, there was no restriction on password length. Current NIST guidance advises a password to be minimum 15 characters in length (NIST 2025). Due to passwords being stored as fixed, 128-character hashes, this could not be controlled at the model level. Therefore, additional validation was added to the setPassword Django action which only allows the change if the entered new password is >= 15 characters. In addition to this, the user form component was refactored to contain a state bound to the new password input field. Depending on the length of the values current state, defined by user input, graphics representing weak, moderate and strong passwords render at the base of the form. A prop had to be passed to this component defining if it was a reset password or login form as it was not desirable to render the graphics on the login page.

6.5.3. Story #33 – Make modal reusable component for any page that it's on

Modals were extensively used in the application leading to a lot of repetition of code between pages. To counter this, a modal component was created. This takes a Boolean parent state as a prop for the modal buttons to bind to for toggling and a type to define what contents are rendered, depending on the page.

6.5.4. Story #34 – Make lists a reusable component for any page they are on

As well as modals, lists (such as drillholes and programs) were extensively used and extensively repeated on many pages. A reusable component was also generated for these, taking data to list, a subtype and a type to conditionally render different list types

depending on the page. Completion of this component resulted in the release of backend and frontend versions v2.1.0.

6.5.5. Stories #3, #10, #14, #16 – Filtering drillholes and drill programs

It was decided that filtering drillholes and programs in both the logging and drilling portal would be appropriate to do using a search bar, as ID's are the only truly filterable attributes of both tables in the project scope.

To achieve this, a search bar component was constructed for all pages containing lists of drillholes or programs (Appendix 23). This search bar takes a bindable parent state, a clear function and a type as prop. The type prop is required to conditionally render two different inputs, as the IDs for drillholes and programs are different data types (string and number). Search state is set inside the parent pages.

A new state was created to hold the data from the page load in the parent pages. This state is derived from the search state, meaning that as the search changes on user input, the state of the data array changes by filtering the data array down to those with IDs containing the search value.

All list components were adjusted to use this state rather than page load data, resulting in dynamic rendering of list items depending on search bar input.

6.5.6. Story #9 - Download a .csv template for uploading drillholes

A .csv file was designed with headers corresponding to those required for uploading a drillhole and was added to the /static/files folder in the frontend project. An anchor tag pointing to this folder with the download property and styled as a button was added to the drill program page (w3schools 2026c). On selection, this .csv downloads to the user.

6.5.7. Story #8 - Add drillhole by uploading .csv

This application was designed to upload multiple drillholes to a drill program at once. Firstly, an additional endpoint was added to the backend to receive multiple records.

A form was created in the frontend inside of the modal component with a multipart ENCTYPE (w3schools 2026d) and a file input field set to accept csv files (MDN 2026b).

Papaparse was recommended by perplexity as a suitable parser for converting CSV data to Javascript objects (Papaparse 2026).

A server action was created for this page. This server action retrieves the .csv file from the form data, extracts the array buffer bytes (MDN 2026c) and decodes them using a text decoder (MDN 2026d).

The decoded csv text string is then parsed using papaparse.parse (Appendix 24) as an array of drillholes. Programid and userid are appended to each object on the array and then uploaded via the API endpoint mentioned previously.

6.5.8. Story #35 - Download logs, holes as .csv

As outlined in the story, it was advantageous for users to be able to download logs from the overview page in .csv format.

Due to the already client heavy nature of the drillhole logging page, it was decided to handle this process within the client using buttons and client-side functions.

These associated functions create a new variable from the loaded page data for the desired log, convert this data into a .csv string using papa.unparse. A new csv blob object is then created from the .csv string (MDN 2026e). An object URL is created for the blob, acting as a pointer (MDN 2026f). A common practice for triggering the physical file download is to create a false anchor tag with the blob URL and download attribute and then artificially click it using the DOM API (Appendix 25). This was implemented in this case to achieve download to the user. On completing this feature Version 2.2.0 was released.

6.5.9. Story #27 - Tighter error message display

It was noted that much of the server-side error handling would redirect users to the logout route or nothing would happen or change if something was done incorrectly in the application.

To counter this, SvelteKit errors were leveraged (svelte 2026b). All routes using SSR were given a +error.svelte page. All redirects were replaced with throwing of SvelteKit errors. These errors force rendering of the closest error page. Error props were passed alongside the thrown errors to display a message (Bulma 2026c) to the user and offer a route back to the page to retry. On completing this feature, Version 2.2.1 was released.

6.5.10. Story #37 – Docker ready drilltek web

This application is not yet production ready, however, to allow ease of access to other developers to test it, it was desirable to have the codebase deployable to a development environment.

Docker compose was chosen to address this (Appendix 26). This task proved difficult due to the developers lack of experience with docker compose.

Firstly, a single mono-repo was created by combining both repos with history (https://github.com/JoshuaSmiles0/DrillTek).

Docker files were created for each component application, providing the blueprint for their subsequent container construction.

A docker-compose.yml file was then constructed to provide instructions for bringing the application online with containerised components.

Difficulties arose in several areas.

Firstly, the migrations in the Django app were not running by default. To address this, a migration task was created to be run before the Django app was started.

Secondly, the migration task began to run before the PostgreSQL DB container had started resulting in the entire application failing to start. To address this a health check was applied to the DB container, and the migration only starts when this is considered 'healthy'.

Lastly, data was not persisting once containers were destroyed. To address this, a volume was attached to the PostgreSQL container, used on container creation, persisting container creation and destruction.

7. Conclusions

7.1. Project takeaways

The project has shown that a user friendly Drillhole management and logging application can be created using fully open-source tools. Django makes It easy to get an integrated database up and running and provides advantageous out of the box security tools.

Despite this, the project is still in a development phase. Careful consideration would need to be given on how to deploy this. Docker compose is useful as a limited orchestration tool, but this application would likely need to be deployed on an on-premises web server, separated from the public internet. Consideration would also need to be given to how 3D modelling software's would access the database. Many of these rely upon an ODBC link to access this data, with few able to use the Django endpoints that have been or could be created to support this.

7.2. Learnings

Development of this project involved learning a new framework (Django Rest Framework) not encountered before. This allowed for programming a web backend using python which was also not previously encountered. Developing the frontend gave greater confidence in developing reactive, client-heavy pages rather than just sticking purely to server rendered pages, which the developer is now more comfortable with. Dealing with file upload and download was also new to the developer and was an interesting experience and valuable skill to learn to learn due to its pervasiveness in extractive industry systems. The limited learning around docker compose was also a valuable experience, as containerisation Is also an area of interest to the developer.

7.3. Future work

7.3.1. DrillSight System

As noted in the original project specification, there was to be a semi physical component to this project covering rig alignment. It would be desirable to take the project forward to address this issue in full.

7.3.2. DrillTek Web App

Several features related to the web application were left incomplete in the icebox on the project Scrumban Board.

Story #31 - RBAC to sections of site would be key to complete as in most operations junior members of staff are often only responsible for logging and not to be given access to drill plans.

Story #22 - Export sample pdf and .csv would be required to generate data required for offsite assaying labs to be aware of samples to process.

Story #24 – Import Assays would need to be completed as there would have to be a vehicle for passing in returned assays after lab analysis. If this problem was addressed, it would also be critical to prevent editing of mineral logs after import, as editing any log results first in its deletion, which would also delete the returned results from the same table.

Story #25 – Page Pagination could clear up the user interface and allow the user to choose how much data they want to display on the page at one time. This would be implementable with list type pages such as the drilling and logging portals.

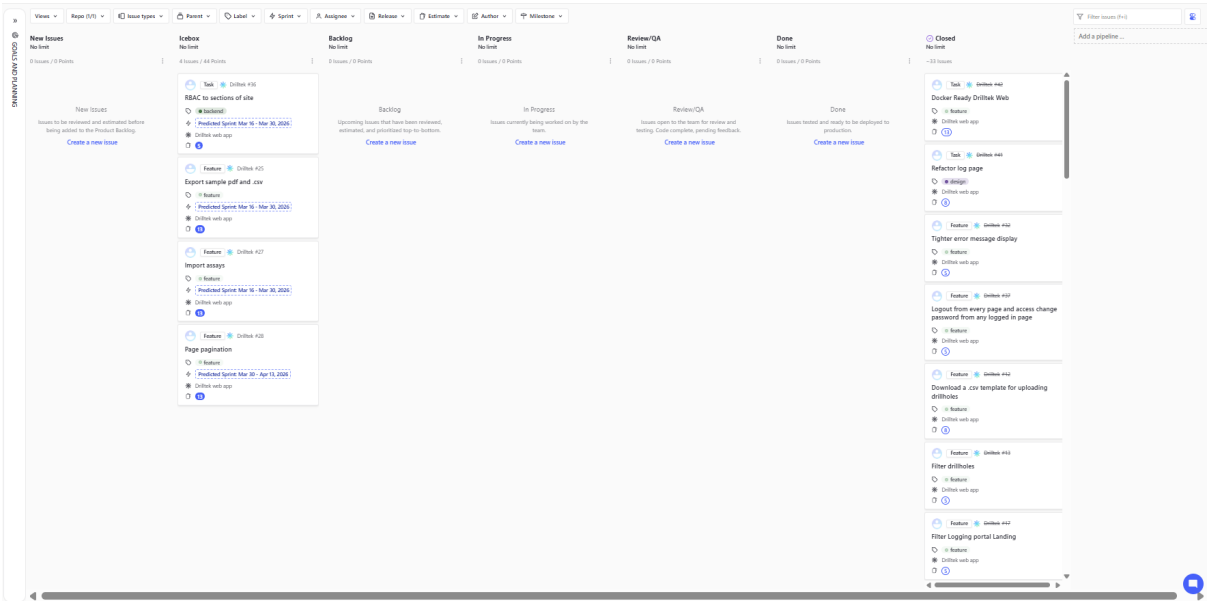


Figure 20 Final status of Scrumban board at end of project. Note all outlined user stories are closed due to being included in full released versions of the application. All that remain are the features outlined above.

8. References

ArrayBuffer (2026) *ArrayBuffer*. Available at: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/ArrayBuffer (Accessed 21-03-2026)

Array.push() (2026) *Array.push()*. Available at: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/push (Accessed 15-03-2026)

Atlassian (2026) *Git-flow Workflow*. Available at: https://www.atlassian.com/git/tutorials/comparing-workflows/gitflow-workflow (Accessed 28-03-2026)

ALS Global (2026) *GeoticLog*. Available at: https://www.alsglobal.com/en/geoanalytics/data-generation/geological-data-collection-and-generation/geoticlog (Accessed 14-02-2026)

Bulma (2026) *Table*. Available at: https://bulma.io/documentation/elements/table/ (Accessed 05-03-2026)

Bulma (2026b) *Select*. Available at: https://bulma.io/documentation/form/select/ (Accessed 07-03-2026)

Bulma (2026c) *Message*. Available at: https://bulma.io/documentation/components/message/#docsNav (Accessed 22-03-2026)

ChablaChanda (2024) *create-drillhole-synthetic-data*. Available at: https://github.com/ChabalaChanda/create-drillhole-synthetic-data (Accessed 14-02-2026)

Devico (2026) *Devialigner*. Available at: https://www.devico.com/product/devialigner/ (Accessed 28-03-2026)

DjangoProject (2026) *Django FAQ*. Available at: https://docs.djangoproject.com/en/6.0/faq/ (Accessed 19-01-2026)

djangoproject (2026) *PythonDjango*. Available at: https://www.djangoproject.com/ (Accessed 14-02-2026)

draw.io (2025) *draw.io*. Available at: https://app.diagrams.net/ (Accessed 14-02-2026)

Echarts (2026) *Apache e-charts*. Available at: https://echarts.apache.org/handbook/en/how-to/custom-series (Accessed 14-03-2026)

EnterpriseDB (2026) *Postgresql Installation*. Available at: https://www.enterprisedb.com/postgresql-tutorial-resources-training-1?uuid=867f9c7f-7be7-44ed-b03f-103a0a430d51&campaignId=postgres_rc_18 (Accessed 19-01-2026)

Faraazmalak (2012) *Bore-Hole-Manager*. Available at: https://github.com/faraazmalak/Bore-Hole-Manager/tree/master/Source (Accessed 14-02-2026)

Figma (2026) *figma*. Available at: https://www.figma.com/ (Accessed 14-02-2026)

Firebird (2026) *Firebird sql*. Available at: https://www.firebirdsql.org/ (Accessed 14-02-2026)

Frappe (2026) *Frappe-Charts*. Available at: https://frappe.io/charts/docs (Accessed 14-03-2026)

Frappe (2026b) *Frappe-gantt*. Available at: https://github.com/frappe/gantt (Accessed 14-03-2026)

Geeksforgeeks (2023) *Software Requirement Specification (SRS) Format*. Available at: https://www.geeksforgeeks.org/software-engineering/software-requirement-specification-srs-format/ (Accessed 14-02-2026)

Geeksforgeeks (2025) *Top 10 Tech Stacks for Software Development That Will Rule 2025*. Available at: https://www.geeksforgeeks.org/blogs/top-tech-stacks-for-software-development-that-will-rule/ (Accessed 14-02-2026)

Geeksforgeeks (2025b) *How to create Table with 100% width, with Vertical Scroll inside Table body in HTML?*. Available at: https://www.geeksforgeeks.org/html/how-to-create-table-with-100-width-with-vertical-scroll-inside-table-body-in-html/ (Accessed 14-03-2026)

Google (2026) *Google Geminis deep research module*. Available at: https://gemini.google.com/ (Accessed 14-02-2026)

Google Play (2026) *Mxdeposit*. Available at: https://play.google.com/store/apps/details?id=net.minalytix.mxdeposit (Accessed 28-03-2026)

IBM (2026) *IBMDB2*. Available at: https://www.ibm.com/products/db2 (Accessed 14-02-2026)

IBM (2026b) *IBM best practice*. Available at: https://www.ibm.com/docs/en/targetprocess/atp/saas?topic=model-user-stories (Accessed 14-02-2026)

Ieudis (2017) *SmartDholes*. Available at: https://github.com/opengeostat/SmartDHOLES/blob/master/smart_drillholes_gui/smart_drillholes (Accessed 14-02-2026)

JekayinOluwa Olabemiwo (2021) *Django Restful API + PostgreSQL*. Available at: https://dev.to/entuziaz/django-rest-framework-with-postgresql-a-crud-tutorial-1l34 (Accessed 28-01-2026)

Kevinalexandr19 (2025) *corelab.py*. Available at: https://github.com/kevinalexandr19/corelab?tab=readme-ov-file (Accessed 14-02-2026)

kckuei (2024) *lithology*. Available at: https://github.com/kckuei/lithology (Accessed 14-02-2026)

LayerChart (2026) *LayerChart*. Available at: https://www.npmjs.com/package/layerchart (Accessed 14-03-2026)

Loop3D (2022) *Dh2Loop*. Available at: https://github.com/Loop3D/dh2loop (Accessed 14-02-2026)

MariaDB (2026) *MariaDB*. Available at: https://mariadb.org/ (Accessed 14-02-2026)

MDN (2026) *Array.push()*. Available at: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/push (Accessed 15-03-2026)

MDN (2026b) *<input type="file">*. Available at: https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/input/file (Accessed 21-03-2026)

MDN (2026c) *ArrayBuffer*. Available at: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/ArrayBuffer (Accessed 21-03-2026)

MDN (2026d) *TextDecoder*. Available at: https://developer.mozilla.org/en-US/docs/Web/API/TextDecoder (Accessed 21-03-2026)

MDN (2026e) *Blob*. Available at: https://developer.mozilla.org/en-US/docs/Web/API/Blob (Accessed 21-03-2026)

MDN (2026f) *URL: createObjectURL() static method*. Available at: https://developer.mozilla.org/en-US/docs/Web/API/URL/createObjectURL_static (Accessed 21-03-2026)

Michal Dróżdż (2025) *Setting up a django api with django rest framework drf a beginners guide*. Available at: https://medium.com/@michal.drozdze/setting-up-a-django-api-with-django-rest-framework-drf-a-beginners-guide-cee5d61f00a6 (Accessed 26-01-2026)

Microsoft (2026) *Microsoft SQL Server*. Available at: https://www.microsoft.com/en-ie/sql-server (Accessed 14-02-2026)

Mount Sopris (2026) *CoreCAD*. Available at: https://mountsopris.com/wellcad/core-logging-software/ (Accessed 14-02-2026)

MySQL (2026) *MySQL*. Available at: https://www.mysql.com/products/enterprise/ (Accessed 14-02-2026)

NIST (2025) *How Do I Create a Good Password?*. Available at: https://www.nist.gov/cybersecurity/how-do-i-create-good-password (Accessed 29-03-2026)

NPM (2026) *LayerChart*. Available at: https://www.npmjs.com/package/layerchart (Accessed 14-03-2026)

Oracle (2026) *Oracle*. Available at: https://www.oracle.com/europe/ (Accessed 14-02-2026)

Papaparse (2026) *Papaparse docs*. Available at: https://www.papaparse.com/docs#config (Accessed 21-03-2026)

Perplexity (2026) *Perplexity AI*. Available at: https://www.perplexity.ai/ (Accessed 14-02-2026)

Postgresql (2026) *Postgresql*. Available at: https://www.postgresql.org/ (Accessed 14-02-2026)

RockWare (2026) *LogPlot*. Available at: https://www.rockware.com/logplot-in-the-environmental-and-geotechnical-industries/ (Accessed 14-02-2026)

rogermarjoribanks (2015) *Diamond-drill-core-log-sheets*. Available at: http://rogermarjoribanks.info/diamond-drill-core-log-sheets/ (Accessed 28-03-2026)

Rowland Hill (2025) *Geoscience*. Available at: https://github.com/rolandhill/geoscience (Accessed 14-02-2026)

rubyonrails (2026) *Ruby-On-Rails*. Available at: https://rubyonrails.org/ (Accessed 14-02-2026)

Seequent (2026) *Experience-40-70 reductions in drillhole loading with leapfrog*. Available at: https://www.seequent.com/experience-40-70-reductions-in-drillhole-loading-with-leapfrog/ (Accessed 28-03-2026)

Seequent (2026) *MX Deposit*. Available at: https://www.seequent.com/products-solutions/mx-deposit/ (Accessed 14-02-2026)

SETU (2025) *Lab-19a-donation-svelte-01-shell*. Available at: https://tutors.dev/lab/wit-hdip-comp-sci-2024-full-stack-1/unit-6-sveltekit/topic-19-svelte-routing/side-unit/book-2-donation-svelte-1/01 (Accessed 19-01-2026)

Shutterstock (2026) *core-drilling-rig*. Available at: https://www.shutterstock.com/search/core-drilling-rig (Accessed 28-03-2026)

Shutterstock (2026b) *Geologist-field*. Available at: https://www.shutterstock.com/search/geologist-field (Accessed 28-03-2026)

SQLite (2026) *SQLite*. Available at: https://sqlite.org/ (Accessed 14-02-2026)

svelte (2026) *Style Binding*. Available at: https://svelte.dev/playground/90ae426e584a44069d9519ed7ac67ac4?version=3.29.4#H4sIAAAAAAAACm2SwW7bMAyGX4XRYd6AOG2D9aLEKdpddu9u8w6KzdhcFcmQ6CSe4XcfZNkJWvQk4vtJ_SSlXhh1RCHFD628hy_wyp1GeCFTkqkkvChPhViKA2n0Qv7uBXdNyA9ALOfq56ZZ-RNqDmyvPH7GC2sYDXshxdYXjhre5SZnjQyqYDohZHBQ2uNMC6utgwwStkfFNpn5uSYOyckYXLFGZnSvjSrIVEGOIPWRSHhsLpskN9u7m7vZ7ltma8AaWWgq3rL-6zfIdtBfW1rEaBjGbn_ZqtK4CLfEynjLIk1hW9IJirDHbK5-giRGCUhIkmEHabrLzS1TRnkuiB4_UWsbHEo6xevrNfjwMFkucgOwV8Vb5WxrynTckYR-PIcNBHlm43qGTUD9u-UMucnFZOVs043j1OvoNRqNYuiyD0HOZyq5lvCwvm8um4hqpKpmCev7G7s1JsHZTum9bnHWrCvRpU6V1HoJD7eqknyjVSfhoHFmf1vPdOjS6dNIKNAwuklVmiqTEuPRf1Cm2cfRJ3SwhlNP_zD0ejVlp4wnJmskrL77kYa98Gp6u_eTrx8_nzJ-zU3OU_32Lu5PLAXjhYVk1-LwZylYkT6TKYUc__jwH0t1KuN6AwAA (Accessed 14-03-2026)

Svelte (2026b) *Errors*. Available at: https://svelte.dev/docs/kit/errors (Accessed 22-03-2026)

svelteUX (2026) *Svelte-ux*. Available at: https://svelte-ux.techniq.dev/ (Accessed 14-03-2026)

Tablogs (2026) *TabLogs*. Available at: https://tablogs.com/features/app (Accessed 14-02-2026)

versus (2026) *versus.com*. Available at: https://versus.com/en/arm-cortex-a35-vs-arm-cortex-a57-vs-arm-cortex-a73-vs-arm-cortex-a76 (Accessed 14-02-2026)

WilliamsB39 (2023) *Drill-Hole-Desurveying*. Available at: https://github.com/WilliamsB39/Drillhole-Desurveying (Accessed 14-02-2026)

w3schools (2026) *Html form input types*. Available at: https://www.w3schools.com/html/html_form_input_types.asp (Accessed 07-03-2026)

w3schools (2026b) *Parse float*. Available at: https://www.w3schools.com/jsref/jsref_parsefloat.asp (Accessed 07-03-2026)

w3schools (2026c) *HTML <a> download Attribute*. Available at: https://www.w3schools.com/tags/att_a_download.asp#:~:text=The%20download%20attribute%20specifies%20that,file%20after%20it%20is%20downloaded (Accessed 20-03-2026)

w3schools (2026d) *HTML <form> enctype Attribute*. Available at: https://www.w3schools.com/tags/att_form_enctype.asp (Accessed 21-03-2026)

Zenhub (2026) *Zenhub*. Available at: https://app.zenhub.com/ (Accessed 14-02-2026)

9. Appendices

9.1. Appendix 1 – Software specification + AI Input

AI Input:

I am Looking to create a 3 Tier Webapp

All components of the webapp should be free and entirely open source, no paid tiers. Components should contain as many out of the box features as possible. The development time for this application is 3 months so components should be fast to set up and deploy.

The application should contain a Javascript or typescript web client, a backend API that does not need to be javascript or typescript based and a relational database. The Web client should interact with the database to create, read, update and delete data through the backend API.

The web client component should be created using a client side javascript framework with potential to be written in typescript. Examples I have researched and have experience with are React ,Svelte and next.js. This part of the application should be styled using a suitable component first CSS framework rather than a utility first framework and not rely on vanilla CSS. I have encountered Bulma, Material Ui and bootstrap in this arena. The selected client side framework and css framework should easily be able to be

configured to run on small screen sizes, like a phone and a tablet. I have experience with React and Svelte in this arena.

The backend API component does not need to be Javascript based. It should have security configuration features out of the box rather than relying heavily on external libraries. This component does not require a user interface as it will be accessed via created API endpoints by the web client component. Examples I have come across in my research are Express, Django, Ruby on rails, Hapi and laravel. I have experience in Express and Hapi and also have some experience in Python programming. This should be written in an interpreted language rather than compiled language by personal preference for this application.

The database component must be a relational database, preferably written in SQL or a similar language. The database component will be connected to the API component therefore must be compatible with this. Based on the fact that the components in the project should be entirely open source, I have narrowed down potential database technologies to MariaDB, PostgreSQL, SQLITE or firebird.

I will also attach a rough software specification outlining how I think the application should function. Based on the above and the following specification, what set of technologies should I use and why. Could you return a pros and cons list for each technology including sources of information for the reasons for these pros and cons and suggest a top 5 set of technology combinations for me to use.

I will also attach my project introduction to give more context to my desired outcome for this application.

Please also take into consideration that I would like to learn new skills and languages throughout the course of development but also that I am constrained to a 3 month development period with no chance of extension.

Software Specification:

General description

The intended application is a diamond drill management application for the mining industry. The application should be comprised of a database for storing information, a front end application to allowing the user to interact with the database and a backend api to facilitate the communication. There is also another front end application that is connected to a physical RPI based device that retrieves parameter data from the database and sends measured survey data to the database.

The functions of this product Is to provide a web interface to create drill programs comprised of drillholes. The user should be able to log lithological, structural and assay data and pass this to a database. The user should be able to approve data sent to the application by the physical RPI device and then commit this data to the application.

Functional requirements

WEB APP

A user should be able to be created

A user should be comprised of a username and a password at a minimum. The user should not be able to be created by anyone. The user should be created by an administrative user or super user

A user should be able to login with the credentials provided by the administrative user after creation

A user should be able to create a drillhole program or 'campaign' with a unique user specified ID , a location, type and target specified. The user should be able to populate this program with associated drillholes. Drillholes should be comprised of a unique ID , x y and z coordinates , a dip, azimuth and length. The drillhole details should be automatically aggregated into a total number of holes attribute and total length attribute. Users should be able to create these manually in a web input form or also by uploading a .csv file

In terms of units of measurement for these attributes

X coordinates - should be unitless

Y coordinates - should be unitless

Z coordinates - should be unitless

Dip - should be in angular degress

Azimuth - should be in angular degrees

Length - should be in meters

Total holes - should be unitless

Total length - should be in meters

Drill program id - should me unitless

Drill hole id - should be unitless

Users should not be able to create duplicate entries of drill programs or drillholes. If possible it would be good to have drillhole ids increment based off a starting value.

After creation of a drillholes , a user should be able to retrieve it. They should then be able to create drillhole logs associated with the retrieved drillhole, this should be done in a single area of the app, potentially in different tabs on the same page. The user should be able to create a lithological log, a structural log, an alteration log and a log containing the samples that the user would like to generate to send to the lab. The user should be able to generate a PDF with these lists of samples. The samples should have a unique, sequential ID for each sample

Lithological logs should be comprised of rows of records. Each record should have to and from depth fields measured in meters, a lithological code selected from a predefined drop down and a section for writing a comment. Structural and alteration logs should also have the same to and from fields and description box as well as the predefined drop down but these drop downs should reference different predefined lists related to the types of logs these are.

The mineral logs should also have to and from depths and comments but also an ID field. The first row in the log should allow the ID to be input but then should be autopopulated

with subsequent records. This autopopulated record should be able to be manually changed however

The user should have the option to generate a PDF report outlining the samples to be analysed in the laboratory from the mineral log records assocaited with the drillhole they are logging

The user should be able to retrieve surveys pending approval uploaded from the survey device and approve these. This data should be then passed to a database table storing survey results

These results will have a depth in meters, a dip in angular degrees and an azimuth in angular degrees

SURVEY WEB APP

The survey web app will be connected to the rpi based survey tool. This should be able to retrieve planned drillhole data from the database that was put in by the web app user. The survey web app user should should also have to log in with credentials provided by an admin user.

The survey web app user should be able to select a drillhole to start surveying. The planned information should be communicated to the rpi based device. The device should communicate back the current dip and azimuth value until they match the planned values retrieved. The web app user should then be able to send this data to the database to await retrieval. If not approved by the web app user, the process should be able to be repeated and resent. The previous attempts should be kept and not removed .

Interface requirements

The portion of the application known as the web app should be built using a javascript/html front end framework. The portion of the application known as the survey web application should be built using the same framework but should be able to be

scaled down to a tablet sized window. These web apps should be styled using a component first css framework rather than a utility first css framework. The backend application should control communication between the users of both web apps and the underlying database. This backend application does not need a web interface although a simple one might be useful for testing purposes during development. This backend should be able to support a rest API to allow the communication and also allow for multiple connections as it should be possible to connect to from both frontend applications. It is not a requirement for this API to be Javascript based but it must be able to communicate with the relational database that is chosen. The underlying database , which is facillitated to communicate to the users via the api and web applications should be a relational database that is able to be connected to the rest api.

Performance requirements

The web app should be able to send and receive data in a timely manner. These could be large amounts of data. This should appear seamless to the user and not require much time where the user is left waiting for data. This should be reactive.

The survey application should be able to also retrieve data in a timely manner. However, when the main function of the application is running it should be able to receive data in close to real time as the user needs to be able to stop moving the instrument as soon as the values match the planned values for the drillhole. This application does not need to be very large but must be able to quickly receive data from its associated RPI device.

The database must be able to store large amounts of data as datasets produced in this sort of environment are often large. This should also be able to store a large amount of different types of data.

The Api portion of the application should be able to serve responses to request from the user rapidly.

Design constraints

The web and survey applications should be built using a client side Javascript/html framework and styled using a component first css framework. These applications should be able to run in most modern browsers and not restricted to one. The framwork chosen for both should be able to run on smaller devices also as the survey application would be designed to run on a phone or tablet device, this should not restrict the ability to build a web application for a desk top PC. These applications should be able to connect to a compatible rest API. The API should be compatible with the technology used to construct the web and survey applications but does not need to be based in any particular language. The API application should be compatible with servers running linux or windows. The database underlying the API should be relational and preferably written in SQL or equivalent. This must be compatible with the API connecting it to the users.

Non functional constraints

All level of the application should be able to support strong authentication strategies. TLS should be configurable throughout the components and HTTPS should be supported. There should be mechanisms in place to prevent exposure of data outside of the application. Data in the application would be considered sensitive so should not be exposed and technologies used should be capable of preventing this natively.

The application should be constructable within 3 months so technologies should be quick to pick up and compose due to this timing restraint.

The web , api and databases need to be compatible and should provide libraries to allow for this to happen easily due to the constraint in timing.

The web applications and API should be deployable across the web using a service or cloud solutions. The database should be able to be installed on a machine, physical or virtual or could be created using a provided service.

All tools used have to be totally open source by nature and free.

Schedule and budget

The project should be possible to complete within 3 months and for free using open source components.

9.2. Appendix 2 – Perplexity response

I am Looking to create a 3 Tier Webapp_ _All compo.pdf

9.3. Appendix 3 – Gemini Response

prompt_only.pdf

9.4. Appendix 4 – User stories

Epics

DrillTek web app

- Drilling portal
- Logging portal
- Logging area
- Survey Portal

Drillsight physical aspect

Drillsight mobile app

The Second 2 epics will be allocated sprint space at the end - might be out of scope looking at the time left on the project

Separate to remember for 17-01-2025

- Pagination of lists as a separate story
- Create a user as a DB admin as a separate story

Definite Icebox

Import assays

Survey portal

Survey App

Survey tool

DrillTek web app

Story 1: Log in

As a user

I should be able to log in to the web application

So that I can access the rest of the application

Assumptions:

The application is to be supported by a Django backend and

Postgresql database. A user table will need to be created in the

Database with the attributes outlined in the database design

Documentation. The backend will need to be configured to support

Retrieval of this user. The API backend will require an authentication

Mechanism to be set up and a session will need to be created for the user

For further requests during the life of the application.

Acceptance criteria:

Scenario: Successful user login

Given I am on the landing page

And I have valid credentials provided by the database admin

When I click the login button

Then a modal should appear with an input form

When I enter valid credentials

And click the login button

Then I should be redirected to a page to proceed to the different parts of the site

Scenario: Unsuccessful user login

Given I am on the landing page

And I have invalid credentials or no credentials

When I click the login button

Then a modal should appear with an input form

When I enter invalid or no credentials

Then I should be presented with an appropriate error message

Story 2: Drilling portal landing

As a user

I should be able to enter the drilling portal

So that I can see a list of drill programs

Assumptions:

The application is to be supported by a Django backend and

Postgresql database. A drill program table will need to be created in the

Database with the attributes outlined in the database design

Documentation. The backend will need to be configured to support

Retrieval of all drill programs so that they can be listed on the portal landing

page. There should be buttons alongside each of these listed programs that

Will eventually lead to the user being able to open the program

Acceptance critera:

Scenario: Successful Drilling portal landing

Given that I am on the page displaying all portals

And I am an authenticated user

When I click the button to enter the drilling portal

Then I should be taken to the drilling portal

And all drill programs should be listed with an 'open' button next to each

Story 3: Filter drilling portal

As a user

I should be able to filter drill programs in the drilling portal by attributes of drill programs or

By using a search bar

So that I can identify the drill program I wish to access

Assumptions:

The drill programs should already have been loaded by visiting the page. It would be

Ideal not to have to recall the API endpoint supporting the retrieval. The user should be able

To filter by multiple parameters not just one at a time.

Acceptance criteria:

Scenario: Successful filtering by parameters

Given that I am on the drilling portal landing page

And I am an authenticated user

When I click to filter

Then select the filters I wish to apply

Then The list of drill programs should dynamically filter down to just those meeting the criteria

Scenario: Unsuccessful filtering by parameters

Given that I am on the drilling portal landing page

And I am an authenticated user

When I click to filter

Then select the filters I wish to apply

And no programs meet the criteria

Then a message should be displayed to me stating that no programs match the criteria

Scenario: Successful filtering using search bar

Given that I am on the drilling portal landing page

And I am an authenticated user

When I type into the search bar

Then select search

Then The list of drill programs should dynamically filter down to the program matching the name typed into the search bar

Scenario: Unsuccessful filtering using search bar

Given that I am on the drilling portal landing page

And I am an authenticated user

When I type into the search bar

And no drill program with a matching name exists

Then select search

Then a message stating there are no matching programs should be displayed

Story 4: Create drill program

As a user

I should be able to create a drill program

So that I can add drillholes to it

Assumptions

A modal should appear allowing the user to fill in details about the drill program. The number of drillholes and meters should not be filled out by the user but should be calculated dynamically based on the drillholes added to it. There will need to be an end point to the django backend to post a drillprogram. The meters and total holes should default to 0 on creation

Acceptance criteria:

Scenario: Successful addition of drill program

Given that I am on the drilling portal landing page

And I am an authenticated user

When I select 'Add'

Then a modal should appear with a form to fill drillhole details

When I fill in my program details

And select Add

Then I should be redirected to the drill program page of the program just added

Scenario: Unsuccessful addition of a drill program

Given that I am on the drilling portal landing page

And I am an authenticated user

When I select 'Add'

Then a modal should appear with a form to fill drillhole details

When I fill in my program details

And select Add

And input details violate datatypes or are missing

Then The form should display a helpful message based on the scenario whilst not giving away details of the database

Story 5: Open drill program

As a user

When I select the open button on a drill program in the drilling portal

I should be redirected to a page displaying the details of the drill program and drillholes associated with it

Assumptions:

A drillhole table will need to be created in the post gres database with the drill program table as a related table. It will be optional that a program has drillholes as outlined in the database design

So displaying these will be based on if any are related. Each drillhole listed should have a button to be able to open its details. There should be buttons to be able to add a drillhole below, upload a cluster of drillholes as a csv or download the csv template for that upload. These functions will be implemented in other stories but they should be present in this story. There should be a button to initiate editing the drill program also.

Acceptance criteria:

Scenario: Successful navigation to a drill program

Given that I am on the drilling portal landing page

And I am an authenticated user

When I select open next to any drill program

Then I should be redirected to the details page of the drill program

And the details, associated drillholes, buttons to add drillholes should be present

Story 6: Edit drill program

As a user

When I select the edit button

I should be greeted by a modal form, prepopulated with the program details

So that I can edit the drill programs details

Assumptions:

This will require a post endpoint in the django backend to be able to replace details in the Postgresql database. This form should be in the form of a modal and should be populated by the current details. If the name is changed this should be replaced in the drillhole table so that those records are not stranded. This should contain the same validation as adding a program. This will likely require a recall on the endpoint as details will have changed on the program

Acceptance criteria:

Scenario: Successful update of a drill program

Given that I am on the drill program details page

And I am an authenticated user

When I select edit on the drill program details

Then a modal form should appear prepopulated with the current details

Then I update the details

And select ok

Then I should be redirected back to the original page with the new details visible

Scenario: Unsuccessful update of a drill program

Given that I am on the drill program details page

And I am an authenticated user

When I select edit on the drill program details

Then a modal form should appear prepopulated with the current details

Then I update the details

And select ok

And input details violate datatypes or are missing

Then The form should display a helpful message based on the scenario whilst not giving away details of the database

Story 7: Add drillhole using form

As a user

I should be able to add a drillhole by manually entering details

So that I can associate drillholes with my drill program

Assumptions:

When the user selects the add button this should open a modal form so that the user can manually type out the details of the drillhole they wish to add. This will require a drillhole table to be constructed in the Postgresql database as outlined in the database design or similar. This will also require a post endpoint in the django API. This should have

validation built in to help the user should they violate the table data types or leave a field blank. This might require an api call when returning to the drill program details page so that the new hole appears

Acceptance criteria:

Scenario: Successful manual addition of a drillhole

Given that I am on the drill program details page

And I am an authenticated user

When I select the 'add drillhole' button

Then a modal form should appear

Then I fill the form and select 'submit'

Then I should be returned to the drill program details page

And the newly added drillhole should appear

Scenario: Unsuccessful manual addition of a drillhole

Given that I am on the drill program details page

And I am an authenticated user

When I select the 'add drillhole' button

Then a modal form should appear

Then I fill the form and select 'submit'

And input details violate datatypes or are missing

Then The form should display a helpful message based on the scenario whilst not giving away details of the database

Story 8: Add drillhole by uploading .csv

As a user

I should be able to batch upload drillholes by uploading a csv

So that I can upload a group of holes to my drill program

Assumptions:

This may require a different API endpoint for uploading multiple drillholes to a program as a pose to a single drillhole. Research should be done on how to handle .csv files during the development of this feature as currently a method is unchosen.

Acceptance criteria:

Scenario: Successful upload of a group of drillholes using a .csv

Given that I am on the drill program details page

And I am an authenticated user

When I select the 'upload .csv' button

Then a modal form with a file selector should appear

Then I should be able to select a file from my file system

And select 'ok'

Then I should be returned to the drill program details page

And the newly added drillhole should appear

Scenario: Unsuccessful upload of a group of drillholes using a .csv

Given that I am on the drill program details page

And I am an authenticated user

When I select the 'upload .csv' button

Then a modal form with a file selector should appear

Then I should be able to select a file from my file system

And select 'ok'

And input details violate datatypes or are missing

Then The form should display a helpful message based on the scenario whilst not giving away details of the database

Story 9: Download a .csv template for uploading drillholes

As a user

I should be able to download a .csv template compatible with uploading drillholes via .csv

So that I can populate a .csv in the correct format before uploading

Assumptions:

Will need to research how to store a file for the user to download when they select the 'download template' button. Should probably leave the button off until the functionality is in place and develop this as a full 'new' feature. This is a nice to have and should not feature in the minimum viable product.

Acceptance criteria:

Scenario: Successful Download of .csv tempalte

Given that I am on the drill program details page

And I am an authenticated user

When I select the 'download template' button

Then the file should be downloaded to my downloads

Story 10: Filter drillholes

As a user

I should be able to search for a desired drillhole using a searchbar

So that I can find the specific drillhole I want to interact with

Assumptions: This should be just a search bar above the list of drillholes as outlined in the wireframe for the drill program details page.

Acceptance criteria:

Scenario: Successful filtering using search bar

Given that I am on the drill program details page

And I am an authenticated user

When I type into the search bar

Then select search

Then The list of drill holes should dynamically filter down to the hole matching the name typed into the search bar

Scenario: Unsuccessful filtering using search bar

Given that I am on the drill program details page

And I am an authenticated user

When I type into the search bar

And no drill holes with a matching name exists

Then select search

Then a message stating there are no matching holes should be displayed

Story 11: Open drillhole

As a user

I should be able to open a drillhole associated with a drill program

So that I can view and update it

Assumptions:

This should be accessible through the associated drill program. This will likely require an api endpoint to retrieve a single drillhole on access.

Acceptance criteria:

Scenario: Successful opening of drillhole

Given that I am on the drill program details page

And I am an authenticated user

When I select open next to a desired drillhole

Then I am redirected to the page displaying the drillhole details.

Story 12: Edit drillhole

As a user

I should be able to edit a drillhole from its details page

So that I can update its current details

Assumptions: This will likely require a put endpoint for the API so that an update can take place in the on the drillhole table. This should follow the model of the drillhole details page. This will likely require an additional get request to the Api as the details will have been updated

Acceptance criteria:

Scenario: Successful update of drillhole

Given that I am on the drill hole details page

And I select an edit button

Then a modal popup should appear containing a form prepopulated with the current details

Then after filling the form

And selecting 'submit'

Then I should be redirected back to the same drill hole details page

Scenario: Unsuccessful update of drillhole

Given that I am on the drill hole details page

And I select an edit button

Then a modal popup should appear containing a form prepopulated with the current details

Then after filling the form

And selecting 'submit'

And input details violate datatypes or are missing

Then The form should display a helpful message based on the scenario whilst not giving away details of the database

Story 13: Logging Portal Landing

As a user

I should be able to enter the logging portal

So that I can see the list of drill programs available to log

Assumptions: This will require an api endpoint to retrieve drill programs. This should only retrieve drill programs that have drill holes associated and not 'empty' programs

Acceptance criteria:

Scenario: Successfully accessing logging portal

Given that I am on the page displaying all portals

And I am an authenticated user

When I click the button to enter the logging portal

Then I should be taken to the logging portal

And all drill programs should be listed with an 'open' button next to each

Story 14: Filter Logging portal Landing

As a user

I should be able to filter drill programs in the logging portal by attributes of drill programs or

By using a search bar

So that I can identify the drill program I wish to access

Assumptions:

The drill programs should already have been loaded by visiting the page. It would be

Ideal not to have to recall the API endpoint supporting the retrieval. The user should be able

To filter by multiple parameters not just one at a time.

Acceptance criteria:

Scenario: Successful filtering by parameters

Given that I am on the logging portal landing page

And I am an authenticated user

When I click to filter

Then select the filters I wish to apply

Then The list of drill programs should dynamically filter down to just those meeting the criteria

Scenario: Unsuccessful filtering by parameters

Given that I am on the logging portal landing page

And I am an authenticated user

When I click to filter

Then select the filters I wish to apply

And no programs meet the criteria

Then a message should be displayed to me stating that no programs match the criteria

Scenario: Successful filtering using search bar

Given that I am on the logging portal landing page

And I am an authenticated user

When I type into the search bar

Then select search

Then The list of drill programs should dynamically filter down to the program matching the name typed into the search bar

Scenario: Unsuccessful filtering using search bar

Given that I am on the logging portal landing page

And I am an authenticated user

When I type into the search bar

And no drill program with a matching name exists

Then select search

Then a message stating there are no matching programs should be displayed

Story 15: Open drill program logging

As a user

When I select the open button on a drill program in the logging portal

I should be redirected to a page displaying the list of drill holes in the program

Assumptions:

A drillhole table will need to be created in the post gres database with the drill program table as a related table. It will be optional that a program has drillholes as outlined in the database design

So displaying these will be based on if any are related. Each drillhole listed should have a button to be able to open the logging pages for the hole

Acceptance criteria:

Scenario: Successful navigation to a drill program

Given that I am on the logging portal landing page

And I am an authenticated user

When I select open next to any drill program

Then I should be redirected to the page listing all associated drillholes

Story 16: Filter Drillhole logging

As a user

I should be able to search for a desired drillhole using a searchbar

So that I can find the specific drillhole I want to interact with

Assumptions: This should be just a search bar above the list of drillholes as outlined in the wireframe for the loggingid page.

Acceptance criteria:

Scenario: Successful filtering using search bar

Given that I am inside of the loggingid page

And I am an authenticated user

When I type into the search bar

Then select search

Then The list of drill holes should dynamically filter down to the drillhole matching the name typed into the search bar

Scenario: Unsuccessful filtering using search bar

Given that I am on the loggingid page

And I am an authenticated user

When I type into the search bar

And no drill holes with a matching name exists

Then select search

Then a message stating there are no matching holes should be displayed

Story 17: Open drillhole logging

As a user

I should be able to open a drillhole associated with a drill program

So that I can log the drillhole

Assumptions:

This should be accessible through the associated drill program. This will likely require an api endpoint to retrieve a single drillhole on access. This will also require the retrieval of any existing drillhole information. This page should contain a tab-type system as outlined in the wireframe where the user will be able to switch between tabs in a relatively seamless manner

Acceptance criteria:

Scenario: Successful opening of drillhole

Given that I am on the loggingid page

And I am an authenticated user

When I select open next to a desired drillhole

Then I am redirected to the logging area for that drillhole on the lithological tab

Story 18: drillhole lithology logging

As a user

I should be able to populate a lithology log for a drillhole

So that I can record drill hole lithology information

Assumptions:

This should be accessible via a lithological tab as outlined in the conceptual wireframe for the page. This should be a rerendering of the page rather than a totally separate page as this and the other tabs should appear seamless, as if the user is flicking between them. If lithological information already exists, this should be prepopulated here and editable, the user should be able to edit and recommit the data without a separate edit option. A save button should only appear if there has been input added or something has been changed in the input. There should be a graph (technology tbd) that populates dynamically as information is input into the form, representing a 'graphical log'. Validation should be present and display messages to the user if something goes wrong. Lith code as represented in the database design should be selectable from a dropdown list . The date logged should not be visible but should be updated when a user submits or edits.

Acceptance criteria:

Scenario: Successful lithological logging of drillhole

Given that I am on the lithological tab of the drillhole logging area

And I am an authenticated user

When I input information into the 'log'

Then the graph displaying a graphical representation of the log

Then I select 'submit'

Then the information will commit to the database

Scenario: Unsuccessful lithological logging of a drillhole

Given that I am on the lithological tab of the drillhole logging area

And I am an authenticated user

When I input information into the 'log'

Then the graph displaying a graphical representation of the log

Then I select 'submit'

And I have input invalid data into the form

Then a message should be displayed describing the validation errors

Story 19: drillhole structure logging

As a user

I should be able to populate a structure log for a drillhole

So that I can record drill hole structure information

Assumptions:

This should be accessible via a structure tab as outlined in the conceptual wireframe for the page. This should be a rerendering of the page rather than a totally separate page as this and the other tabs should appear seamless, as if the user is flicking between them. If structure information already exists, this should be prepopulated here and editable, the user should be able to edit and recommit the data without a separate edit option. A save button should only appear if there has been input added or something has been changed in the input. There should be a graph (technology tbd) that populates dynamically as information is input into the form, representing a 'graphical log'. Validation should be present and display messages to the user if something goes wrong. structure code as represented in the database design should be selectable from a dropdown list. The date logged should not be visible but should be updated when a user submits or edits.

Acceptance criteria:

Scenario: Successful structure logging of drillhole

Given that I am on the structure tab of the drillhole logging area

And I am an authenticated user

When I input information into the 'log'

Then the graph displaying a graphical representation of the log

Then I select 'submit'

Then the information will commit to the database

Scenario: Unsuccessful structure logging of a drillhole

Given that I am on the structure tab of the drillhole logging area

And I am an authenticated user

When I input information into the 'log'

Then the graph displaying a graphical representation of the log

Then I select 'submit'

And I have input invalid data into the form

Then a message should be displayed describing the validation errors

Story 20: drillhole alteration logging

As a user

I should be able to populate a alteration log for a drillhole

So that I can record drill hole alteration information

Assumptions:

This should be accessible via a alteration tab as outlined in the conceptual wireframe for the page. This should be a rerendering of the page rather than a totally separate page as this and the other tabs should appear seamless, as if the user is flicking between them. If alteration information already exists, this should be prepopulated here and editable, the user should be able to edit and recommit the data without a separate edit option. A save button should only appear if there has been input added or something has been changed in the input. There should be a graph (technology tbd) that populates dynamically as information is input into the form, representing a 'graphical log'. Validation should be present and display messages to the user if something goes wrong. alteration code as represented in the database design should be selectable from a dropdown list . The date logged should not be visible but should be updated when a user submits or edits.

Acceptance criteria:

Scenario: Successful alteration logging of drillhole

Given that I am on the alteration tab of the drillhole logging area

And I am an authenticated user

When I input information into the 'log'

Then the graph displaying a graphical representation of the log

Then I select 'submit'

Then the information will commit to the database

Scenario: Unsuccessful alteration logging of a drillhole

Given that I am on the alteration tab of the drillhole logging area

And I am an authenticated user

When I input information into the 'log'

Then the graph displaying a graphical representation of the log

Then I select 'submit'

And I have input invalid data into the form

Then a message should be displayed describing the validation errors

Story 21: drillhole mineral logging

As a user

I should be able to populate a mineral log for a drillhole

So that I can record drill hole mineralisation information

Assumptions:

This should be accessible via a mineral tab as outlined in the conceptual wireframe for the page. This should be a rerendering of the page rather than a totally separate page as this and the other tabs should appear seamless, as if the user is flicking between them. If mineral information already exists, this should be prepopulated here and editable, the user should be able to edit and recommit the data without a separate edit option. A save

button should only appear if there has been input added or something has been changed in the input. There should be a graph (technology tbd) that populates dynamically as information is input into the form, representing a 'graphical log'. Validation should be present and display messages to the user if something goes wrong. Texture and sample type as represented in the database design should be selectable from a dropdown list. The assay information (zn,pb,ag,fe) should not be visible or editable as this will be imported at a later stage. The date sampled field should be updated when the user submits or edits.

Acceptance criteria:

Scenario: Successful mineral logging of drillhole

Given that I am on the mineral tab of the drillhole logging area

And I am an authenticated user

When I input information into the 'log'

Then the graph displaying a graphical representation of the log

Then I select 'submit'

Then the information will commit to the database

Scenario: Unsuccessful mineral logging of a drillhole

Given that I am on the mineral tab of the drillhole logging area

And I am an authenticated user

When I input information into the 'log'

Then the graph displaying a graphical representation of the log

Then I select 'submit'

And I have input invalid data into the form

Then a message should be displayed describing the validation errors

Story 22: Export sample pdf and .csv

As a user

I should be able to export a pdf and csv of the input sample

So that these can be supplied to the chosen laboratory

Assumptions: This should be accessible as a button on the mineral log tab. The button should only be selectable if information has already been committed about the drillhole as the user should not be able to export a log that is incomplete. The .csv and pdf should be downloaded to the users downloads. The format of these documents should be assessed during the design phase of this story, there are no predefined designs for this but it should contain relevant information from the mineral log table

Acceptance criteria:

Scenario: Successful export of sample .csv and pdf

Given that I am on the mineral tab of the drillhole logging area

And I am an authenticated user

When I select the 'export data' button

Then a .csv and pdf summarising the data should appear in my downloads

Story 23: Log summary page

As a user

I should be able to view a summary page of all the different logs and their graphics

So that I can take in a summary view of the logged information

Assumptions: This should follow the wireframe of the page as outlined in the design pages. This page will be read only and may or may not require a fresh API call to retrieve all of the logs. To save on page space, the individual tables should be scrollable boxes. The graphic should show all of the different logs as graph bars side by side. This should be accessible via any of the log tabs.

Acceptance criteria:

Scenario: Successful opening of log summary

Given that I am on any of the logging tabs

And I am an authenticated user

When I select the 'open summary' button

Then I will be taken to a summary of all the logs

And each log will be displayed as a scrollable table on the page

And a graphic with all the logs as bars will be present on the page

Story 24: Import assays

As a user

I should be able to import assays returned from the lab

So that these can be stored in the database

Assumptions: There should likely be an area built for this in the overall landing page for the application past login. This will likely be in .csv format. If the database allows the storage of files there should likely be a way of iterating over these so the user can assess them before they are committed to the database as actual assay values.

Acceptance criteria:

Scenario: Successful import of assay data

Given that I am on the landing page

And I am an authenticated user

When I select the 'import assays' tab

Then I will be taken to a page showing all the assays available to import

Then I should be able to select a button to open the assays for assessment

Then I should be able to commit the data to the database.

Story 25: Page pagination

As a user

If I am on a page with a list of information

I should be able to define the number of results to display and move between pages of results

Assumptions: If I am on a page where there are lists like the drillholes or drill programs I should be able to define pages of results to display and then cycle between these pages. The results to display per page should be defined as a drop down list and the page number should be like drop down lists. Pagination is supported by Bulma

Acceptance criteria:

Scenario: Successful definition of results per page

Given that I am on a page that supports pagination

And I select the results per page dropdown

And I select a number of results

Then the page should re-render with the desired number of results

Scenario: Successful movement between pages

Given that I am on a page that supports pagination

And I select the next page of results

Then the page should re-render with the next set of results

Story 26: Create users as admin

As a database admin

If I am in the RDBMS

I should be able to create a user profile so that users can log in to the application

Assumptions: There is the possibility to create an admin area with in the application however, just an SQL script to automate this slightly would probably be sufficient in scope of the application. The password should be stored as a hash but the password will need to be present so that it can be supplied to the user. This could be reassessed where the user creates their own password but the admin supplies the email address. This is due to the fact that this would be an organisational app and not just anyone should be able to signup

Acceptance criteria:

Given that I am in the RDBMS

And I run the create user script

Then a user should be created

And I should be able to provide their email and potentially password to the user for login

9.5. Appendix 5 – Screen designs and initial notes

WEB APP

Home Page:

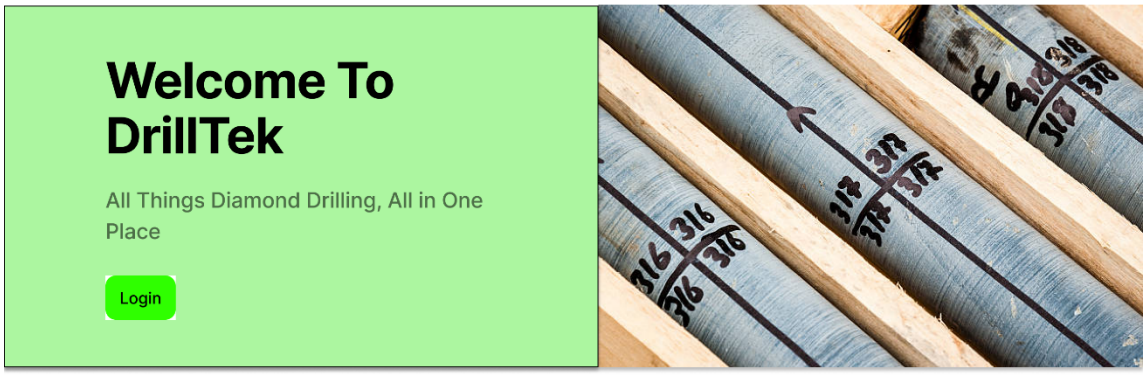


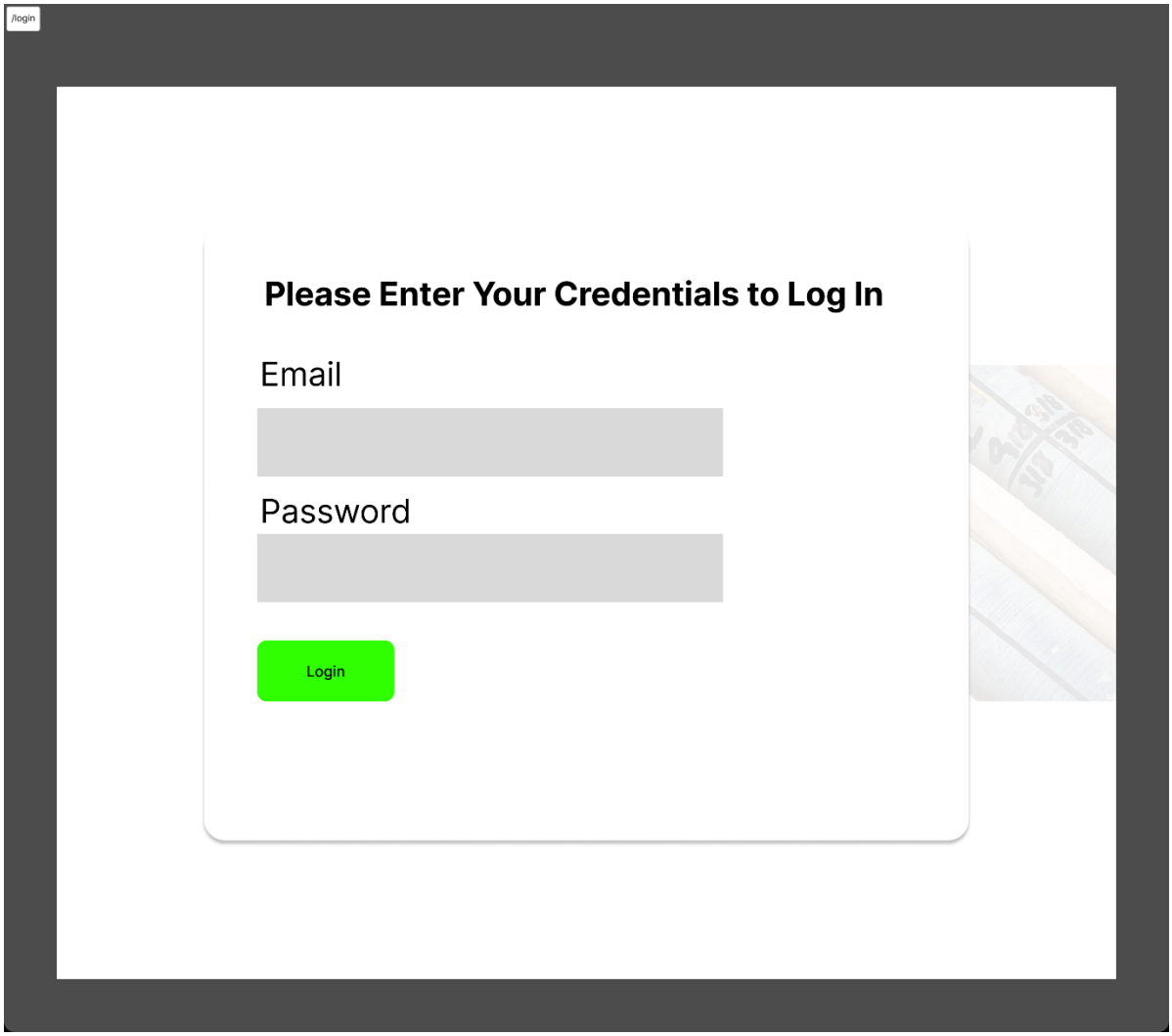
Image stored in project folder.

Could get away with Bulma success for the login - must see what success lighter looks like and see if this is appropriate.

Will use Bulma columns to achieve this between the 2 parts - center this on the page also

Might be able to achieve this with a Bulma hero component rather than components in columns - experiment with whatever is simpler to code - start with hero as might be more straight forward

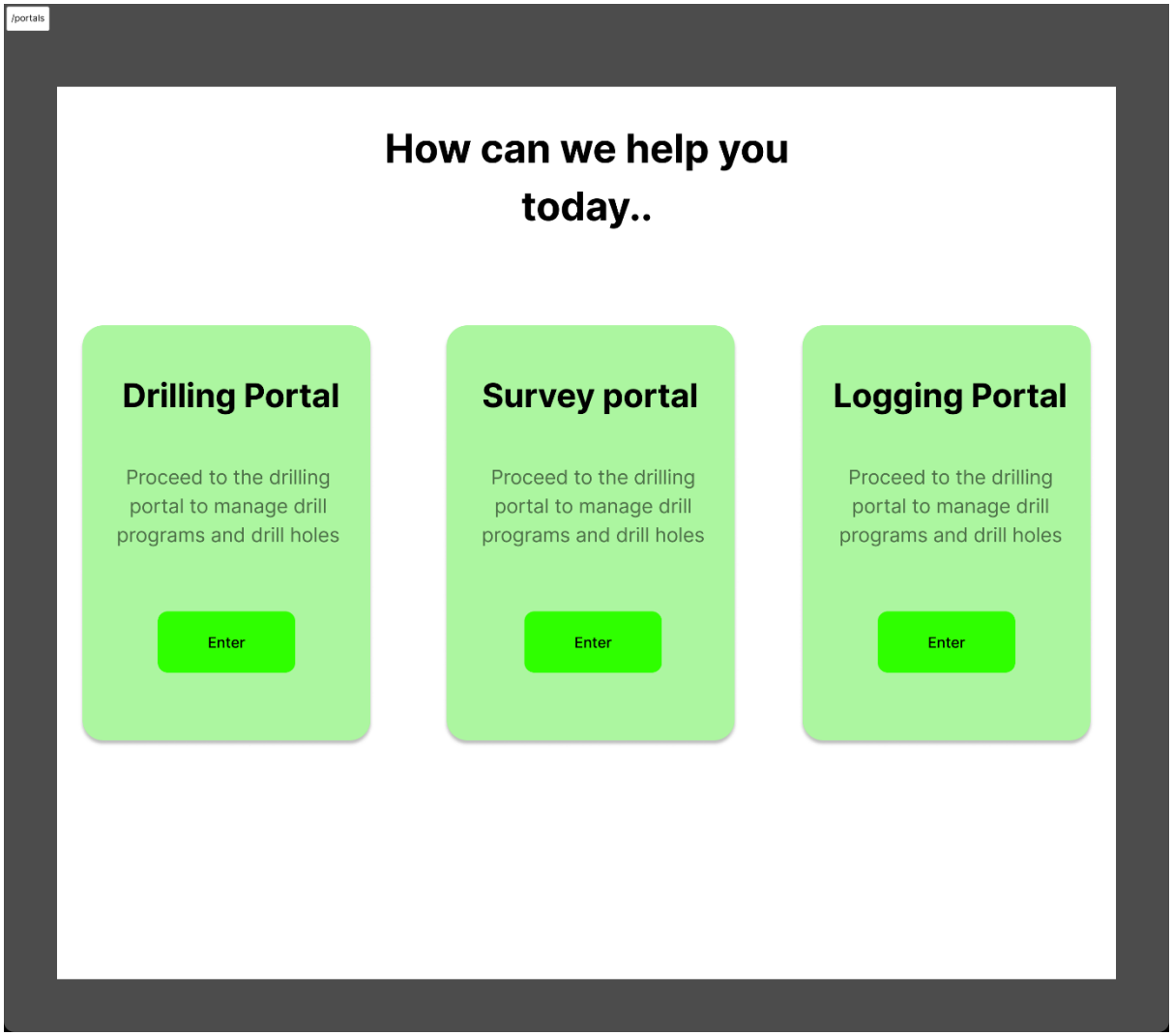
Login:



Will need to overlay this over the original page with Bulma modal - have done this before should be easy.

Bulma form for the login form

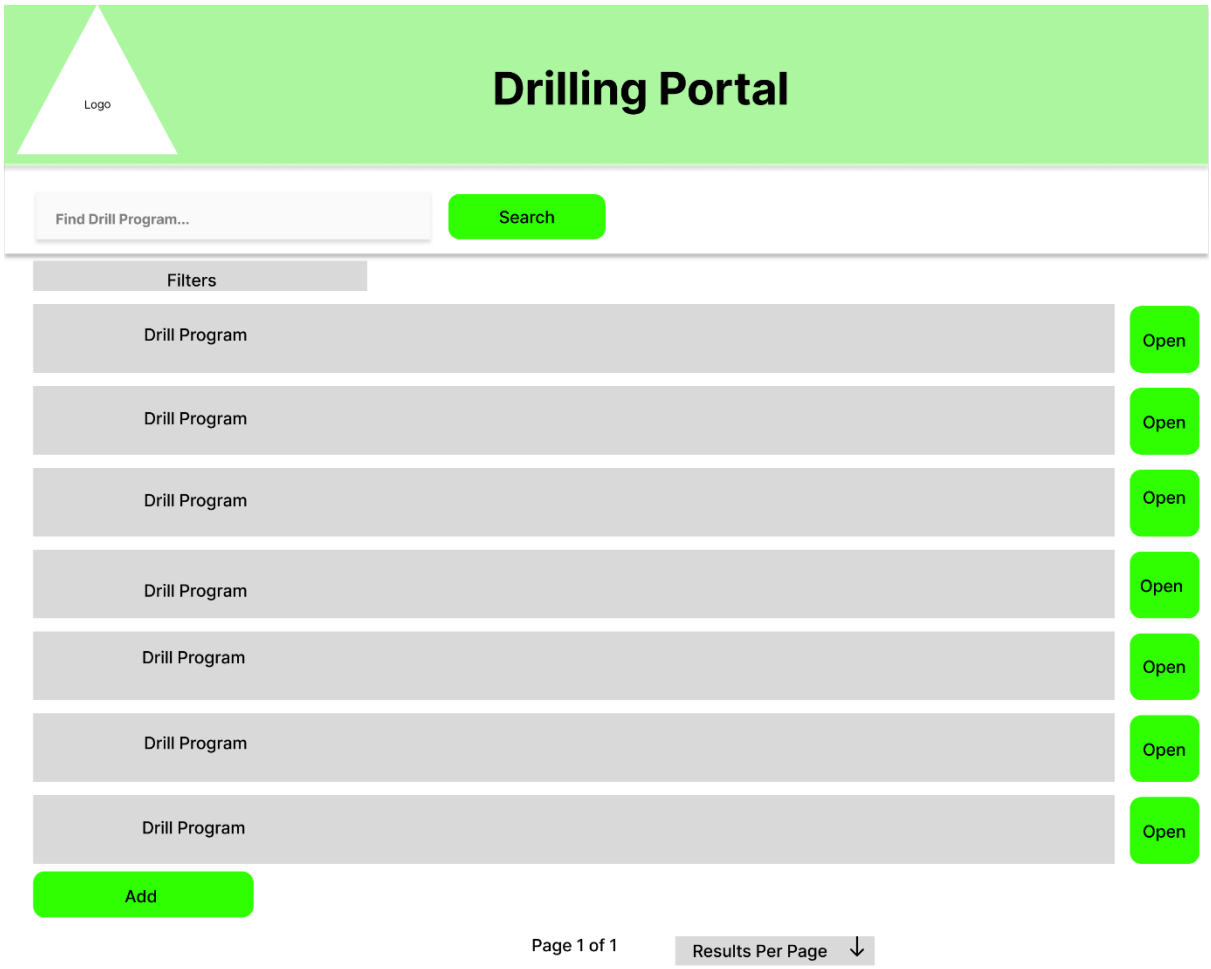
Portals:



Will be able to achieve this by using Bulma columns - might need to round this using vanilla css but appears to be is rounded class. Simple page to code with button links

Will probably need to change button colour on hover however

Drilling Portal



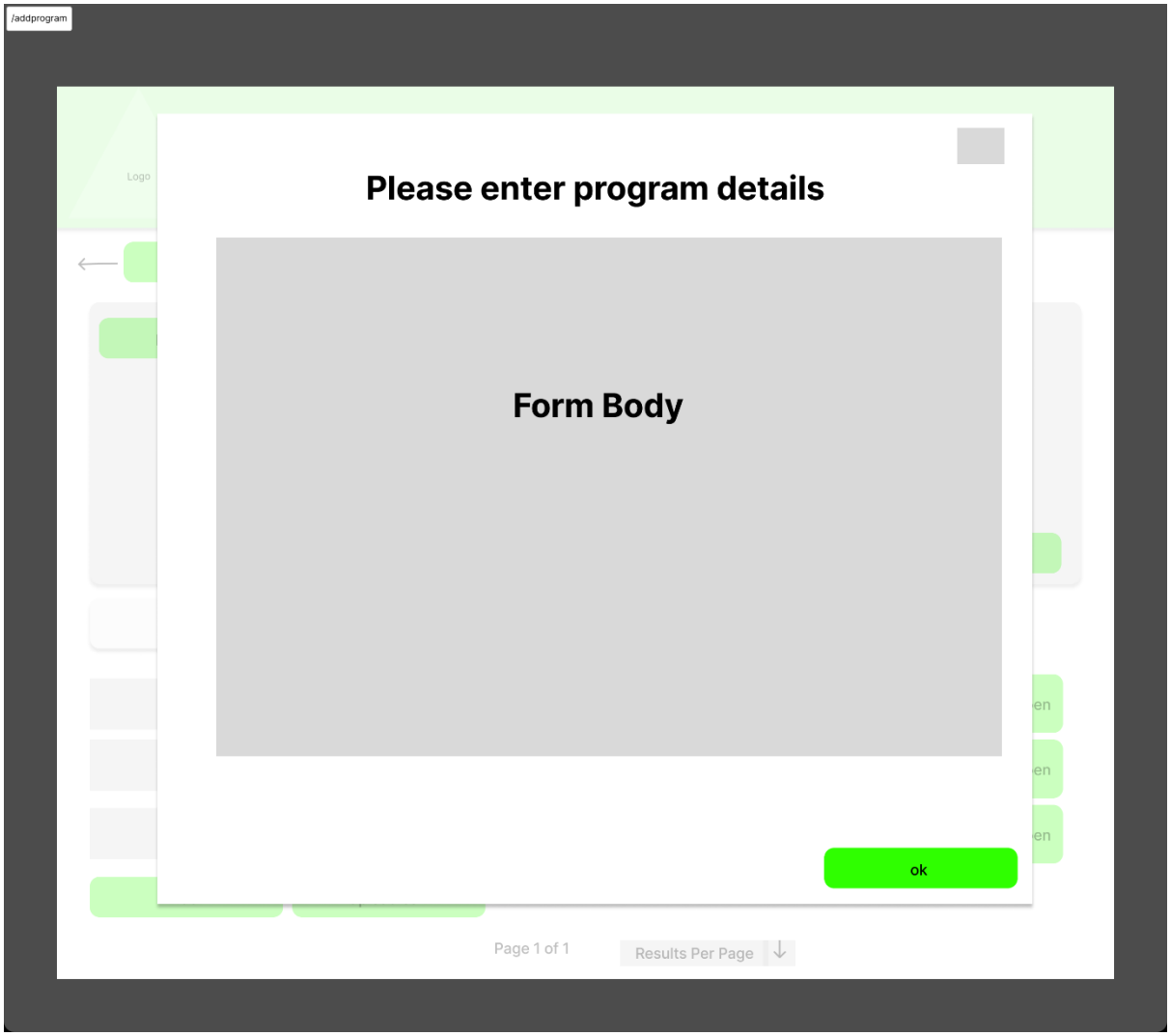
Search bar should be doable with Sveltekit using runes. Filters should work in the same way but will need to be separate

Cards of some sort should work for the rows or could just do a table of boxes - white with drop shadow in Bulma?

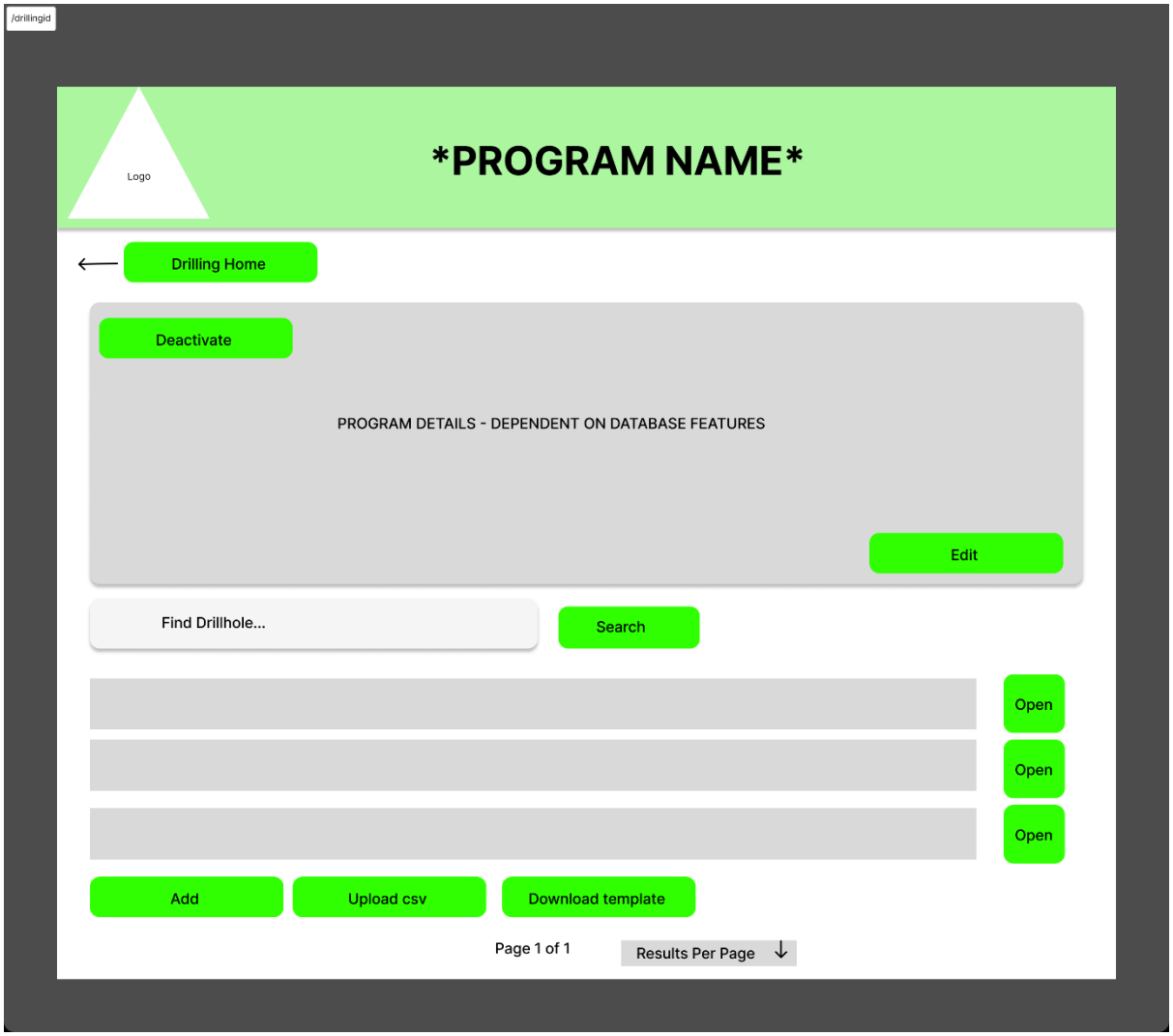
Bulma pagination could be done for the pages - results per page could be a set rune in Sveltekit

Go home with logo or do a footer? - see how this works in practice - could be a later addition

Add drill program:



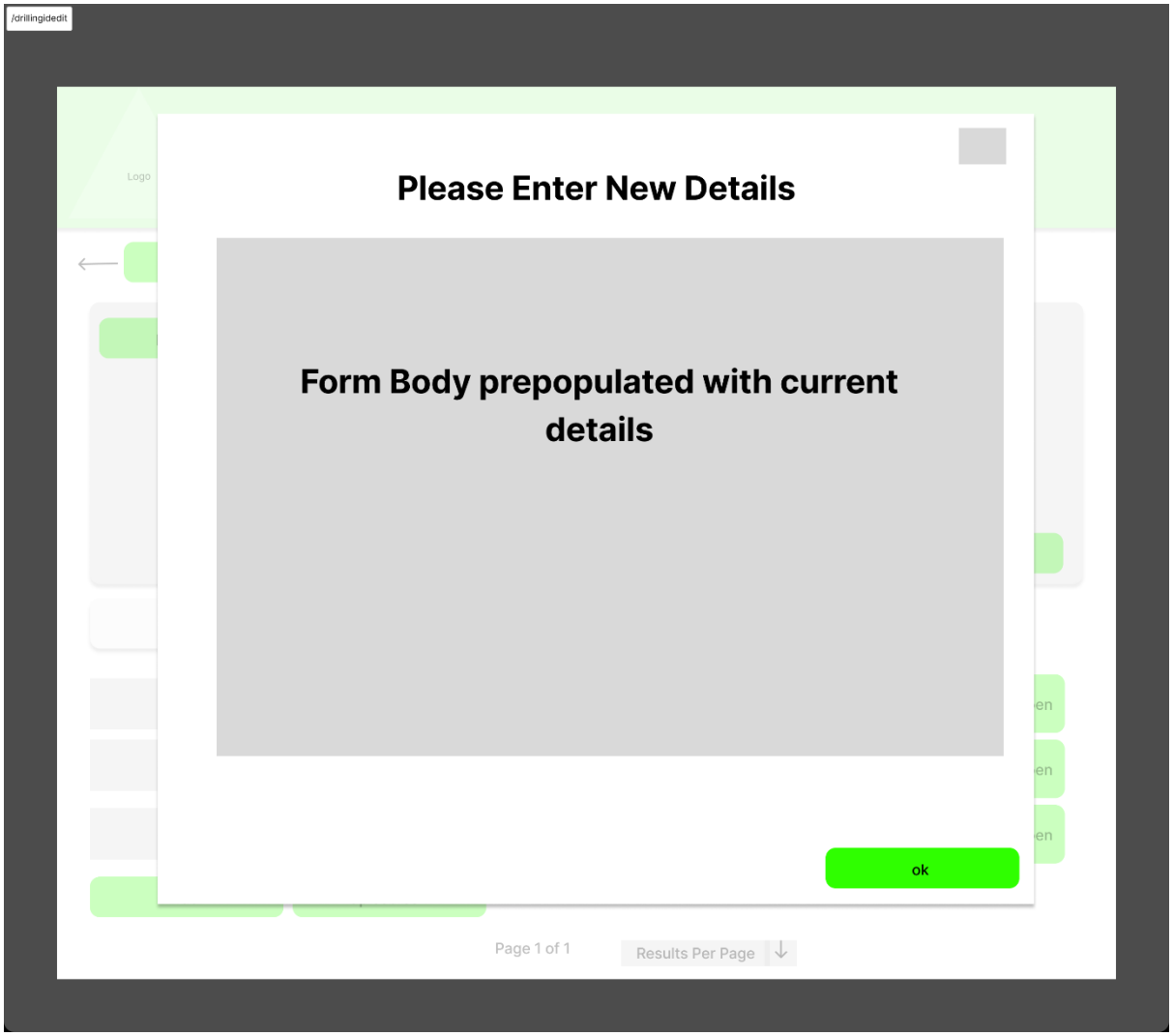
Program Details:



Very much the same as the above - will have a modal for editing/adding/uploading .csv - opening drillhole can go

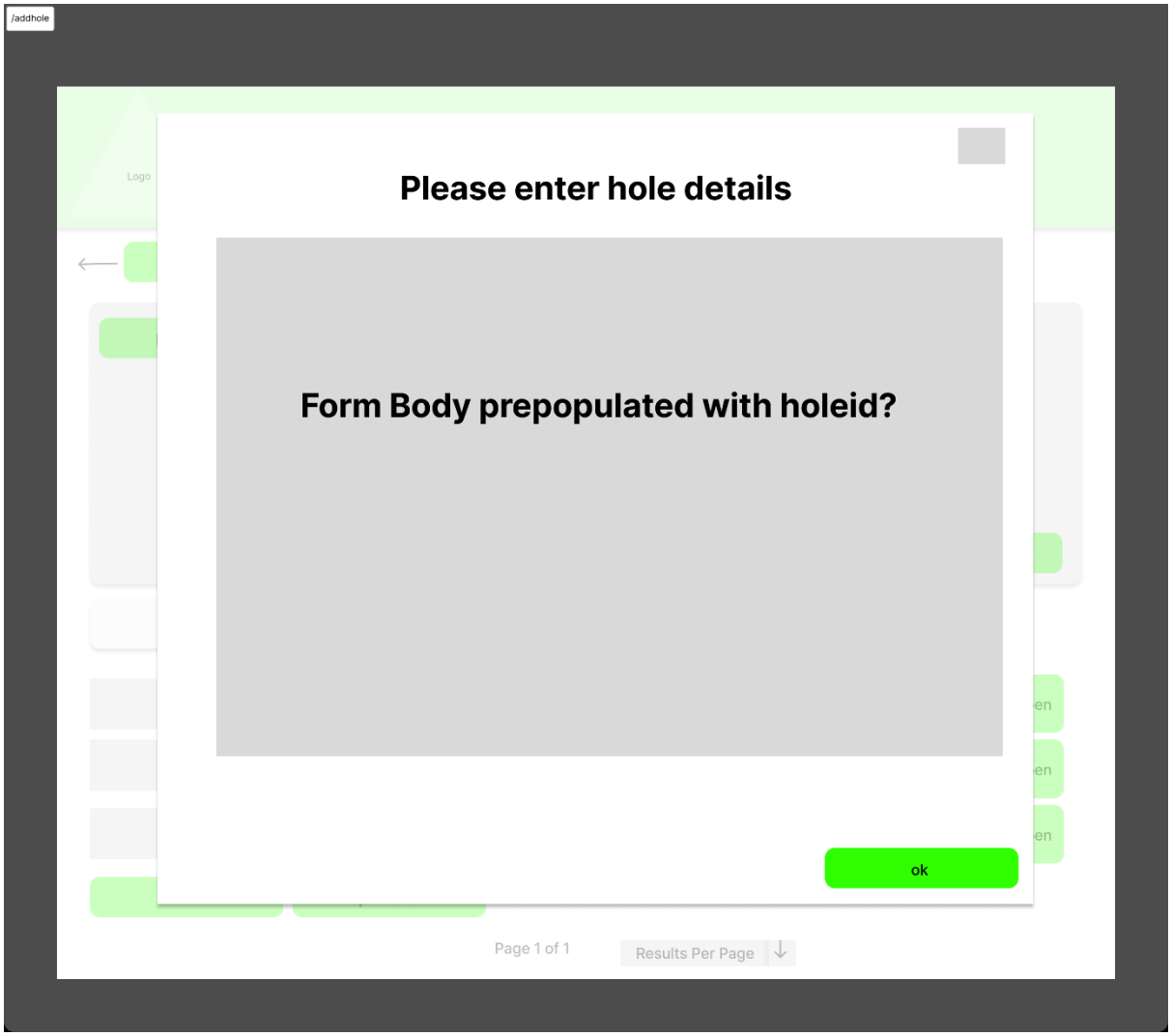
To a separate page

Edit program:



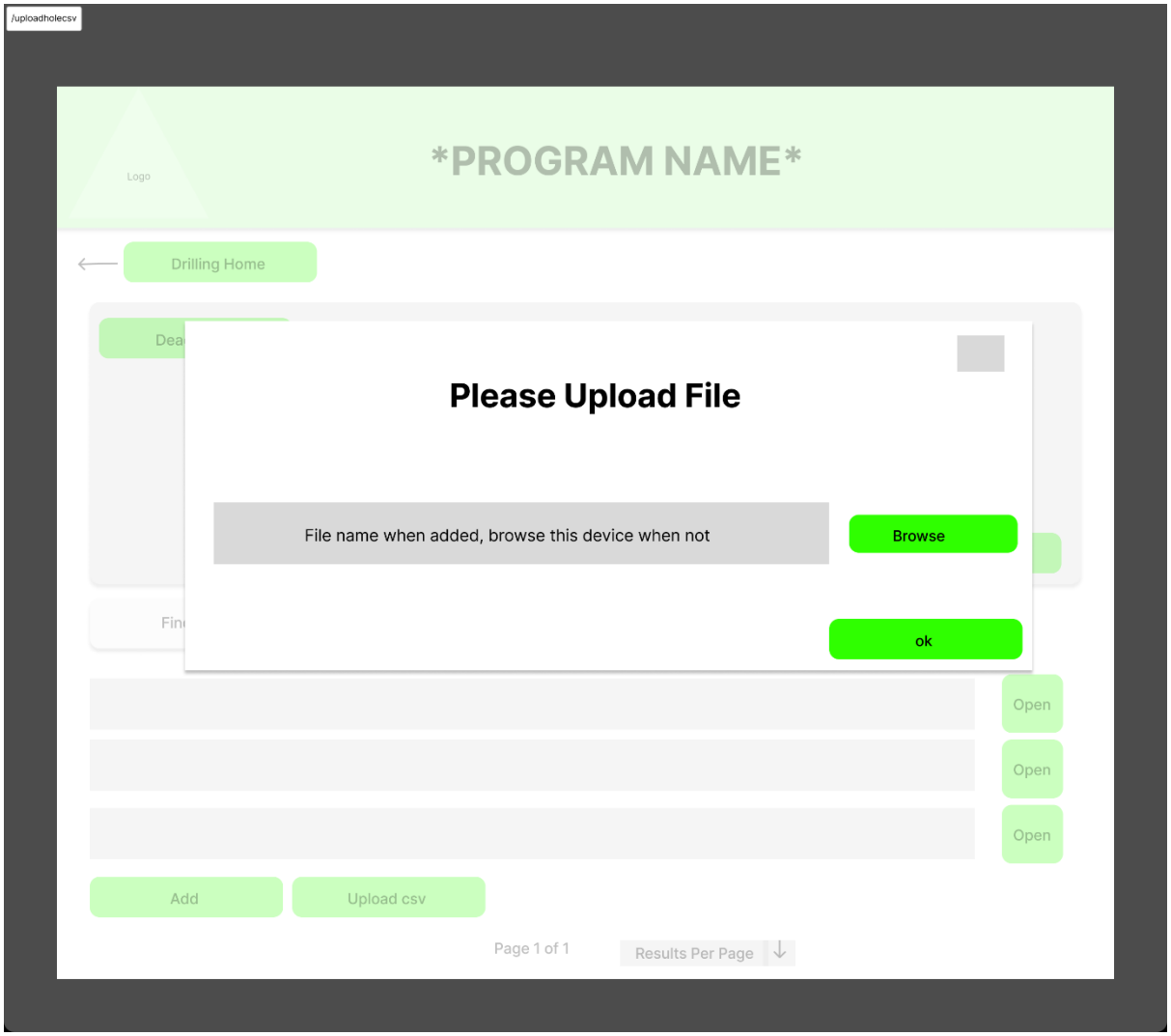
Will need to leverage Bulma modal for this . Contents of form to be decided when database schema completed - as this will

Define what form fields are needed

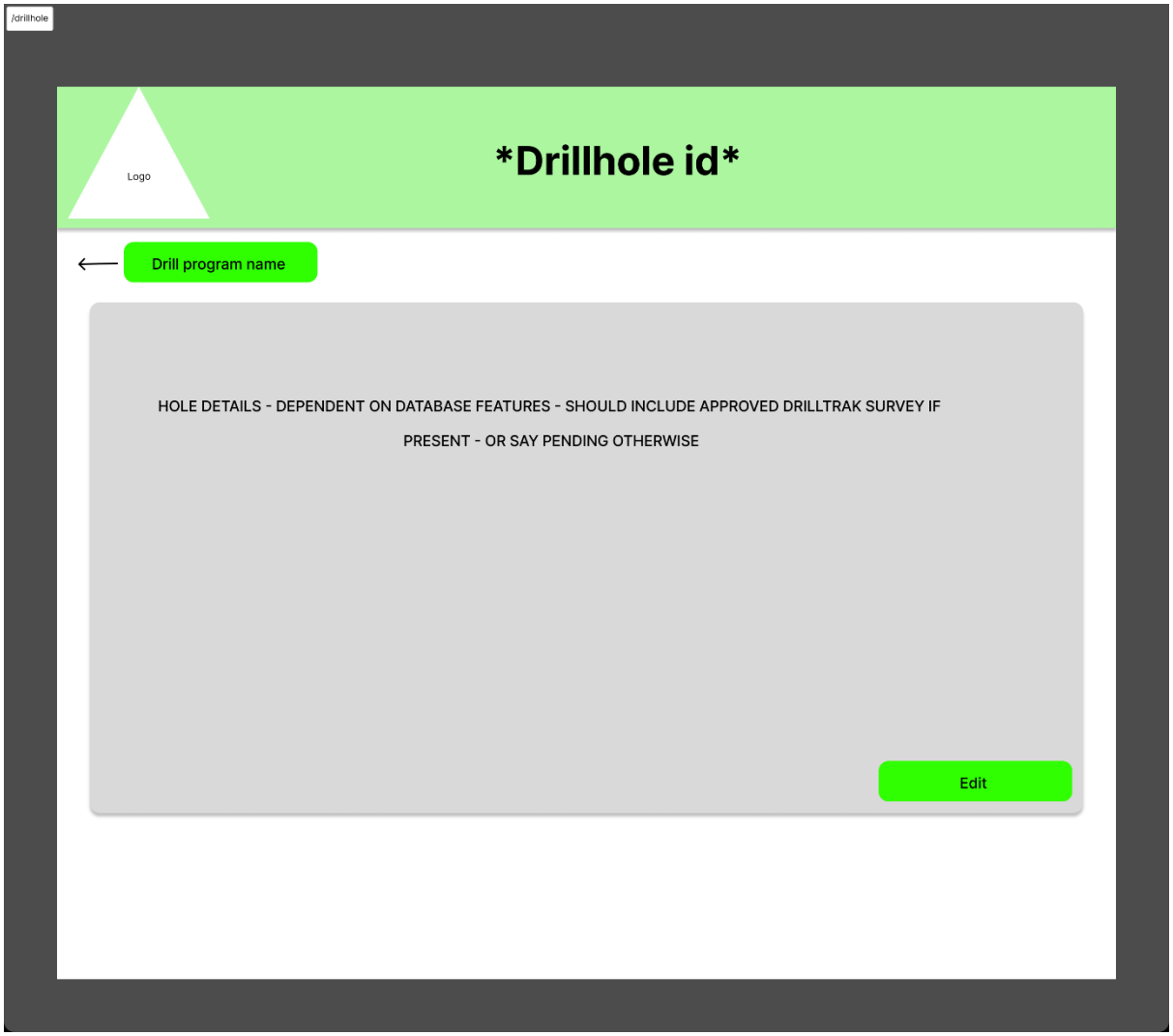


Same Bulma modal - holeid could be prepopulated as this will be a unique key in the database - could populate dependent on the

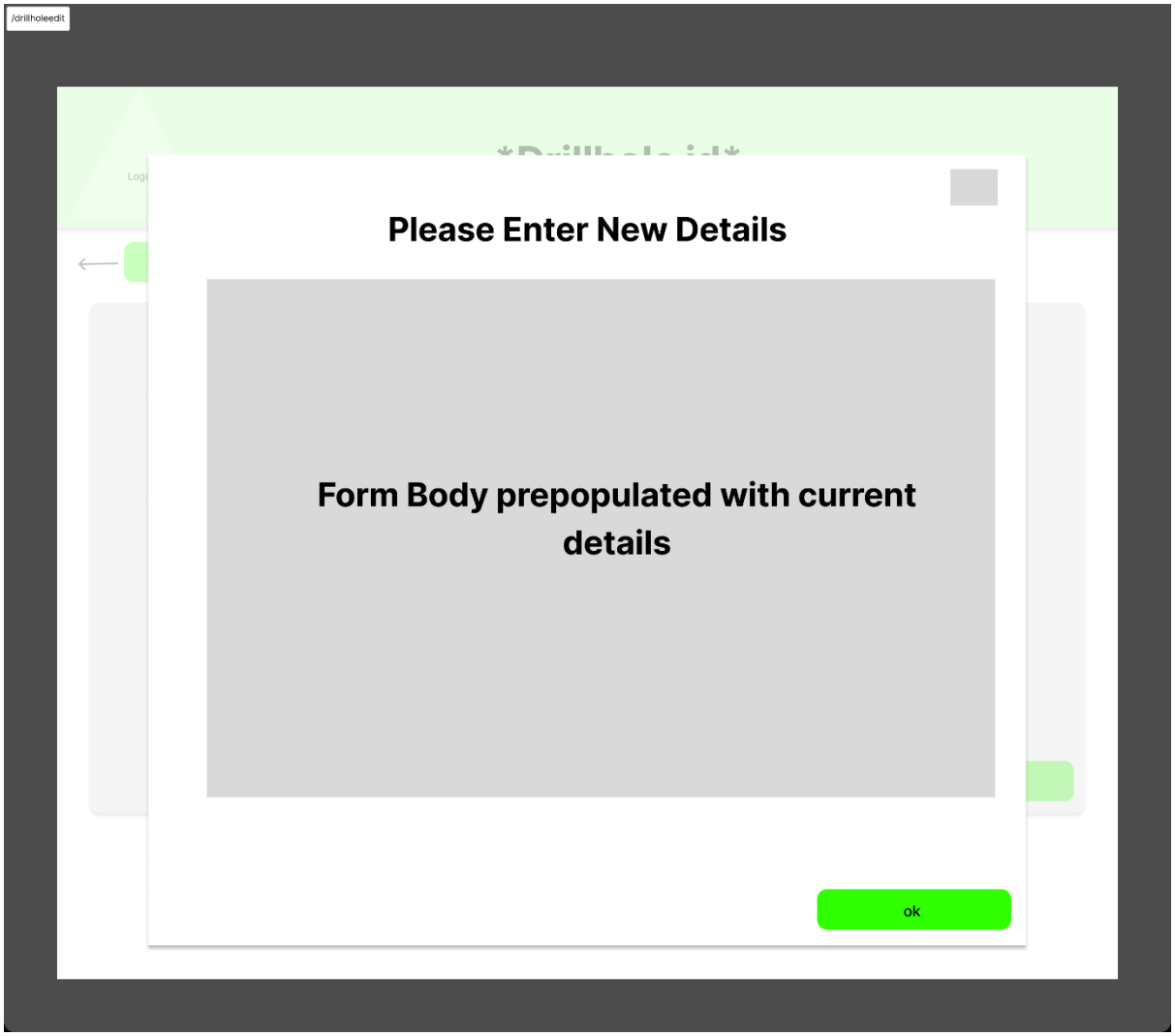
Type of program there is - prepopulate the table with multiple or have this increment - could work this out



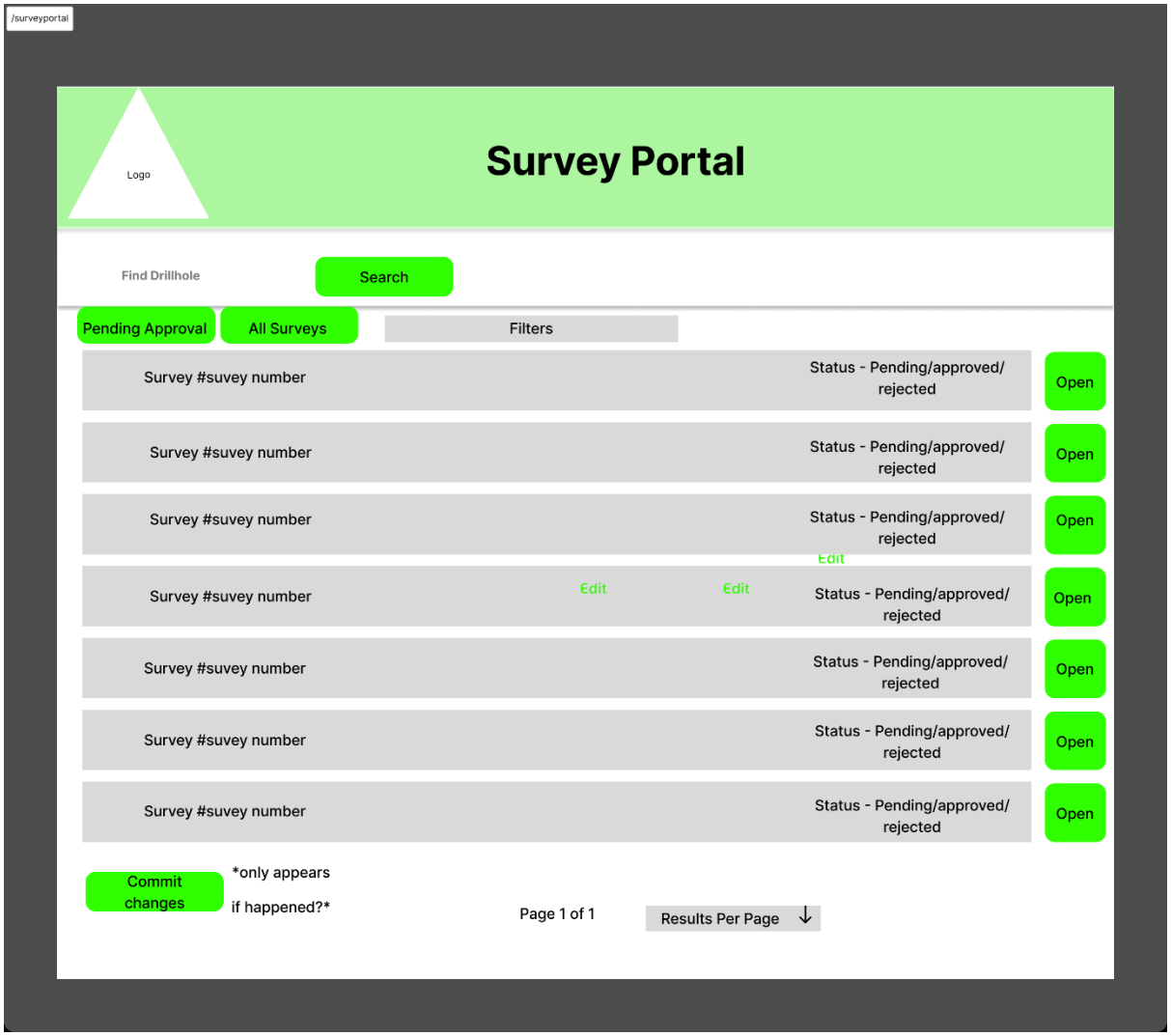
Will need to be able to select a .csv file from the device - this filename should appear when the browsed file is selected ready to be uploaded



Just details - should be pretty easy to implement pretty much



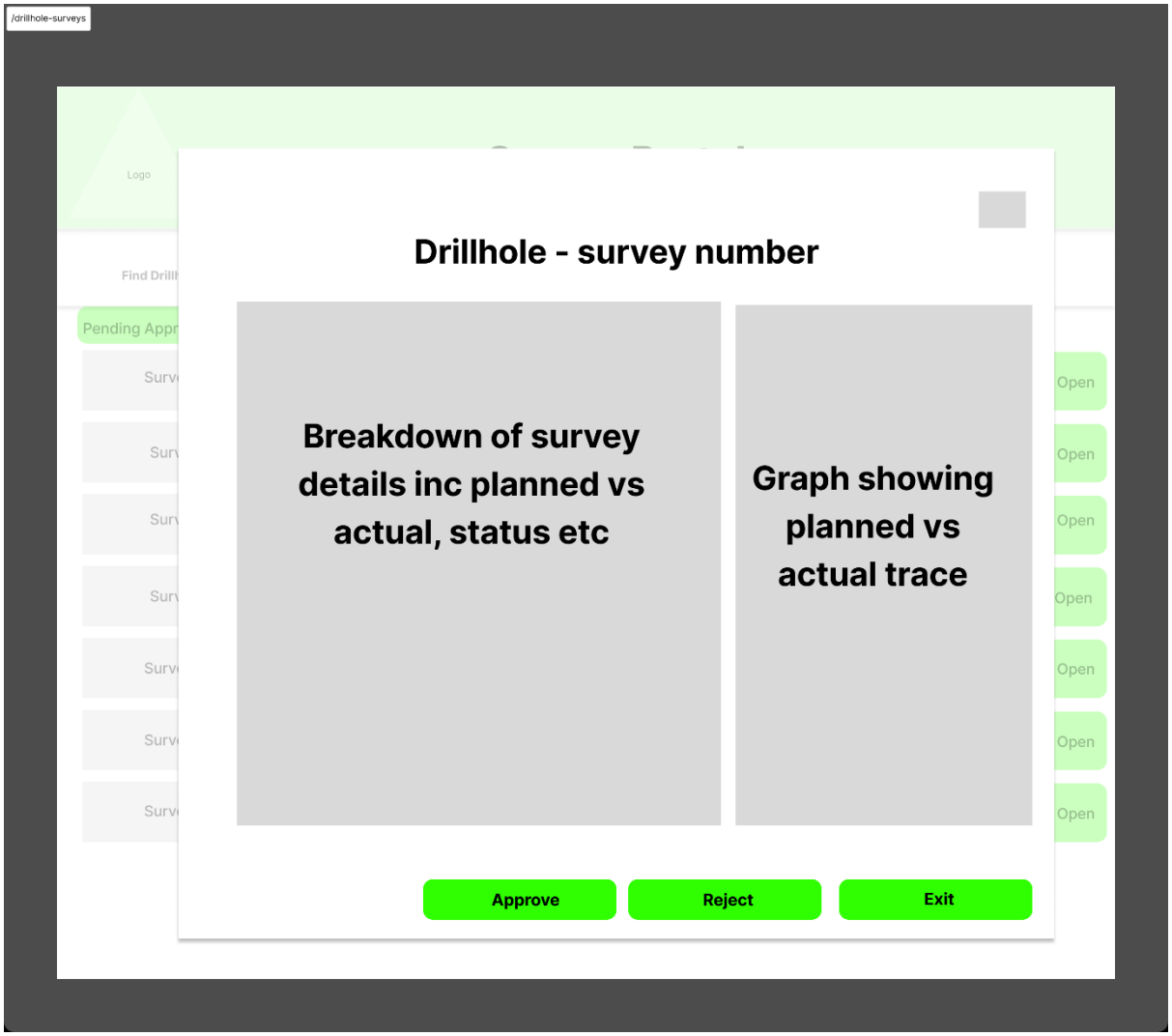
Same modal as editing the drill program details



Lists all surveys taken on all holes - could be automatically filtered by active? - no editing or uploading here? - or should there be as a backup for

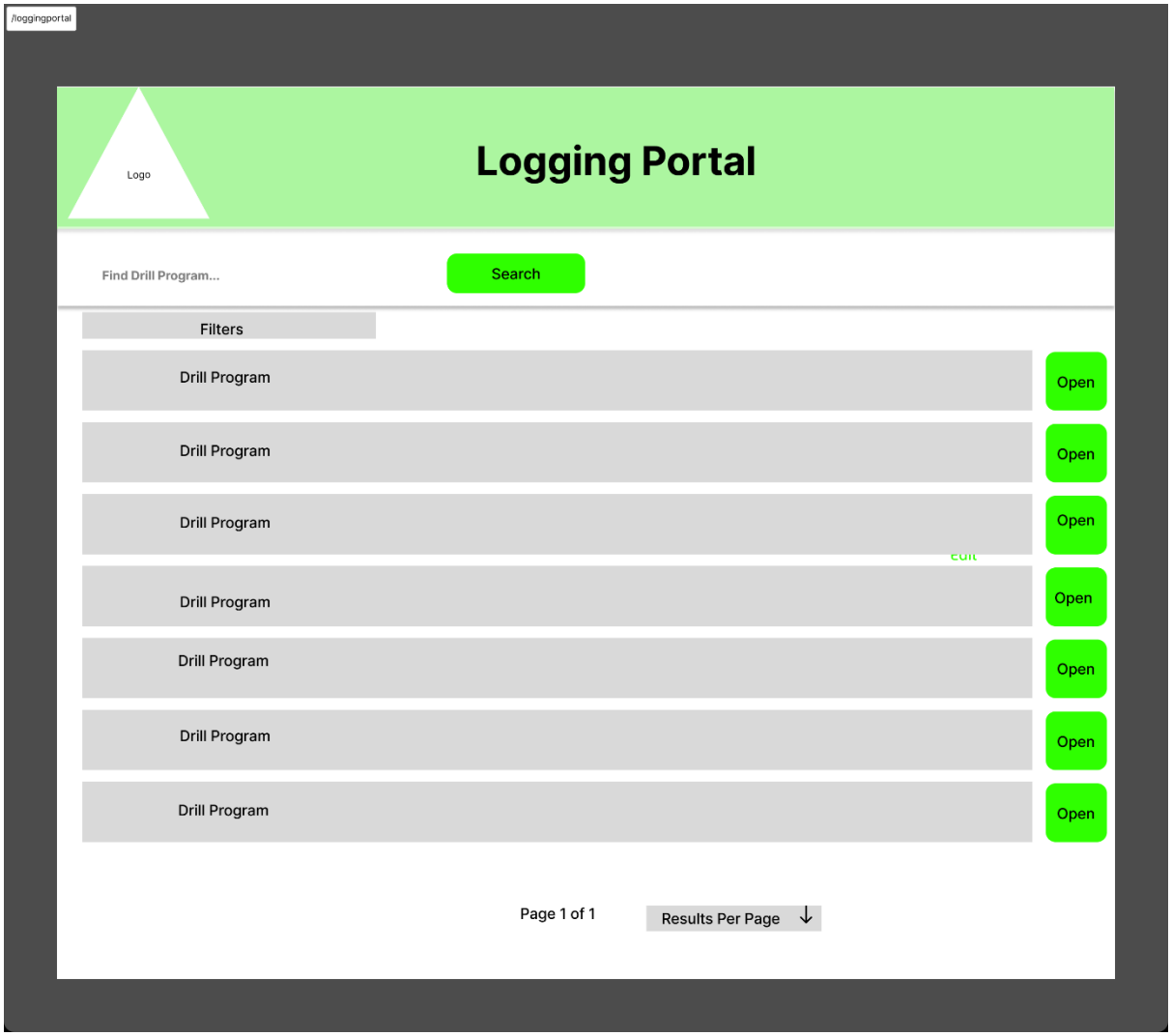
The other surveys - if this needs to be done offline - need to look at offline support for the app? Needs a mechanism for committing to the database

This is a current idea - to be refined during the sprints phase - might not work this way in reality

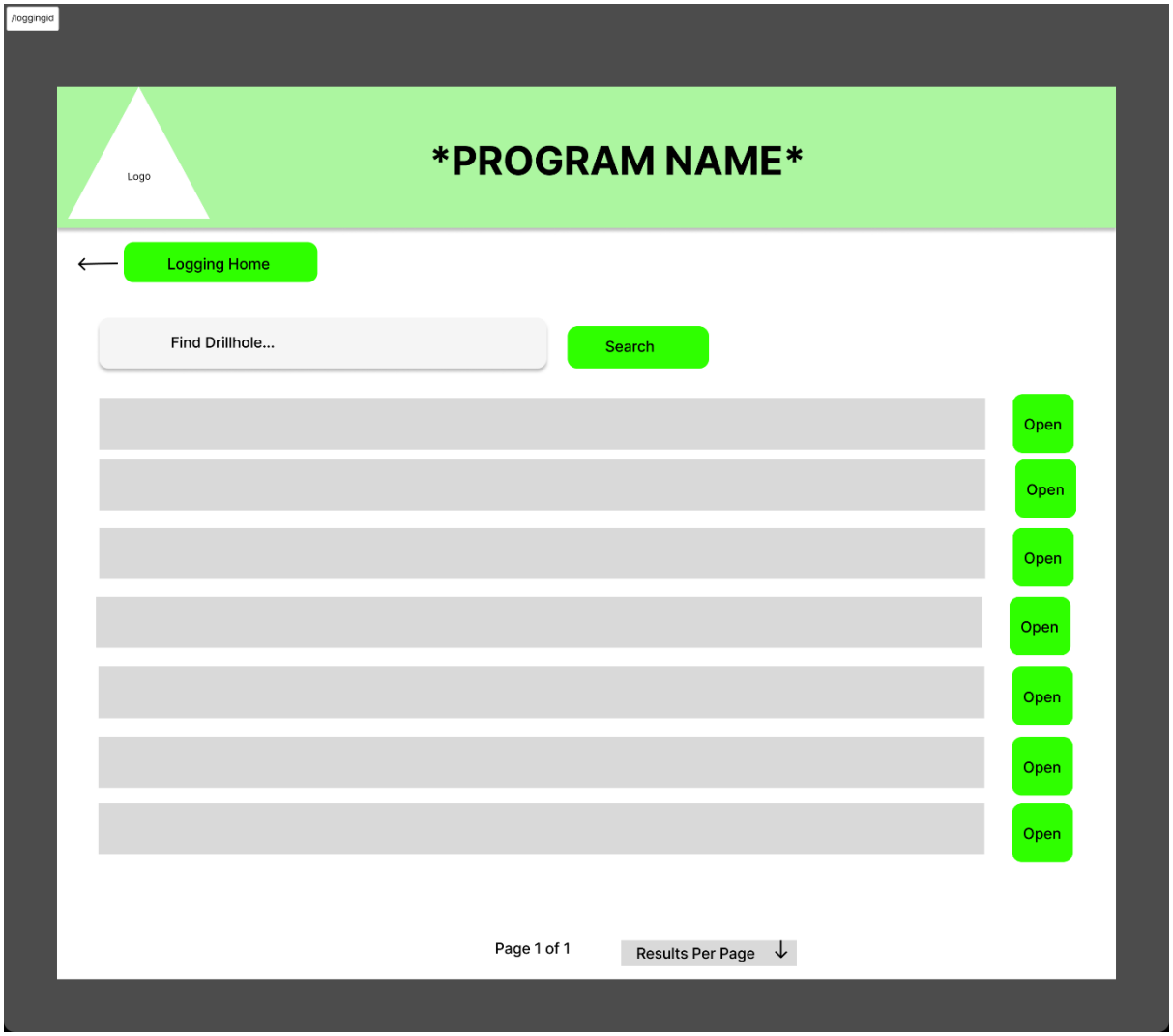


Modal to bring up the details - a graph showing the planned vs actual sprint might be critical to be able to do - approve/reject/exit without doing anything

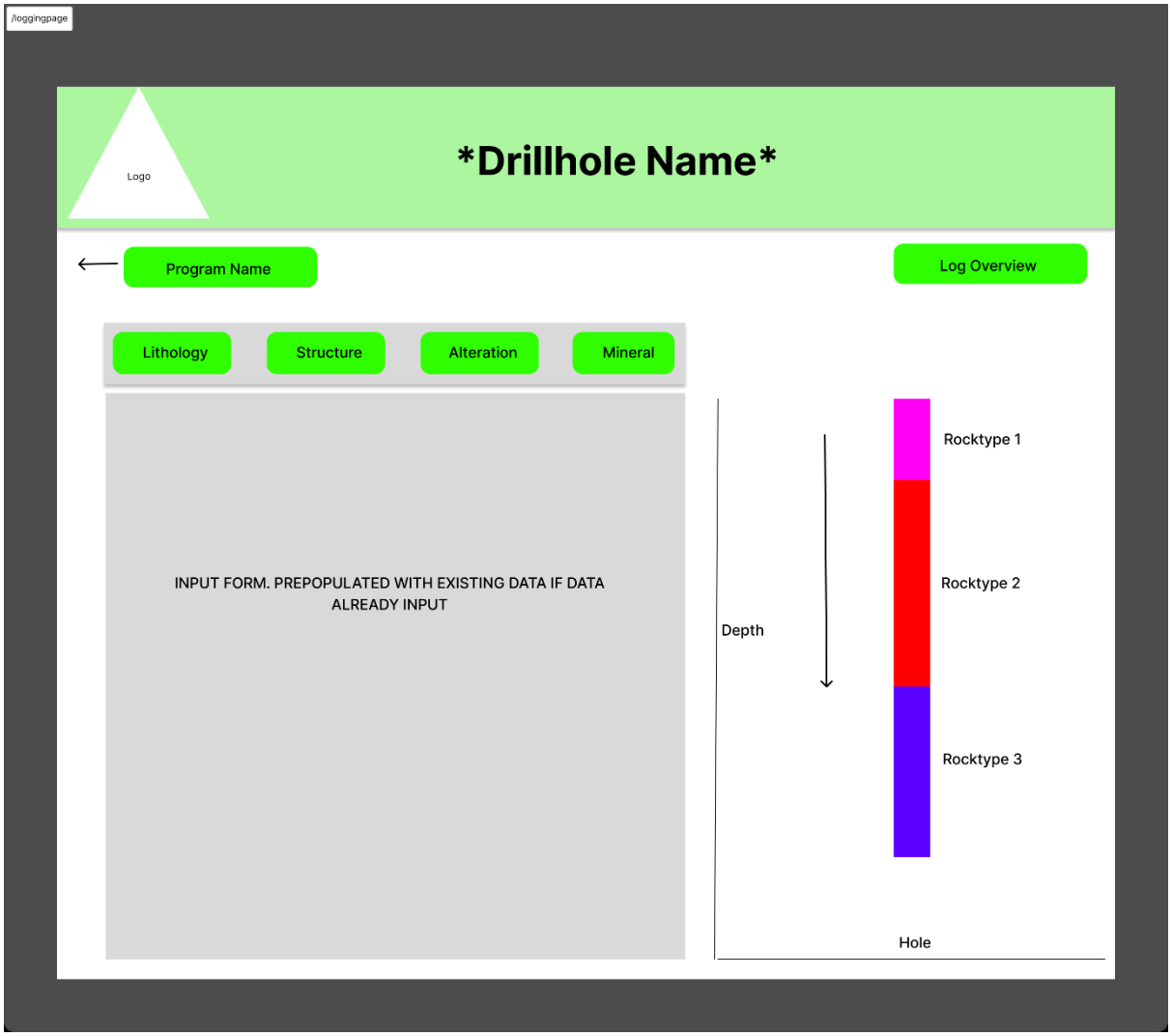
Maybe blur out the other selection and not be active if theres already been a selection made?



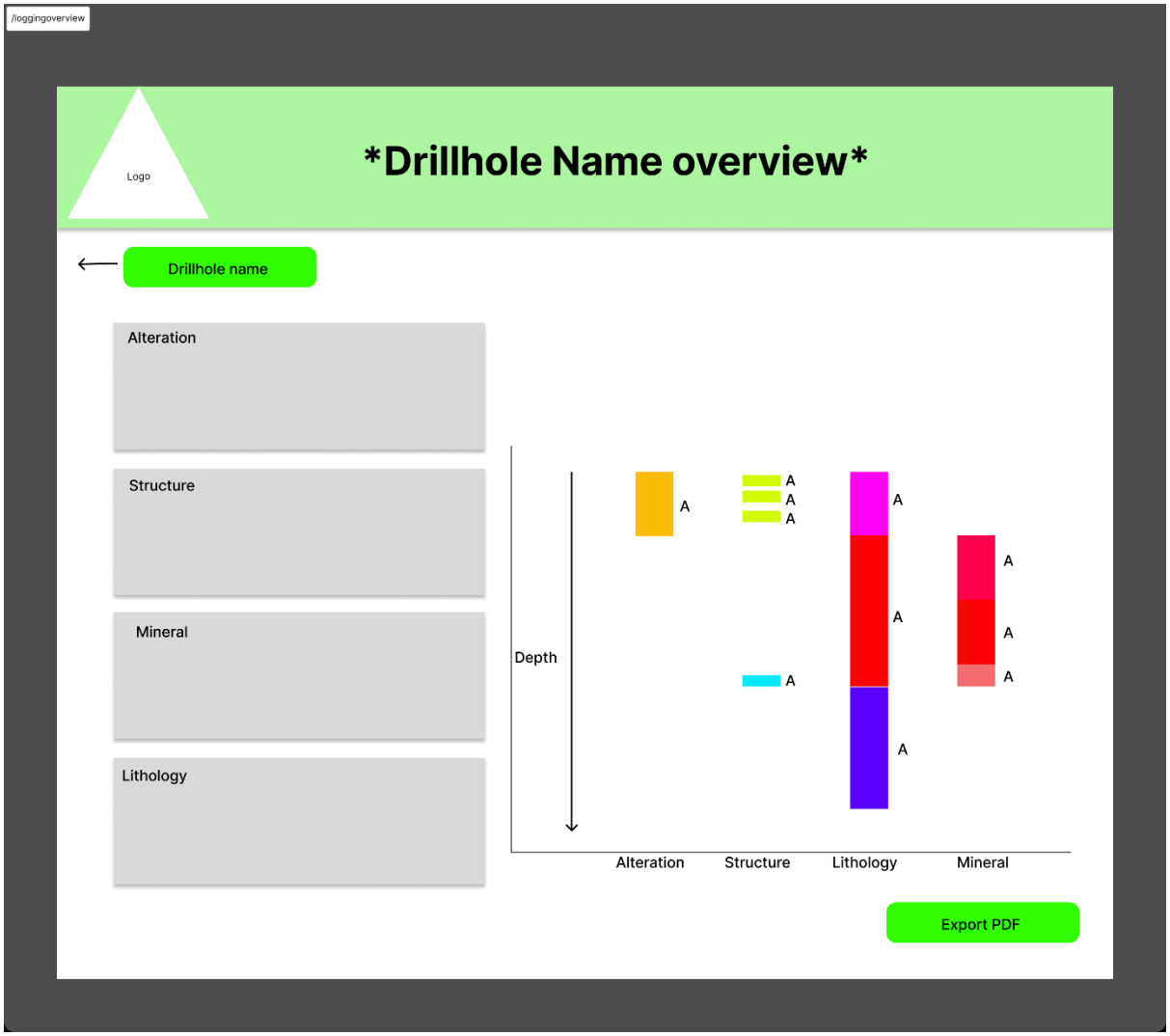
Simple like the other similar pages



Simple like the other similar pages

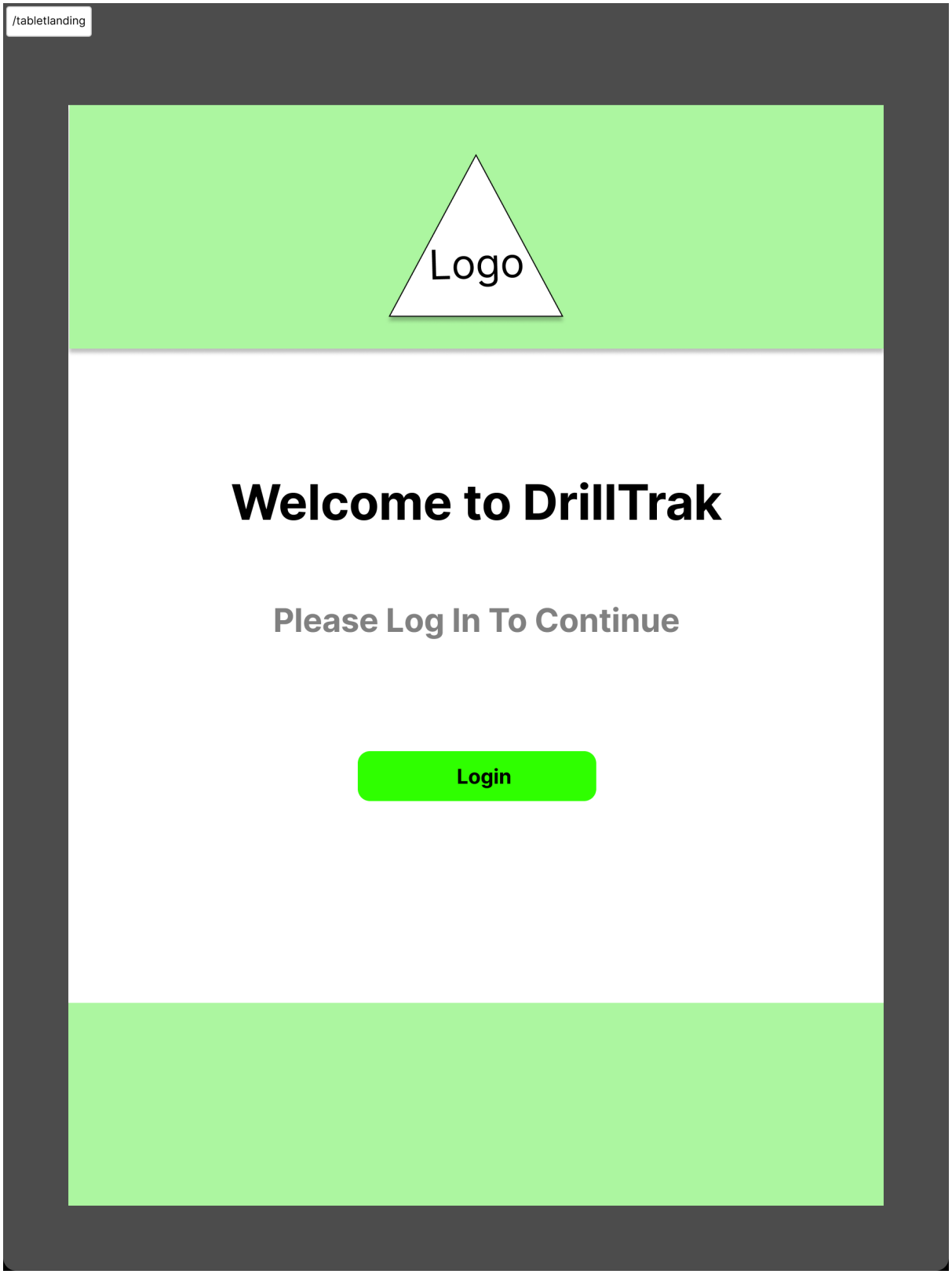


Might need to have the graph dynamically rerender - chartjs looks like a simple way to do this - frappe charts or apache echarts if comolex . Different tabs for each kind of log



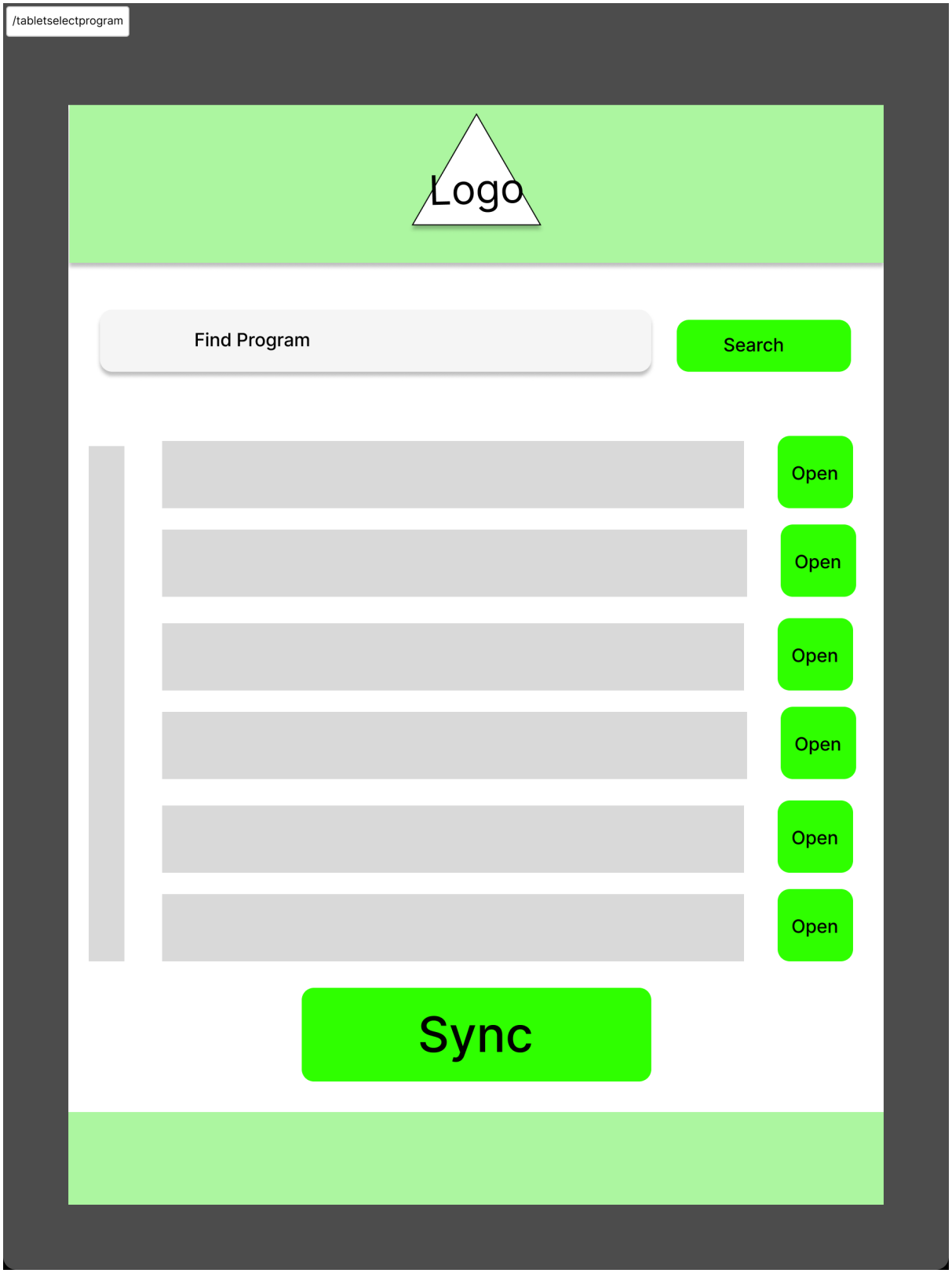
Readable only logs on this page - option to export a pdf if this isnt too much work - nice to have rather than a must have

SURVEY APP



Simple enough login page - like the web app but needs to be scaled down to look right on tablet - as with all

The subsequent pages

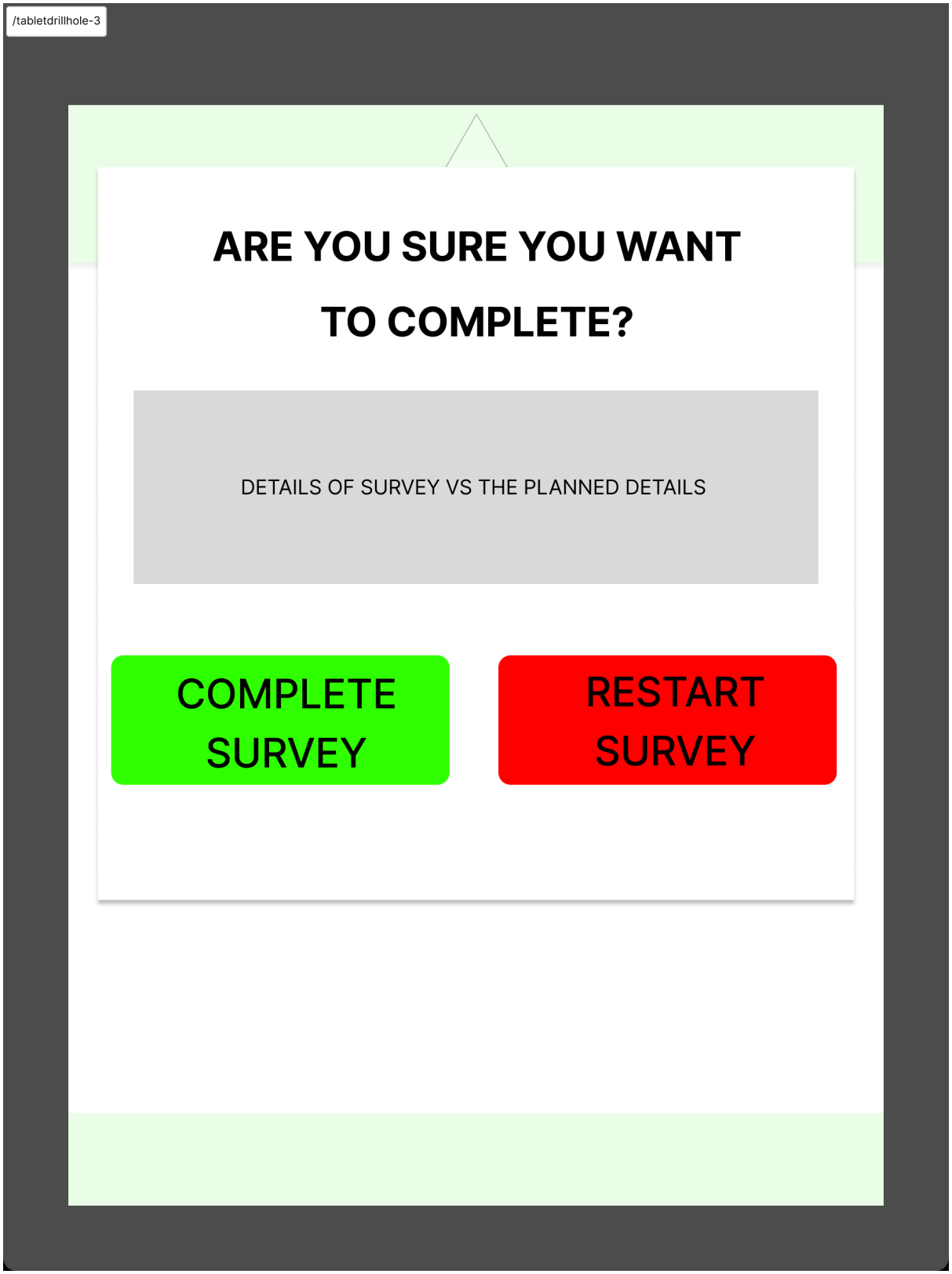


Possible sync button - depending on how we handle offline - could we keep this in a cache - or should the prototype use online only - assess how difficult this is - maybe this goes entirely

Lists drillholes in the program that the user can select

Program details and allows the user to start a survey - straight forward

Page connected live to the survey device - this will be displaying what the tool is generating - maybe using web sockets - needs a data stream - persistent connection - this will be important



Final completion of the survey

9.6. Appendix 6 – Project Diary

19-01-2025

- Installed Postgresql DB server and pgadmin using https://www.enterprisedb.com/postgresql-tutorial-resources-training-1?uuid=867f9c7f-7be7-44ed-b03f-103a0a430d51&campaignId=postgres_rc_18
- Installed python for windows from the microsoft store - also contains the pip library
- Installed Django on the device using https://docs.djangoproject.com/en/6.0/faq/
- Updated backend readme to reflect the installation instructions for windows
- Initialised Svelte project and pushed to frontend repo - installation using https://tutors.dev/lab/wit-hdip-comp-sci-2024-full-stack-1/unit-6-sveltekit/topic-19-svelte-routing/side-unit/book-2-donation-svelte-1/01

20-01-2025

- Created my drilltek database using Pgadmin 4
- Created the user table as defined in the database design and inserted a mock user to use for testing later on - had to create an enum type rather than a native enum datatype in mysql - found here https://www.postgresql.org/docs/current/datatype-enum.html
- Created database repo and pushed starter code up here

21-01-2025

- Unable to complete any functionalty this day, no internet connection

22-01-2025

- Created branches on frontend and backend to accommodate login feature which will be the first feature for the application. Also created a development branch

23-01-2025

- No development today, focused on certification

24-01-2025

- No development today, focused on certification

25-01-2025

- No development today, focused on certification

26-01-2025

- Changed password hash to not null in user table - going to create a user with no password to start with - supply the email and get the user to create a password when they first login
- Django rest framework seems to be the way to go - must look up how to use this with postgres to get the first endpoint up and running https://medium.com/@michal.drozdze/setting-up-a-django-api-with-django-rest-framework-drf-a-beginners-guide-cee5d61f00a6
- Follow this tutorial potentially https://sourcery.blog/how-to-build-api-with-django-rest-framework-and-postgresql/ - for getting started with Django and postgres

27-01-2026

- Initialised Django Project
- Initialised Django app
- Installed Psycopg2-binary library for connecting to postgres
- Created postgres user for django app
- Updated settings file to include django app and rest_framework
- Updated settings file to include database connection details
- Granted necessary permissions to Django app to allow connection
- Tested connections success by running migration and using inspectDB function
- Installed python-dotenv

- Abstracted db password to .env file
- Pushed to DB main branch
- Pushed to backend login branch

28-01-2026

- Decided to drop user table and have this created by django instead - makes more sense this way rather than referencing an existing table
- Created user model to be uploaded
- Overwrote the previous migration - tested, this is up on the postgres database now
- Created test api endpoint to test and understand how this works using https://dev.to/entuziaz/django-rest-framework-with-postgresql-a-crud-tutorial-1l34
- This works by creating a model, running migration to transfer to the connected database, create a serializer to convert the data, create a view that represents the endpoint, create a new url for the endpoint, reference these inside the main urls folder to allow this to happen
- Next thing is to create a login endpoint so that I can start working on the frontend page and close out this item. Likely for stuff to be pushed into next sprint as getting to grips with the technology

29-01-2026

- Decided to use viewsets + @action for developing the api endpoints based on a conversation with perplexity where it advised me that the most efficient method out of the 5 possible methods would be this one - where the level of flexibility for me would be high
- Decided on methodology for register and login for the frontend and how this might guide the backend?

30-01-2026

- No Work completed

31-01-2025

- No work completed

01-02-2025

- No work completed

02-02-2026

- No work completed

03-02-2026

- No work completed

04-02-2026

- No work completed

05-02-2026

- No work completed

06-02-2026

- No work completed

07-02-2026

- No work completed

08-02-2026

- No work completed

09-02-2026

- Planned interim report

10-02-2026

- No Work Completed

11-02-2026

- No Work completed

12-02-2026

- No work completed

13-02-2026

- No work completed

14-02-2026

- Interim report completed

15-02-2026

- Interim report formatted and submitted

16-02-2026

- Finally settled on Class + Action for implementing the end points. Finalised checkUser and setPassword actions to allow for user to set first password. Will need to be refactored to store a hash of the password instead of a hard password- will research this. https://www.django-rest-framework.org/api-guide/serializers/ used to research how to use serializers, https://www.sololearn.com/en/Discuss/2889482/how-to-split-a-dictionary how to split out dictionaries from serialized input, https://medium.com/@altafkhan_24475/patch-method-of-apiview-in-django-rest-framework-e7c0d574a47f for how to do a patch request using class + action used to define how to do these methods ,

https://www.w3schools.com/python/python_dictionaries_access.asp how to access keys in a dictionary, https://docs.djangoproject.com/en/6.0/ref/request-response/ to understand request and response objects, AI response used here in AI page of notebook, https://www.geeksforgeeks.org/python/function-based-views-django-rest-framework/ used to investigate options for views in django rest framework

17-02-2026

- Managed to hash passwords using django library - did this based on a conversation with perplexity AI found "C:\Users\joshb\Documents\hdipComputerScience\Project\AI Conversations for final report\I am using Django rest framework to create an API..pdf"
- Method for checking password now checks if password exists and hashes password before serializing to database
- Created skeletal login endpoint, which checks password in reverse of make_password also learned about from above perplexity conversation

18-02-2026

- Investigated the use of JWTs using django and landed on Django-rest-framework simple-jwt.
- Learned how to use using "C:\Users\joshb\Documents\hdipComputerScience\Project\Ai conversations\How can I implement JWT in Django Rest Framework..pdf" with perpelxity
- Populated this into my login function. On successful credentials, now vends a short lived access token and a longer lived refresh token, trade off between security and UI as access token alone would time out but cant be revoked. Refresh token can be revoked if the access token is stolen - trade off
- Need to edit check user function to actually check if users password has been reset yet - use an ENUM on the database for this - will need another migration for this

19-02-2026

- No Work completed

20-02-2026

- No Work Completed

21-02-2026

- No Work completed

22-02-2026

- Created Boolean signedUp to control flow to either set password or proceed to login. Created in models and migrated to database. Modified checkUser function to take this into account and either respond to change password or procced to login.
- Installed Axios in frontend and prepared the frontend application with structure to complete the login feature. Added bulma from content delivery network
- Did some refreshing of svelte structure before starting frontend pages

23-02-2026

- Created landing page + modal to open first sign in page
- This will need to do the check user and redirect to certain pages based on this

24-02-2026

- Created service for checking user against input email and then redirect to wherever this is required - grafted up mock pages for resetting first time password reset or redirecting to login.
- May need to change the backend to require the old password to be input for the resetting of the password also - may be able to re-purpose this for a user area later on also.
- May also need to pass the email as a prop into the component also or find a local storage solution to persist the email so that user does not need to enter email twice as this is required in the API request - either hide in local storage and have nothing entered by the user or have this pre-filled in the form - will decide on this solution next development session

25-02-2026

- Created session type for modelling session objects when tokens are returned from API on login.
- Created service methods for accessing setPassword and login endpoints.
- Discovered Axios does not like 204 status return codes so replaced these with 200 status codes
- Created parameterised routes for logging in or setting passwords depending on the result of the check user. Email stored in the URL params instead of getting user to retype email. With server actions for logging in and setting password corresponding to the API service methods
- Tested setpassword functionality and appears to work. This redirects user to the login route if successful or back to dashboard if not. Could probably do with error messages to the user but this can be dressed up later once functionality is in place
- Used perplexity for figuring out an alternative to localStorage when carrying out the actions on the server rather than in the client. Also used to debug an error with the wrong URL being constructed and to validate areas of my code to track down the issue. Conversation can be found here: "C:\Users\joshb\Documents\hdipComputerScience\Project\Ai conversations\how can I write to local storage in svelte.pdf"

26-02-2026

- Created server method to login. Calls api service then wraps returned session data from service into cookie for validation
- Created dummy layout page to be manipulated as required later on with content being injected into slot tag
- Created layout server file to grab cookie if exists and return session from cookie to all subsequent pages if exists - is being used to control auth into the pages that require auth after login
- Created main portals page as the gateway into the app after login. Buttons currently go nowwhere, will be defined during other user stories
- Main portal gated by page server load - taking session details from layout. If this exists the user enters the page else is redirected to the landing page

- Added logout server function and page. When user hits logout - cookie is cleared and authorization header for axios is cleared. Probably not safe to just do this in production. Added a potential story to list to create a system for blacklisting the refresh token before logging out and deleting the cookie to prevent stealing
- Card styling taken from bulma https://bulma.io/documentation/helpers/flexbox-helpers/#justify-content
- Drop shadow override and clearing auth header from axios learned in conversation with perplexity "C:\Users\joshb\Documents\hdipComputerScience\Project\Ai conversations\how do I protect routes in svelte.pdf"
- Need to merge login branches into development and checkout new branch for next story

27-02-2026

- Merged backend and frontend login features into development branch
- Created new branch for drillingportallanding feature

28-02-2026

- Learned how to gitignore and that I should gitignore pycache file
- Created drill program model for database
- Created serializer for drill program
- Created new get method to retrieve all drill programs
- Created endpoint URL for method
- Attached auth to new endpoint, JWT auth not working for protected endpoint due to lack of user_id resolvable on decoding JWT, need to create a django user for the application - research how this is done

01-03-2026

- No work completed

02-03-2026

- No work completed

03-03-2026 - 04-03-2026

- Created migration for creating a database service user for the application. Now generating tokens on login is for this user as this is a requirement of django simplejwt
- Created a route for generating a new access token based on a valid refresh token which is not protected but requires a refresh token to be able to do. Access tokens invalidate after 15 mins, refresh token after a week but could be blacklisted and application should generate a black list later on
- Created SSR infrastructure for drilling portal where data is loaded and passed in on page load.
- Experimented with may ways of carrying out refresh token function. First attempted a utility for accessing this during the load but could not access status codes in a type safe way. Then tried to use axios interceptors to run the process if a 401 error was intercepted by axios but this would only work with a client rendered setup which this page does not carry out and I don't wish for the project architecture. Settled on including a test check for token validity in the server load for the layout and carrying out the refresh if a 401 is hit before returning a valid session. This was investigated throughly using perplexity AI rather than slowly google searching. Could probably replace the test endpoint with a dedicated test endpoint in the API at a later stage, noted for generated issues to get to at the end
- Created drilling portal landing page populated with current programs In the database. Current functionality does not exist, to be covered in subsequent issues
- Completed function - merged into development branch and closed off
- Most of work done this day was done with conversations with perplexity "C:\Users\joshb\Documents\hdipComputerScience\Project\Ai conversations\each block should have a key error svelte.pdf"

04-03-2026

- Created and tested create endpoint for drill programs

05-03-2026

- Added userid to be returned as part of the login process to be stored in session in backend
- added modal for adding program. User will put in programid, orebody, location and target with other fields being input on the server level and in the backend
- Added userid to the session type and is now defined when cookie or new cookie is set to allow for use in create drillhole and other upcoming methods
- Created new type to handle commited drill program subtracting dates as these are added automatically on creation of the record in the backend. Created service function to create drill programs. This returns a status after adding. Created server action for this. Server action refreshes page if drill program successfully created. If not successfully created, checks for a 401 response code returned from service function, attempts to refresh token using previously created util function (which now returns session if successful) and uses this session to retry the request, if unsuccessful logs the user out. Had to return this session as could not re-get the new cookie session details within the running function.
- Used perplexity to understand how to extract axious error messages on api result to then return and use as logic to define if a refresh is needed because access token is expired
- Used perplexity to debug my utils function as this was not working
- Perplexity conversation here - "C:\Users\joshb\Documents\hdipComputerScience\Project\Ai conversations\async createDrillProgram(token_string, program_Add.pdf"
- Create drill program issue completed

05-03-2026

- Created endpoint for retrieving a drill program by program ID. Created an endpoint url for this and tested. Test successful and appears to behave as expected in postman.
- Changed this endpoint to reference a query parameter as body is ignored in get requests in django, investigated this on perplexity to learn how this works ("C:\Users\joshb\Documents\hdipComputerScience\Project\Ai conversations\what is href without resolve.pdf")
- Created Parameterised drillprogram page and server file parameterised

with programid (learned about with perplexity "C:\Users\joshb\Documents\hdipComputerScience\Project\Ai conversations\what is href without resolve.pdf") . Edited drillingportal links to open drillprogram to

include this parameter.

- created service function to call get program by id endpoint. Call endpoint in page load using params from page url. ("C:\Users\joshb\Documents\hdipComputerScience\Project\Ai conversations\what is href without resolve.pdf") Return program and session to page on load.
- Basic pagestructure setup with tabulated details (https://bulma.io/documentation/elements/table/) about the program and dummy add drillholes and edit program buttons to be covered in there respective issues.

06-03-2026

- Created api method for editing drill program via patch request.
- Added new serializer in attempt to fix edit program issues serverside. editprogram function was getting confused and executing a create record instead of update due to input of id in payload so removed. Should likely prevent from changing the PK anyway due to the amount of records that will need to change as FK in other tables. Function now works as expected. "C:\Users\joshb\Documents\hdipComputerScience\Project\Ai conversations\can you pass optional props to a component in svel.pdf"
- Created modals for editing a drill program and adding drillholes in drill program details page. Created edit program form which prepopulates with current details via component props.
- Created new edit drill program form component that takes a program as prop and prefills with current details. Created a new type to reflect the edit drill program details. Created api service function to service this and server action for submitting request edit form submitted from modal. Initially added programid as a field that could be edited but this caused interpreter issues in the backend and resulted in the creation of new programs instead of editing exisiting so this was removed.
- Completed feature

06-03-2026

- Added drillhole model and migrated across to postgres DB. Created serializer to adding drillholes. Created method for adding drillholes and associated API endpoint. Tested with postman and appears to be working as anticipated.
- Need to think about how to update total meters on drillprogram or drop entirely and just have as a visual in the application running off a function?- may require editing otherwise
- "C:\Users\joshb\Documents\hdipComputerScience\Project\Ai conversations\how could i have an autoincrementing string in sql.pdf"

07-03-2026

- Added drillhole model and migrated across to postgres DB. Created

serializer to adding drillholes. Created method for adding drillholes

and associated API endpoint. Tested with postman and appears to be

working as anticipated.

- had to create a new serializer, method and URL for returning drillholes

by program before creating the mechanics for adding a drillhole to a

program. Want these to display on adding a program for testing purposes.

Attempted to use objects.get() for retrieving drillholes from the model

table but this did not work. Research using perplexity "C:\Users\joshb\Documents\hdipComputerScience\Project\Ai conversations\@action(detail=False, methods=['get'])_ def get.pdf" pointed out that

.filter is the correct method for this so will use this for retrieving

many objects going forward.

- Created new type for drillholes. Created new service method for

retrieving drillholes based on program id and included this in page load

for drill program details so that drillholes as well as the program

details are returned to the page. Added to page to include list of

drillholes at base of page. Could separate this listing component out to

a separate component for reuse in pages with that kind of data, have

made note of this as improvement to codebase later on. Created new

drillhole form component for use in add program modal. Created service

method for adding drillholes and new server action referencing this. In

server action had to do some data conversion due to the fact that forms

only return strings before passing the drillhole to server function "C:\Users\joshb\Documents\hdipComputerScience\Project\Ai conversations\@action(detail=False, methods=['get'])_ def get.pdf".

- Had to look up parsefloat method for this https://www.w3schools.com/jsref/jsref_parsefloat.asp
- Had to look up input types for this https://www.w3schools.com/html/html_form_input_types.asp
- Had to reference https://bulma.io/documentation/form/select/ for a select for the type of drillhole field for the user to input into

07-03-2026

- Created method to retrieve hole by id using drillholeserializer. Created and tested endpoint in postman and works as expected.
- Created protected method to get user email by their ID for use in pages

displaying drillhole and program information. Created protected endpoint

to access this.

- Created parameterised route for drillhole. Edited drill program page to

reference this for each hole. Created server rendered page for drillhole

retrieving the drillhole from created service method. Contains table

displaying data and dummy edit button to be worked on next issue.

Noticed that planned by is showing person logged in for both drillhole

and drill program pages. To be fixed

- added service method for retrieving user email by id. Added this to drill program and drill hole pages based on the userid on either of these retrieved. Had to put in some if logic to handle the inital call getting these returning empty objects

- Had issues with using parameter inside page load service call for drillhole Id as this is a numeric. Realised after research with perplexity that this would have to be converted to an int first and fixed this.
- Had to research in perplexity why I might be returning an unknown when trying to access userid on calls in drillhole and program. Realised required some logic to check if the attribute existed on the returned program or drillhole first before calling the get user email by id
- All research done with perplexity "C:\Users\joshb\Documents\hdipComputerScience\Project\Ai conversations\export const load_ PageServerLoad = async ({ paren.pdf"
- Feature completed

07-03-2026

- Created endpoint for editing drillhole based on id and tested with postman, works as expected.
- Created edit drillhole component with props for a drillhole to prepopulate the fields. Created modal in drillhole page for displaying this field. Dummy submit button currently present to be populated with edit method.
- Created server infrastructure for edit drillhole functionality.
- Feature completed

08-03-2026

- Merged development branches into main for new version v1.0.0 and closed issues related.
- Moved new issues into backlog

09-03-2026

- Created dummy files for Modal and list items components which will be used to refactor the code later on as there is some repetition that could be refactored out. Created Logging portal landing and drill program logging pages and populated these. Very little work due to simularity of pages to others already developed

which points to the refactoring that could be carried out later on during improvement of code base.

- Completed logging portal landing feature and open drill program logging feature

10-03-2026

- Created and migrated lithlog table
- Created serializers for lithlog read and adding lithlog. Created endpoints for adding a lithlog and retrieving lith logs by hole id as a precursor to creating the lithlog landing page. Tested with postman and appears to work as anticipated

11-03-2026

- Created service method for retrieving lithlogs from DB and created type

for modelling lithlog record. Created Server function for lithlog page load using holeid param. Initially felt to do separate pages for each log but will now investigate going down the route of conditional rendering so the user never leaves the page. Set up a nav for this and conditional blocks for the page based on corresponding booleans. Currently log info is displayed in a table but this will be changed to a tabular form. Will work on this next before looking at Graphs. These will be covered in the issues related to these different logging areas.

12-03-2026

- Created and migrated models for Alteration logs, Structure logs and mineral logs.
- Created get and post serializers for alteration, structure and mineral logs. More will need to be added should I put an option in to import an assay file.
- Altered texture list for mineral log to include trace and barren as options. Created end points for adding and retrieving alteration logs, structure logs and mineral logs by holeid. Created URLs for endpoints and tested all endpoints and are working as expected.

13-03-2026

- Created types for alteration log, mineral log and structure log. Created get methods in drill tek service for retrieving these logs by holeid.

14-03-2026

- Experimenting with log table for the overview. To be completed as mineral log was failing to render. To investigate shortly.
- Fixed rendering of mineral table in log summary page. Made tables

scrollable. Next step to create rendering logic for landing and overview

so user can switch between these.

- Created open methods for different pages. These set conditonal rendering

variables to false for all sections not associated with the button and

the one associated to true causing only that section to be conditionally

rendered. Also Sets active state to value associated. This is bound to

each button so that when value for the button matches the active

variable then the button turns black as 'selected'. Set tables on

overview back to height and to overflow so these scroll using infile css

style tag.

- Attempted several methodologies for creating a stacked bar graph with

gaps to no avail. Gantt chart was suggested for this but the complexity

of this was too much for what wanting to achieve. To achieve what this

requires would likely require creating a custom graphic as bar charts do

not fit the bill for this purpose. Used frappe charts as a work around

to create the graphic which will only be displayed on the overview page

in the application. Can be revisited if there is time.

- Reconfigured page layout to have fullwidth tables for all pages aside

from overview which also contains the graph.

- Perplexity conversation for these features "C:\Users\joshb\Documents\hdipComputerScience\Project\Ai conversations\can you make a table vertically scrollable and hor.pdf"
- https://svelte.dev/playground/90ae426e584a44069d9519ed7ac67ac4?version=3.29.4#H4sIAAAAAAAACm2SwW7bMAyGX4XRYd6AOG2D9aLEKdpddu9u8w6KzdhcFcmQ6CSe4XcfZNkJWvQk4vtJ_SSlXhh1RCHFD628hy_wyp1GeCFTkqkkvChPhViKA2n0Qv7uBXdNyA9ALOfq56ZZ-RNqDmyvPH7GC2sYDXshxdYXjhre5SZnjQyqYDohZHBQ2uNMC6utgwwStkfFNpn5uSYOyckYXLFGZnSvjSrIVEGOIPWRSHhsLpskN9u7m7vZ7ltma8AaWWgq3rL-6zfIdtBfW1rEaBjGbn_ZqtK4CLfEynjLIk1hW9IJirDHbK5-giRGCUhIkmEHabrLzS1TRnkuiB4_UWsbHEo6xevrNfjwMFkucgOwV8Vb5WxrynTckYR-PIcNBHlm43qGTUD9u-UMucnFZOVs043j1OvoNRqNYuiyD0HOZyq5lvCwvm8um4hqpKpmCev7G7s1JsHZTum9bnHWrCvRpU6V1HoJD7eqknyjVSfhoHFmf1vPdOjS6dNIKNAwuklVmiqTEuPRf1Cm2cfRJ3SwhlNP_zD0ejVlp4wnJmskrL77kYa98Gp6u_eTrx8_nzJ-zU3OU_32Lu5PLAXjhYVk1-LwZylYkT6TKYUc__jwH0t1KuN6AwAA - style binding for svelte navigation
- https://www.geeksforgeeks.org/html/how-to-create-table-with-100-width-with-vertical-scroll-inside-table-body-in-html/ - For scrollable tables
- https://bulma.io/documentation/elements/table/ - for styling of tables
- https://www.npmjs.com/package/layerchart - attempted to use layerchart for gaps in chart but did not have functionality
- https://svelte-ux.techniq.dev/ - attempted to use svelte UX graphs for the above
- https://frappe.io/charts/docs - Frappe charts configuration
- https://echarts.apache.org/handbook/en/how-to/custom-series - looked at apache e charts for stacked bar with gaps
- https://bryntum.com/blog/creating-a-gantt-chart-with-frappe-gantt/ - Looked into frappe gantt but overengineered for the purpose of this and would have been a workaround
- https://github.com/frappe/gantt also frappe gantt
- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/map - mapping to array for data for frappe chart

15-03-2026

- Added method for deleting lith log and created endpoint for this using holeid. Designed to clear DB before upload of new log. Test method before populating the different logs.
- Added method for deleting lith log and created endpoint for this using holeid. Designed to clear DB before upload of new log. Test method before populating the different logs.
- Got adding form rows to work by creating a button for each form that executes a command to add a record to the initial log states. Realised that the empty forms do not have any way of creating new rows due to their reliance on the previous record added in. Created some logic in the get mineral log by holeid to return null if the response is an empty array. This means that I could put or conditional logic in the inital state to create a dummy record if none already exist. Need to add this for all other log types and clean up UI before working with the backend.
- Made it so that all log service methods return null if the response from the API is an empty array. This allowed an optional record to be inserted if the data was ont returned to the client so that form fields rendered. Created delete methods for each log type which was then included in teh form component as a prop so that each record renders a delete button which removes the delete record from the state. Added a save button to the whole page which is designed to save all logs when hit regardless of the page that we are on. Will need to produce method for making API calls to save these logs.
- Added userid and holeid to the optional initial state for row, to the methods for adding these so that each record added to the form has these by default. Added as hidden fields to forms in case I go down form route for submitting to the API (not button which will be tried first)

- https://www.w3schools.com/html/html_form_input_types.asp for input types
- https://bulma.io/documentation/form/general/ for horizontal form fields
- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/push - refresher for array.push
- https://blog.ignacemaes.com/the-easy-way-to-access-the-last-javascript-array-element/ - for accessing properties of an object at the previous array position while adding to an array
- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/split - refresher string.split
- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/pop - refresher array.pop
- https://medium.com/@altafkhan_24475/delete-method-of-apiview-in-django-rest-framework-c227942776d2 - for deleting items from a table in django
- "C:\Users\joshb\Documents\hdipComputerScience\Project\Ai conversations\If I have a form in svelte, How can I append a new.pdf" - Perplexity conversation covering development session

16-03-2026

- Modified Delete lith log view action to reference query parameter

instead of request body as axios was creating an issue with it.

- edited comment field on logs to be optional.
- Created trial method for updating lith layering in refreshes if required so user is able to save log after access token expired. Requires more testing. Added utility method to append on lithology field before upload as this is required but not by user entry.
- Completed function for uploading lith logs and works as expected. Error

messages do not as axios is not serializing the response from calling

the server action for uploading lith logs. Not ideal but can remain as a

bug to fix for now as not application breaking.

- Perplexity debugging conversation - "C:\Users\joshb\Documents\hdipComputerScience\Project\Ai conversations\will json.stringify work on an array of objects.pdf"
- https://www.w3schools.com/js/js_switch.asp javascript switch statement refresher
- https://bulma.io/documentation/components/message/ - bulma message box for displaying error and success messages. This does not work as anticipated due to the fact that axios is not serializing response from calling the action. Will research more if time but should go on fix list

17-03-2026

- Added views for deleting alteration, structure and mineral logs. Created endpoint URLs for each of these.
- Realised that many calls on refresh functions were then still using the original access token, not the refetched one after refresh. Fixed these server actions where this was a gap. Created setAlterationType and setStructureType utilities for use in the saving of logging.
- All Upload Service functions and logging page server functions complete and working as expected. Realised that Overview page is falling over if only log put in, likely because no data for the others. To address and then this area of the application is completed.
- Fixed issue with overview not rendering. Put in condition that records need to be returned to the server before overview page tab will render.

Fixed issue on testing

- Created reusable table component that could be used to replace all tables in the application later on. Currrently using in logging pages aside from overview. Uses object.keys to get the values of the single drillhole object this is designed to use as prop and iterate over them.
- Completed functionality. This represents the stories for all types of logging issues as all of these were completed in tandem

- "C:\Users\joshb\Documents\hdipComputerScience\Project\Ai conversations\Can I do an each over an object to access its prop.pdf" for some debugging

17-03-2026

- Created a dummy endpoint for checking if access token has expired on

page load in the client. Tested using postman and behaves as expected

- Changed fetch endpoint in layout.server to new dummy method created in

backend.

- Completed feature

18-03-2026

- Added simpleJWT token blacklist to apps and migrated. Created endpoint

to blacklist a refresh token taking the refresh token from request body.

Refresh token is validated using RefreshToken Object then committed to

the blacklist. Created endpoint for this and tested with postman and

works as expected.

- Created service method for blacklisting tokens. Modified logout to

include this before clearing headers and cookies.

- Blacklisting tokens learned from perplexity - "C:\Users\joshb\Documents\hdipComputerScience\Project\Ai conversations\Does django maintain a token blacklist by default_.pdf"

18-03-2026

- NIST 2026 guidelines on password length - 15 characters
- Altered set password view action to check if password length is greater
- than or equal to 15. Backend part configured for this feature.

Changed user form to take a set password prop. If true, will conditionally render weak, moderate or strong passwords to the user. Need to look at the error handling inside the error handling issue which will also encompass this.

19-03-2026

- Created modal component taking required data as props with boolean on

each page for toggling this binding to the parent state. Tested and

appears to work.

19-03-2026

- Created List component that takes data, a subtype and type as props.

Will conditionally render a certain list depending on type and subtype

props. More maintainable as all lists in one place. Less code on view

20-03-2026

- Created searchbar component for pages with lists to add some level of filtering. Attempted to use onchange tied to search state but this did not work with svelte 5 as expected from memory. Researched derived state and implemented this instead. programs to be put in list component derived from filtering data.programs using search state value bound to search box. Result is instant filtering as values are types into the input box.
- Research conversation - "C:\Users\joshb\Documents\hdipComputerScience\Project\Ai conversations\how are interactive searchbars made in websites.pdf"
- https://bulma.io/documentation/form/input/ - bulma table styling

20-03-2026

- Created anchor tag with download property for downloading csv template

from static folder in project.

- https://www.w3schools.com/tags/att_a_download.asp#:~:text=The%20download%20attribute%20specifies%20that,file%20after%20it%20is%20downloaded. - where this was learned

21-03-2026

- Created endpoint for uploading multiple drillholes to support upload of

drillholes by .csv. Tested endpoint using postman and works as expected

- Attempted to create a form action with the file as the uploaded form data Discovered papaparse with perplexity Got form working and was able to extract the file object from this but this failed and returned an empty array after running the csv parse with papaparse with a tostring of the file. Perplexity noted that I was missing a critical component to be able to access the data in the csv and not just the tostring which was to fist extract the arraybuffer from the file using .arraybuffer as an array or raw bytes and then to convert this to a javascript string using a textdecoder. This javascript string is then compatible with papaparse.parse function and allows it to be parsed into an array of objects which could then be manipulated and uploaded to the DB via the associated endpoint. Used this to learn how to process csv data and therefore complete this component. Had to also append enctype to the form containing the file input due to the fact that the default enctype encodes all characters which corrupts the data being posted.
- Papaparse parse function learned about here - https://www.papaparse.com/docs#config
- https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/input/file - using file as an input investigated here
- Further reading on array buffers here -https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/ArrayBuffer
- Info on text decoder here - https://developer.mozilla.org/en-US/docs/Web/API/TextDecoder
- https://www.w3schools.com/tags/att_form_enctype.asp - research on enctype attrobute here
- https://www.geeksforgeeks.org/html/html-enctype-attribute/ - info on using enctype and why here

- "C:\Users\joshb\Documents\hdipComputerScience\Project\Ai conversations\how do you import and serialize .csv files in svel.pdf" - additional research here

21-03-2026

- Easiest to do client side as the data is already loaded and can be used. Uses papa.unparse to convert the array of log objects to a .csv string. Create a new blob object of type csv using the .csv string. Then by using URL.createobjecturl with the blob a URL can be created pointing to this blob. Then create an artificial anchor tag directly on the DOM tree , assign the URL pointing to the blob as the href, assign the download property to make this a download anchor and then click this programatically using the DOM api. Created buttons to access functions for each of the log types. Now when user clicks the button it downloads the log as .csv
- Conversation with perplexity "C:\Users\joshb\Documents\hdipComputerScience\Project\Ai conversations\is it possible to construct files on the fly in sv.pdf" to learn how this works
- https://developer.mozilla.org/en-US/docs/Web/API/Blob - learning about what a blob object is and how it works
- https://developer.mozilla.org/en-US/docs/Web/API/URL/createObjectURL_static - what complete objectURL does and how this works

21-03-2026

- Added two buttons to banner component for linking to logout and

changepassword page. Had to add and additional prop passing in the users

email for the change password link. Adjusted all paged with banner to

pass this email input.

- https://bulma.io/documentation/components/navbar/ - for configuring navbar item position for hero

22-03-2026

- Replaced redirects for errors in server actions across all pages that contain them with thrown errors. Created Error pages for these routes containing an error component that contains the status and message. Added buttons to return user to correct page from error page. Satisfied that this catches the majority of potential errors but final testing may catch more so may require further improvements.
- Learned about handling errors in svelte SSR through - https://svelte.dev/docs/kit/errors
- Learned about bulma styling of messages through - https://bulma.io/documentation/components/message/#docsNav
- Completed feature

22-03-2026

- Added

"svelte/no-navigation-without-resolve": "off" as error not deemed relevant for the purposes of this project

- Advised by linter not to use any type for errors in api routes. Don't think this requires immediate attention so have disabled linter from flagging this error.
- Changed ts config to not error on type any as not required for some parts of the application
- Created endpoints for deleting drill programs and drillholes by their

id. To be used in front end

- Updated missing comments for backend
- Created service methods for deleting drillholes by id and programs by id

to be used later.

- Added delete modals for deleting programs and drillholes to details

pages for both. Added server methods for handling these deletes

- Completed comments
- Decided not to refactor to degree required as project coding time ran out. Can be readdressed later

- Code thoroughly cleaned however

23-03-2026

- Merged sub repos into a monorepo with history for submission - will
- Attempted to use docker init to automatically create docker files, did not work, bit of a rabbit hole. Will have to write these manually

24-03-2026

- Created docker file for drilltekFrontend and drilltekBackend
- Drilltek Frontend - uses node 22 base image - npm run dev as starting command
- Drilltek backend - uses python 3.13 slim as base image, uses requirements.txt to get dependency then install
- Initially db based on postgres 14 base image defined and services for drilltekFrontend and backend created
- Issues encountered during trying to compose :
- Migrations not running by default, had to define a migration service to execute manage.py migrate to apply migrations
- Migrate executing too quickly before DB finished initalising so had to add health check in to only run migrate after db initialised
- Made backend initialisation dependent on migrate completion and frontend dependent on backend initialisation to make sure backend available for API calls before frontend useage
- Realised data wasn't persisting after docker compose down, realised no volume attached so attached volume to db
- Some env files not loading as .env moved to parent directory of python project, therefore had to provide path
- Vite had issues running, had to specify port number explicitly in dev script
- "C:\Users\joshb\Documents\hdipComputerScience\Project\Ai conversations\Does docker compose require docker compose files f.pdf"

9.7. Appendix 7 – Password Hashing with Django Perplexity

I am using Django rest framework to create an API..pdf

9.8. Appendix 8 – Django Rest Framework Perplexity

How can I implement JWT in Django Rest Framework..pdf

9.9. Appendix 9 – Passing email parameter Perplexity

how can I write to local storage in svelte.pdf

9.10. Appendix 10 – Protecting Django routes with JWT

how do I protect routes in svelte.pdf

9.11. Appendix 11 – Refreshing JWT

each block should have a key error svelte.pdf

9.12. Appendix 12 – Adding refresh util to actions

async createDrillProgram(token_string, program_Add.pdf

9.13. Appendix 13 – Django Query Params

what is href without resolve.pdf

9.14. Appendix 14 – Django method confusion

can you pass optional props to a component in svel.pdf

9.15. Appendix 15 – Auto incrementing a string in PostgreSQL

how could i have an autoincrementing string in sql.pdf

9.16. Appendix 16 – Retrieving multiple Django objects

@action(detail=False, methods=['get'])_ def get.pdf

9.17. Appendix 17 – Executing API call before input loaded

export const load_ PageServerLoad = async ({ paren.pdf

9.18. Appendix 18 – Charting considerations

can you make a table vertically scrollable and hor.pdf

9.19. Appendix 19 – Prepopulating form with blank record

If I have a form in svelte, How can I append a new.pdf

9.20. Appendix 20 – Iterating over Object properties

Can I do an each over an object to access its prop.pdf

9.21. Appendix 21 – Additional issues identified during development

Story 27: Tighter error message display

As a user

I should be displayed messages if I try to input incorrect data in a form

So that I know if an action has executed successfully ot if I need to change anything

Assumptions:

Only the logging currently displays any sort of message. This should be present for everything and if the user is forced to logout due to non communication from the api. Explaining why - else the user could be redirected to the login page with a message stating they need to log back in because their credentials have expired

Acceptance critera

Given that I am on any page that requires data to be submitted

And I submit some incorrect or malformed data

Then I should be displayed a message stating why the request could not got be fulfilled

Story 28: Minimum password length

As a user

I should have to set a password with a minimim length

So that my account is more secure from brute force password cracking attempts

Assumptions:

Will need to investigate how this can be controlled in Django and if this can be returned else do the checking in the client and control this client side. Possibly could bind this to display length graphics also. Also need to check most up to date NIST guidance on password length

Acceptance critera

Given that I am changing my password

And I submit a password that is insufficient in length

Then I should be prevented from changing password

And a message should be displayed showing this

Story 29: Blacklist Refresh tokens on logout

As an administrator

I expect that refresh tokens should not be reusable after logout

So that an attacker cannot use a refresh token to obtain access tokens

Assumptions:

Currently refresh tokens could be reused to gain a new access token up to their end of life (7 days). This should not be the case, on logout these should be blacklisted so they cant be used again, limits attack surface.

Acceptance criteria:

Given that a blacklisted token is used to try and obtain a new access token

Then a new access token should not be vended

Story 30: Change refresh endpoint

As a user

When I load a page a bespoke test endpoint in the server should be called

So that I am not loading a high volume of data uneccessarily

Assumptions:

Currently on loading a new page, the get drillprograms endpoint is called to test If a 401 is generated and a new access token needs to be obtained before page load. This could increase latency as its returning potentially a lot of data unneccesarily. This should be changed to a bespoke endpoint that doesn't return anything and is in effect, just a ping of a protected endpoint

Acceptance criteria:

Given that I am loading a new page

And I need to call an endpoint to check if a new access token is required

Then The application should call a bespoke protected endpoint

Story 31: RBAC to sections of site

As a logging user

I should not be able to access the drilling section of the site

So that I do not accidentally edit an area of the site I do not have permission for

Assumptions:

Users are created in the DB by administrator with either a geologist or logger role. The loggers should not be able to access the drilling area, only the logging area. This could

probably be done at a high level in the client, but the login should return the role to be accessed

Acceptance criteria:

Given that I login as a logging user

And I attempt to access the drilling area

Then I should be denied and told why

Story 32: Logout from every page and access change password from any logged in page

As a user

I should be able to change my password and logout from any page

So that the application is more navigable and I can change my password if required

Assumptions: Currently there is only the ability to change password on first login. Should put in logout from any page. Should also have a page to be able to change the password at any time and not rely on the admin to do this

Acceptance criteria

Given that I am on any page

Then I should be able to logout

Or access an area where I can change my password

Story 33:Make modal reusable component for any page it is on.

As a developer

all modals in the site should be constructed from one component to adhere to DRY principles

So that the application adheres to DRY principles

Assumptions: Currently every page that has any kind of modal is repeated every time. This could do with being a reusable component. Analyze the structure of these modals and create a reusable component to replace them

Acceptance criteria

Given that I am on a page that uses a modal

Then this should be called from a component not repeated code

Story 34:Make lists reusable components on any page that requires them

As a developer

all list type sections of the application should be called from a reusable component

So that the application adheres to DRY principles

Assumptions: Currently each page that has list items (drill program, drillholes) are actually constructed on the page. These should be converted into a single reusable component instead to adhere to DRY principles

Acceptance criteria:

Given that I am on a page that has a list

Then this should be called from a component not repeated code

Story 35: Download logs, holes as .csv

As a user

I should be able to download all log types as .csv

So that I could import them into other applications that require csv

Assumptions: This would be best suited coming from the overview page in logging for logs and the drill program page for drillholes, just download a .csv formatted to the user

Acceptance criteria:

Given that I am on the log overview page

Or a drill program page

Then I should be able to hit a button and download the desired data as .csv

Story 36: Refactor log page

As a developer

I should have a refactored log page

So that the code is more manageable and editable

Assumptions : The log page works but is very long with a lot of code. Could probably do with being looked at and refined

Acceptance criteria:

Given that I am updating the log page

Then I should have limited difficulty because this is more manageable

9.22. Appendix 22 – Blacklist JWTs

Does django maintain a token blacklist by default_.pdf

9.23. Appendix 23 – Search bars for filtering

how are interactive searchbars made in websites.pdf

9.24. Appendix 24 – Upload holes via .csv

how do you import and serialize .csv files in svel.pdf

9.25. Appendix 25 – Downloading log .csvs

is it possible to construct files on the fly in sv.pdf

9.26. Appendix 26 – Containerisation for development testing

Does docker compose require docker compose files f.pdf

9.27. Appendix 27 – System relations

User(userId,email,passwordHash,fName,lName,role)

Primary Key userId

DrillProgram(programId,orebody,location,target,totalHoles,totalMeters,userId,datePlanned)

Primary Key programId

Foreign Key userId references User(userId)

Drillhole(holeId, xCoord,yCoord,zCoord,dip,azimuth,type,length,programId)

Primary Key holeId

Foreign Key programId references DrillProgram(programId)

DrillsightSurvey(index,measuredDip,measuredAzi,holeId,userId,timeStamp)

Primary Key index

Foreign Key holeId references Drillhole(holeid)

Foreign Key userId references User(userId)

Survey(index,depth,inc,azi,holeId)

Primary Key index

Foreign Key holeId references Drillhole(holeId)

AlterationLog(index,to,from,comment,alterationCode,alterationType,holeId,dateLogged)

Primary Key index

Foreign Key holeId references Drillhole(holeId)

LithLog(index,to,from,comment,lithCode,lithology,holeId,dateLogged)

Primary Key index

Foreign Key holeId references Drillhole(holeId)

StrucureLog(index,to,from,comment,structureCode,structureType,dip,holeId,dateLogged)

Primary Key index

Foreign Key holeId references Drillhole(holeId)

MineralLog(sampleId,to,from,estimate,zn,pb,ag,fe,comment,sampleType,texture,holeId,dateSampled)

Primary Key sampleId

Foreign Key holeId references Drillhole(holeId)

9.28. Appendix 28 – Glossary

- 3D Modeling - Creating geological models from drillhole data (Leapfrog, MX Deposit)
- Agile - Iterative development using 2-week sprints
- API - Application Programming Interface (frontend-backend communication)
- Blynk - IoT platform for mobile device control
- Bulma - CSS framework for UI styling
- CSV - Comma-Separated Values (drillhole/assays file format)
- Django - Python web framework (DrillTek backend)
- DRF - Django Rest Framework (API endpoints)
- Gitflow - Git branching workflow (main/dev/feature branches)
- IoT - Internet of Things (DrillSight connectivity)
- JWT - JSON Web Token (user authentication)
- Kanban - Visual task workflow system
- PgAdmin4 - PostgreSQL GUI management tool
- PK - Primary Key (database record ID)
- Postman - API testing tool
- PostgreSQL - Open source relational database
- RDBMS - Relational Database Management System
- RPi4 - Raspberry Pi 4 (DrillSight hardware)
- Scrum - Agile framework with sprints/backlogs
- Scrumban - Scrum + Kanban hybrid (Zenhub board)
- SvelteKit - Svelte metaframework (DrillTek frontend)
- SQL - Structured Query Language (PostgreSQL queries)
- SSR - Server-Side Rendering (SvelteKit pages)
- UI/UX - User Interface / User Experience design
- Zenhub - Project management platform (Scrumban boards)

9.29. Appendix 29 – Ethics checklist

Ethics_Report.pdf